

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH

KHOA ĐIỆN ĐIỆN TỬ

BỘ MÔN KỸ THUẬT MÁY TÍNH – VIỄN THÔNG

ĐỒ ÁN TỐT NGHIỆP



HCMUTE

**THIẾT KẾ VÀ TRIỂN KHAI BỘ XỬ LÝ
1-D CNN TRÊN FPGA CHO PHÂN LOẠI
TÍN HIỆU ĐIỆN NÃO EEG**

NGÀNH CÔNG NGHỆ KỸ THUẬT MÁY TÍNH

SINH VIÊN: NGUYỄN TRẦN HÙNG ANH

MSSV: 22119163

NGUYỄN HOÀNG ĐĂNG DUY

MSSV: 22119171

TP. HỒ CHÍ MINH – 06/2026

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH

KHOA ĐIỆN ĐIỆN TỬ

BỘ MÔN KỸ THUẬT MÁY TÍNH – VIỄN THÔNG

ĐỒ ÁN TỐT NGHIỆP



HCMUTE

**THIẾT KẾ VÀ TRIỂN KHAI BỘ XỬ LÝ
1-D CNN TRÊN FPGA CHO PHÂN LOẠI
TÍN HIỆU ĐIỆN NÃO EEG**

NGÀNH CÔNG NGHỆ KỸ THUẬT MÁY TÍNH

SINH VIÊN: NGUYỄN TRẦN HÙNG ANH

MSSV: 22119163

NGUYỄN HOÀNG ĐĂNG DUY

MSSV: 22119171

HƯỚNG DẪN: PGS TS. ĐỖ DUY TÂN

TP. HỒ CHÍ MINH – 06/2026

Tp. HCM, ngày 25 tháng 6 năm 2026

NHIỆM VỤ ĐỒ ÁN TỐT NGHIỆP

Họ và tên sinh viên 1: Nguyễn Trần Hùng Anh MSSV: 22119163

Họ và tên sinh viên 2: Nguyễn Hoàng Đăng Duy MSSV: 22119171

Ngành: CNKT Máy Tính

Giảng viên hướng dẫn: PSG. TS. Đỗ Duy Tân

Ngày nhận đề tài: 01/03/2026.

Ngày nộp đề tài: 25/06/2026.

- Tên đề tài:** Thiết kế và triển khai bộ xử lý 1-D CNN trên FPGA cho phân loại tín hiệu điện não EEG.
- Các số liệu, tài liệu ban đầu:** kiến thức cơ bản về các môn Thiết kế hệ thống và vi mạch tích hợp, Thiết kế mạch tích hợp VLSI, Kỹ thuật số, Học máy và trí tuệ nhân tạo,....
- Nội dung thực hiện đề tài:** Đề tài thực hiện xây dựng hệ thống phân loại tín hiệu EEG sử dụng mô hình 1-D CNN dựa trên ResNet và triển khai trên nền tảng SoC FPGA Kria KV260. Kết quả phân loại được truyền qua WebSocket để hiển thị tín hiệu và nhận dự đoán theo thời gian thực.

TRƯỞNG BỘ MÔN
(Ký & Ghi rõ họ tên)

GIẢNG VIÊN HƯỚNG DẪN
(Ký & Ghi rõ họ tên)

BẢN NHẬN XÉT KHÓA LUẬN TỐT NGHIỆP

(Dành cho giảng viên hướng dẫn)

Đề tài: THIẾT KẾ VÀ TRIỂN KHAI BỘ XỬ LÝ 1-DCNN
TRÊN FPGA CHO PHÂN LOẠI TIẾN HIỆU ĐIỆN NÃO EEG

Sinh viên: + Nguyễn Đoàn Hùng Anh

MSSV: 22.11.91.63

+ Nguyễn Hoàng Đăng Duy

MSSV: 22.11.91.71

Hướng dẫn: PGS.TS. Đỗ Duy Tân

Nhận xét bao gồm các nội dung sau đây:

1. Tính hợp lý trong cách đặt vấn đề và giải quyết vấn đề; ý nghĩa khoa học và thực tiễn:

Đặt vấn đề rõ ràng, mục tiêu cụ thể; đề tài có tính mới, cấp thiết; đề tài có khả năng ứng dụng, tính sáng tạo.

Đặt vấn đề rõ ràng, hợp lý.

2. Phương pháp thực hiện/ phân tích/ thiết kế:

Phương pháp hợp lý và tin cậy dựa trên cơ sở lý thuyết; có phân tích và đánh giá phù hợp; có tính mới và tính sáng tạo.

Hợp lý.

3. Kết quả thực hiện/ phân tích và đánh giá kết quả/ kiểm định thiết kế:

Phù hợp với mục tiêu đề tài; phân tích và đánh giá/ kiểm thử thiết kế hợp lý; có tính sáng tạo/ kiểm định chặt chẽ và đảm bảo độ tin cậy.

Phù hợp.

4. Kết luận và đề xuất:

Kết luận phù hợp với cách đặt vấn đề, đề xuất mang tính cải tiến và thực tiễn; kết luận có đóng góp mới mẻ, đề xuất sáng tạo và thuyết phục.

Kết luận phù hợp.

5. Hình thức trình bày và bố cục báo cáo:

Văn phong nhất quán, bố cục hợp lý, cấu trúc rõ ràng, đúng định dạng mẫu; có tính hấp dẫn, thể hiện năng lực tốt, văn bản trau chuốt.

Hình thức trình bày hợp lý.

6. Kỹ năng chuyên nghiệp và tính sáng tạo:

Thể hiện các kỹ năng giao tiếp, kỹ năng làm việc nhóm, và các kỹ năng chuyên nghiệp khác trong việc thực hiện đề tài.

Phù hợp.

7. Tài liệu trích dẫn

Tính trung thực trong việc trích dẫn tài liệu tham khảo; tính phù hợp của các tài liệu trích dẫn; trích dẫn theo đúng chỉ dẫn APA.

Đúng qui chuẩn.

8. Đánh giá về sự trùng lặp của đề tài

Cần không định đề tài có trùng lặp hay không? Nếu có, đề nghị ghi rõ mức độ, tên đề tài, nơi công bố, năm công bố của đề tài đã công bố.

Có các tiến so với các đề tài tương tự.

9. Những nhược điểm và thiếu sót, những điểm cần được bổ sung và chỉnh sửa*

Chỉnh sửa các lỗi chính tả.

Phân tích rõ hơn các kết quả thảo luận.

10. Nhận xét tinh thần, thái độ học tập, nghiên cứu của sinh viên

Tốt

Đề nghị của giảng viên hướng dẫn

Ghi rõ: "Báo cáo đạt/ không đạt yêu cầu của một khóa luận tốt nghiệp kỹ sư, và được phép/ không được phép bảo vệ khóa luận tốt nghiệp"

Báo cáo đạt yêu cầu của 1 KLTN. Được phép bảo vệ KLTN.

Tp. HCM, ngày 25 tháng 6 năm 2022

Người nhận xét

(Ký và ghi rõ họ tên)

Đỗ Duy Tân

LỜI CẢM ƠN

Để hoàn thành đề án “Thiết kế và triển khai bộ xử lý 1-D CNN trên FPGA cho phân loại tín hiệu điện não EEG”, đầu tiên nhóm thực hiện đề tài xin gửi lời cảm ơn chân thành đến thầy Đỗ Duy Tân đã trực tiếp hướng dẫn trong quá trình chọn cũng như thực hiện đề tài.

Nhóm cũng xin chân thành cảm ơn quý thầy cô khoa Điện – Điện tử đã truyền đạt rất nhiều kiến thức nền tảng và tạo điều kiện thuận lợi để người thực hiện đề tài có thể hoàn thành đề tài này.

Trong quá trình thực hiện đề tài, nhóm cũng đã nhận được sự giúp đỡ và động viên từ phía gia đình, thầy cô, bạn bè. Đây chính là sự động viên to lớn thúc đẩy người thực hiện đề tài cố gắng hoàn thành đề tài. Mặc dù đã có nhiều cố gắng và nỗ lực thực hiện nhưng do kiến thức cũng như khả năng bản thân còn nhiều hạn chế nên trong quá trình thực hiện đề tài không thể tránh khỏi những sai phạm, thiếu sót. Rất mong nhận được lời chỉ dẫn, góp ý quý báu của quý thầy cô.

Xin chân thành cảm ơn!

TP.HCM, Ngày 26 tháng 06 năm 2026

Sinh viên thực hiện

LỜI CAM ĐOAN

Nhóm sinh viên gồm Nguyễn Trần Hùng Anh, Nguyễn Hoàng Đăng Duy thực hiện đề tài “Thiết kế và triển khai bộ xử lý 1-D CNN trên FPGA cho phân loại tín hiệu điện não EEG” dưới sự hướng dẫn của thầy PGS TS. Đỗ Duy Tân xin cam đoan các nội dung như sau:

1. Sản phẩm của Đồ án tốt nghiệp là do nhóm sinh viên thực hiện, không mượn, thuê, mua từ người khác.
2. Quyền báo cáo Đồ án tốt nghiệp là do nhóm sinh viên tự viết, không sao chép hoặc sử dụng bất kỳ tài liệu nào mà không được trích dẫn hoặc không được phép sử dụng.
3. Kết quả thực hiện trong quyền báo cáo bao gồm hình ảnh, độ chính xác của mô hình là hoàn toàn đúng với mô hình, phần cứng nhóm đã thực hiện. Nhóm sinh viên cam đoan các nội dung trên là hoàn toàn chính xác và chịu trách nhiệm hoàn toàn với những cam đoan trên.

TP. Hồ Chí Minh, tháng 06 năm 2026

Sinh viên thực hiện

Tóm tắt

Luận văn thực hiện nghiên cứu, thiết kế và triển khai hệ thống phân loại tín hiệu EEG trên nền tảng SoC FPGA, hướng đến mục tiêu xây dựng một giải pháp xử lý tín hiệu sinh học có khả năng hoạt động trên thiết bị nhúng với tài nguyên phần cứng hạn chế. Trong đề tài, tín hiệu EEG được lấy từ tập dữ liệu Bonn EEG và được xử lý theo từng đoạn tín hiệu có chiều dài cố định trước khi đưa vào mô hình phân loại. Mô hình được sử dụng là mạng nơ-ron tích chập một chiều 1-D CNN dựa trên ý tưởng ResNet, cho phép trích xuất đặc trưng trực tiếp từ tín hiệu EEG dạng một chiều mà không cần chuyển đổi sang ảnh thời gian–tần số hoặc sử dụng các đặc trưng thủ công.

Hệ thống được triển khai theo mô hình đồng thiết kế phần cứng–phần mềm trên nền tảng Kria KV260. Trong đó, phần Processing System đảm nhiệm các tác vụ điều khiển, đọc dữ liệu, tiền xử lý, nạp tham số mô hình, kích hoạt bộ tăng tốc phần cứng, đọc kết quả đầu ra và thực hiện các lớp xử lý cuối như Global Average Pooling và Fully Connected. Phần Programmable Logic được thiết kế để tăng tốc các phép toán chính của mạng CNN, bao gồm tích chập một chiều, ReLU, MaxPooling và phép cộng residual. Việc phân chia chức năng giữa PS và PL giúp tận dụng khả năng điều khiển linh hoạt của phần mềm và khả năng xử lý song song của phần cứng FPGA.

Dữ liệu và trọng số của mô hình được lượng tử hóa sang định dạng fixed-point Q10.6 nhằm giảm độ phức tạp tính toán và phù hợp với kiến trúc phần cứng. Các khối phần cứng được xây dựng theo kiến trúc gồm nhiều Processing Element hoạt động song song, kết hợp với các bộ nhớ cục bộ và bộ điều khiển để điều phối quá trình đọc dữ liệu, đọc trọng số, thực hiện phép tính và ghi kết quả. Ngoài ra, hệ thống còn tích hợp chương trình điều khiển trên Linux và giao tiếp WebSocket để truyền dữ liệu, trạng thái xử lý và kết quả dự đoán đến giao diện hiển thị theo thời gian thực.

Mục lục

Danh mục hình ảnh.....	x
Danh mục bảng.....	xii
Các từ viết tắt.....	xiii
CHƯƠNG 1 GIỚI THIỆU	1
1.1 Giới thiệu	1
1.2 Đóng góp và tính mới của đề tài.....	1
1.3 Một số công trình nghiên cứu liên quan	2
1.4 Mục tiêu nghiên cứu	5
1.5 Giới hạn đề tài.....	5
1.6 Phương pháp nghiên cứu	6
1.7 Bố cục đề tài	7
CHƯƠNG 2 CƠ SỞ LÝ THUYẾT.....	8
2.1 Board SK-KV260-G Kria KV260 Vision AI Starter Kit	8
2.1.1 Giới thiệu	8
2.1.2 Cấu trúc vi điều khiển	10
2.2 Bộ dữ liệu Bonn EEG	15
2.2.1 Giới thiệu	15
2.2.2 Lịch sử phát triển của điện não học	16
2.3 Giao thức AMBA và hệ thống bus AXI4.....	17
2.3.1 Tổng quan về giao thức AMBA.....	17
2.3.2 Phân loại các giao thức trong họ AMBA	18
2.3.3 Tín hiệu và kết nối AXI4 Full	20
2.3.4 Chi tiết tín hiệu của AXI4 Full trong việc đọc và ghi dữ liệu...	29

2.4	Kiến trúc CNN nền tảng	32
2.4.1	Mạng CNN	32
2.4.2	Mạng 1-D CNN.....	33
2.4.3	Mạng ResNet.....	35
2.4.4	Các thành phần sử dụng trong mô hình	37
CHƯƠNG 3 THIẾT KẾ HỆ THỐNG		42
3.1	Yêu cầu hệ thống	42
3.1.1	Yêu cầu về kiến trúc mô hình 1-D CNN.....	42
3.1.2	Yêu cầu về phần cứng	42
3.2	Thiết kế hệ thống	43
3.2.1	Thiết kế tổng quát	43
3.2.2	Mô hình Deep Learning	44
3.2.3	Kiến trúc phần cứng trên FPGA	50
3.2.4	Phần mềm trên FPGA	69
3.2.5	Web hiển thị	84
CHƯƠNG 4 KẾT QUẢ VÀ ĐÁNH GIÁ		88
4.1	Giới thiệu nội dung đánh giá	88
4.2	Môi trường mô phỏng và phương pháp kiểm tra.....	88
4.2.1	Môi trường mô phỏng	88
4.2.2	Phương pháp kiểm tra	88
4.3	Định nghĩa testcase và kết quả mô phỏng	89
4.3.1	Định nghĩa testcase	89
4.3.2	Kết quả chạy mô phỏng	89
4.4	Chạy thực nghiệm trên FPGA	91

4.4.1	Phương pháp đánh giá.....	91
4.4.2	Kết quả đánh giá	91
4.5	Đánh giá tài nguyên phần cứng	93
4.5.1	Kết quả Implementation.....	93
4.5.2	Kết quả Power Analysis	96
4.6	So sánh với các nghiên cứu liên quan.....	98
CHƯƠNG 5 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....		102
5.1	Kết luận.....	102
5.2	Hướng phát triển	103
Tài liệu tham khảo		105

Danh mục hình ảnh

Hình 2.1 Board SK-KV260-G Kria KV260 Vision AI Starter Kit	9
Hình 2.2 Sơ đồ bố trí các cổng kết nối và thành phần phần cứng trên board	10
Hình 2.3 Quá trình ghi dữ liệu trên AMBA AXI4 Full sử dụng ba kênh chính: kênh địa chỉ ghi, kênh dữ liệu ghi, và kênh phản hồi ghi	20
Hình 2.4 Quá trình đọc dữ liệu trên AMBA AXI4 Full sử dụng hai kênh chính: kênh địa chỉ đọc và kênh dữ liệu đọc	21
Hình 2.5 Sơ đồ thời gian chi tiết của tín hiệu chính cho đọc dữ liệu với Burst trên kênh địa chỉ ghi và kênh dữ liệu ghi	29
Hình 2.6 Sơ đồ thời gian chi tiết của tín hiệu chính cho đọc dữ liệu với Burst trên kênh địa chỉ đọc và kênh dữ liệu đọc	30
Hình 2.7 Sơ đồ thời gian chi tiết của tín hiệu chính cho đọc dữ liệu chồng chéo với Burst trên kênh địa chỉ đọc và kênh dữ liệu đọc.....	31
Hình 2.8 Tổng quan về mô hình ResNet [17]	36
Hình 2.9 Tổng quan về khối Residual Block	38
Hình 3.1 Thiết kế tổng quát của hệ thống	43
Hình 3.2 Cấu trúc mô hình Deep Learning	46
Hình 3.3 Biểu đồ sự thay đổi độ chính xác trên tập huấn luyện và đánh giá theo epoch	49
Hình 3.4 Kiến trúc của bộ xử lý trên FPGA	50
Hình 3.5 Sơ đồ khối Controller	52
Hình 3.6 Sơ đồ khối của SBA	57
Hình 3.7 Sơ đồ khối của PE	61
Hình 3.8 Sơ đồ khối của (a) BM và (b) WM	66
Hình 3.9 Sơ đồ khối của CM.....	67

Hình 3.10 Các thông tin chứa trong một dữ liệu context.....	69
Hình 3.11 Lưu đồ giải thuật hàm ws_queue().....	70
Hình 3.12 Lưu đồ giải thuật hàm send_data()	71
Hình 3.13 Lưu đồ giải thuật hàm send_status()	72
Hình 3.14 Lưu đồ giải thuật hàm send_sample().....	73
Hình 3.15 Lưu đồ giải thuật của hàm send_beat_start()	74
Hình 3.16 Lưu đồ giải thuật của hàm send_beat_pred().....	75
Hình 3.17 Lưu đồ giải thuật của hàm send_result().....	76
Hình 3.18 Lưu đồ giải thuật của hàm callback_eeg()	77
Hình 3.19 Lưu đồ giải thuật hàm fx_preprocess()	78
Hình 3.20 Lưu đồ giải thuật hàm fx_to_float().....	79
Hình 3.21 Lưu đồ giải thuật hàm fx_from_float()	80
Hình 3.22 Lưu đồ giải thuật hàm fpga_thread()	82
Hình 3.23 Lưu đồ giải thuật của hàm main()	83
Hình 3.24 Giao diện hiển thị web	87
Hình 4.1 Kết quả sau khi kiểm thử 900 mẫu EEG.....	91
Hình 4.2 Kết quả Utilization Report sau Implementation	94
Hình 4.3 Kết quả Power Report sau Implementation	96

Danh mục bảng

Bảng 1.1 So sánh với các công trình liên quan	4
Bảng 2.1 Tổng hợp cấu trúc vi điều khiển của SK-KV260-G Kria KV260 Vision AI Starter Kit	13
Bảng 2.2 Phân loại tập dữ liệu Bonn EEG	15
Bảng 2.3 Phân lớp cho tập dữ liệu Bonn EEG	16
Bảng 2.4 Đặc điểm các giao thức bus trong chuẩn AMBA	18
Bảng 2.5 Các tín hiệu ở 5 kênh dùng trong AXI Full	21
Bảng 3.1 Tín hiệu input của controller	52
Bảng 3.2 Tín hiệu output của controller	53
Bảng 3.3 Các tín hiệu input của SBA	57
Bảng 3.4 Các tín hiệu output của SBA	58
Bảng 3.5 Tín hiệu input của PE	62
Bảng 3.6 Tín hiệu output của PE	64
Bảng 3.7 Tín hiệu input của WM/BM	66
Bảng 3.8 Tín hiệu input của CM	68
Bảng 4.1 Kết quả sau quá trình kiểm thử	92
Bảng 4.2 Các thông số về thời gian trong quá trình thực thi	92
Bảng 4.3 Tài nguyên FPGA sau Implementation	94
Bảng 4.4 Phân bổ tài nguyên theo từng khối	95
Bảng 4.5 Các thành phần công suất	97
Bảng 4.6 Phân bổ trong Dynamic Power	97
Bảng 4.7 So sánh giữa các nghiên cứu	99

Các từ viết tắt

Từ viết tắt	Nghĩa tiếng Anh	Nghĩa tiếng Việt
1-D CNN	One-Dimensional Convolutional Neural Network	Mạng tích chập 1 chiều
ALU	Arithmetic Logic Unit	Đơn vị số học và logic
AMBA	Advanced Microcontroller Bus Architecture	Kiến trúc bus vi điều khiển nâng cao
ARM	Advanced RISC Machine	Kiến trúc bộ xử lý ARM
AXI4	Advanced eXtensible Interface 4	Chuẩn giao tiếp AXI phiên bản 4
bCNN	Binarized Convolutional Neural Network	Mạng nơ-ron tích chập nhị phân hóa
BCI	Brain Computer Interface	Hệ thống giao diện não – máy tính
BM	Bias Memory	Bộ nhớ độ lệch
CM	Context Memory	Bộ nhớ ngữ cảnh
CNN	Convolutional Neural Network	Mạng nơ-ron tích chập
CNN-LSTM	Convolutional Neural Network – Long Short-Term Memory	Mạng nơ-ron tích chập kết hợp mạng bộ nhớ ngắn hạn dài hạn
CPU	Central Processing Unit	Đơn vị xử lý trung tâm
CWT-CNN	Continuous Wavelet Transform–Convolutional Neural Network	Mô hình mạng nơ-ron tích chập kết hợp Biến đổi Wavelet liên tục

Từ viết tắt	Nghĩa tiếng Anh	Nghĩa tiếng Việt
DDR	Double Data Rate	Bộ nhớ tốc độ dữ liệu kép
DMA	Direct Memory Access	Truy cập bộ nhớ trực tiếp
DRAM	Dynamic Random Access Memory	Bộ nhớ truy cập ngẫu nhiên động
ECG	Electrocardiogram	Điện tâm đồ
EEG	Electroencephalogram	Điện não đồ
EMG	Electromyography	Điện cơ đồ
FF	Flip Flop	Phần tử nhớ lật
FPGA	Field-Programmable Gate Array	Mảng cổng lập trình được theo trường
GAP	Global Average Pooling	Gộp trung bình toàn cục
GPU	Graphics Processing Unit	Đơn vị xử lý đồ họa
HLS	High Level Synthesis	Tổng hợp mức cao
KNN	K-Nearest Neighbors	Thuật toán K láng giềng gần nhất
LDM	Local Data Memory	Bộ nhớ dữ liệu cục bộ
LSU	Load/Store Unit	Đơn vị nạp/lưu dữ liệu
LUT	Look Up Table	Bảng tra cứu
MAC	Multiply-Accumulate	Nhân-Tích lũy
PE	Processing Element	Phần tử xử lý
PEA	Processing Element Array	Mảng các phần tử xử lý

Từ viết tắt	Nghĩa tiếng Anh	Nghĩa tiếng Việt
PL	Programmable Logic	Logic khả trình
PS	Processing System	Hệ thống xử lý
RAM	Random Access Memory	Bộ nhớ truy cập ngẫu nhiên
SBA	Sharing Buffer Allocator	Bộ cấp phát bộ đệm dùng chung
SoC	System on Chip	Hệ thống trên chip
SRAM	Static Random Access Memory	Bộ nhớ truy cập ngẫu nhiên tĩnh
UART	Universal Asynchronous Receiver/Transmitter	Bộ thu phát không đồng bộ đa năng
WHO	World Health Organization	Tổ chức Y tế Thế giới
WM	Weight Memory	Bộ nhớ trọng số

CHƯƠNG 1 GIỚI THIỆU

1.1 Giới thiệu

Các rối loạn thần kinh liên quan đến não bộ — đặc biệt là động kinh, rối loạn giấc ngủ và các bệnh lý nhận thức — đang trở thành gánh nặng y tế toàn cầu ngày càng nghiêm trọng, ảnh hưởng đến hàng trăm triệu người theo thống kê của WHO. Tín hiệu điện não đồ (EEG) đóng vai trò thiết yếu trong chẩn đoán và theo dõi các rối loạn này, đồng thời ngày càng được tích hợp vào các thiết bị đeo tay thông minh và hệ thống giao tiếp não-máy tính (BCI) để giám sát liên tục.

Để tự động hóa phân tích EEG, các mạng nơ-ron tích chập một chiều 1-D CNN nổi lên như giải pháp phù hợp nhờ khả năng xử lý trực tiếp tín hiệu chuỗi thời gian đa kênh mà không cần tiền xử lý phức tạp. Tuy nhiên, do tần số lấy mẫu cao và bản chất phi tuyến phức tạp, khiến khối lượng tính toán tăng đáng kể. Việc triển khai các mô hình này lên thiết bị có tài nguyên hạn chế vẫn là bài toán mở, đặt ra yêu cầu cấp thiết về thiết kế phần cứng hiệu quả.

FPGA nổi lên như nền tảng lý tưởng cho bài toán này nhờ khả năng tái cấu hình linh hoạt, độ trễ thấp và hiệu quả năng lượng vượt trội so với CPU/GPU trong các tác vụ suy luận thời gian thực. Tuy nhiên, các kiến trúc bộ xử lý 1-D CNN hiện tại chưa được tối ưu hóa đặc thù cho tín hiệu EEG đa kênh, đặc biệt trong việc xử lý hiệu quả các tầng tích chập có kích thước kernel lớn và cơ chế quản lý băng thông bộ nhớ khi số lượng kênh đầu vào cao.

Xuất phát từ những hạn chế trên, đề tài "Thiết kế và triển khai bộ xử lý 1-D CNN trên FPGA cho phân loại điện não EEG" được lựa chọn nhằm đề xuất kiến trúc phần cứng chuyên dụng, tối ưu hóa toàn diện về diện tích, băng thông và hiệu suất suy luận cho ứng dụng phân loại tín hiệu EEG thời gian thực trên thiết bị nhúng.

1.2 Đóng góp và tính mới của đề tài

Những đóng góp chính của đề tài được trình bày như sau:

- **Thứ nhất:** Đề xuất mô hình mạng nơ-ron tích chập một chiều 1-D CNN nhẹ lấy cảm hứng từ kiến trúc ResNet, được tối ưu hóa đặc thù cho tín hiệu EEG. Mô hình sử dụng các Residual Block với shortcut connection giúp ổn định quá trình huấn luyện và hạn chế tiêu biến gradient, đồng thời giảm đáng kể số lượng tham số so với các mô hình CNN thông thường, phù hợp với yêu cầu triển khai trên thiết bị phân cứng nhúng.
- **Thứ hai:** Thiết kế bộ xử lý phần cứng cho mô hình 1-D CNN trên FPGA, bao gồm các thành phần chính: khối Controller điều khiển toàn bộ luồng hoạt động của hệ thống; Processing Element (PE) tích hợp bốn bộ nhớ dữ liệu cục bộ (LDM) và một khối xử lý đại số (ALU) để thực hiện các phép tính MAC, ReLU, MaxPooling và cộng residual; Processing Element Array (PEA) kết hợp với Sharing Buffer Allocator (SBA) để phân phối và điều phối dữ liệu đến các PE hoạt động liên tục và đạt hiệu suất song song tối đa; và khối Weight/Bias Memory lưu trữ toàn bộ trọng số và tham số cấu hình của từng lớp mạng phục vụ quá trình tính toán.
- **Thứ ba:** Phân loại tín hiệu EEG thành ba nhóm: người bình thường, người mắc động kinh trong giai đoạn không lên cơn và người đang trong cơn động kinh. Mô hình được huấn luyện với chiến lược giảm tốc độ học, tối ưu hóa bằng Adam, giúp mô hình hội tụ ổn định trên bài toán phân loại ba lớp.
- **Thứ tư:** Tích hợp và kiểm chứng trên board Kria KV260. Hệ thống được đánh giá về độ chính xác phân loại, thời gian suy luận hướng đến ứng dụng thực tế trong các thiết bị theo dõi não bộ đeo được phục vụ chăm sóc sức khỏe thần kinh.

1.3 Một số công trình nghiên cứu liên quan

Trong những năm gần đây, việc triển khai các mô hình học sâu trên nền tảng FPGA đã thu hút sự quan tâm của nhiều nhà nghiên cứu trong lĩnh vực xử lý tín hiệu y sinh. Nhờ khả năng xử lý song song, tính linh hoạt và hiệu quả sử dụng năng lượng, FPGA được xem là một nền tảng phù hợp để hiện thực các mô hình học sâu trên các hệ thống nhúng. Bên cạnh việc nâng cao hiệu năng phân loại,

các nghiên cứu còn tập trung vào việc thiết kế kiến trúc phần cứng và tổ chức bộ nhớ nhằm sử dụng hiệu quả tài nguyên FPGA.

Nghiên cứu [3] đề xuất hệ thống phân loại tín hiệu EEG động kinh dựa trên mô hình hybrid CNN-LSTM được triển khai trên nền tảng Zybo Z7-10 theo phương pháp đồng thiết kế phần cứng-phần mềm. Trong kiến trúc này, các lớp CNN được thực thi trên bộ xử lý ARM Cortex-A9, trong khi các lớp LSTM và fully connected được tăng tốc trên phần logic khả lập trình của FPGA. Kiến trúc quản lý dữ liệu sử dụng AXI Bus kết hợp DRAM để điều phối dữ liệu giữa hai phần xử lý. Mô hình được huấn luyện và kiểm tra trên tập dữ liệu Bonn EEG cho bài toán phân loại ba lớp gồm normal, interictal và ictal. Tuy nhiên, kiến trúc đồng thiết kế phức tạp đòi hỏi sự phân chia tính toán hợp lý giữa ARM và FPGA, làm tăng độ phức tạp trong quá trình thiết kế và triển khai.

Nghiên cứu [4] đề xuất phương pháp thu thập và xử lý tín hiệu EEG kết hợp mô hình CWT-CNN nhẹ được triển khai trên nền tảng PYNQ-Z2. Hệ thống sử dụng phương pháp Continuous Wavelet Transform để tiền xử lý tín hiệu EEG trước khi đưa vào mạng CNN để phân loại. Kiến trúc quản lý dữ liệu sử dụng AXI4 kết hợp DMA và DDR3 để truyền dữ liệu giữa bộ xử lý ARM và lõi IP trên FPGA. Dữ liệu EEG được tự thu thập bằng thiết bị OpenBCI cho bài toán phân loại trạng thái mắt mở và mắt nhắm. Tuy nhiên, mô hình chỉ giải quyết bài toán phân loại nhị phân đơn giản và chưa được đánh giá trên tập dữ liệu chuẩn, do đó khả năng tổng quát hóa của hệ thống trong các bài toán phân loại EEG phức tạp hơn vẫn còn hạn chế.

Nghiên cứu [5] trình bày hệ thống phân loại EEG động kinh theo thời gian thực trên FPGA Zynq XC7Z030 sử dụng phương pháp trích xuất đặc trưng Hjorth Descriptor kết hợp bộ phân loại KNN. Kiến trúc phần cứng được viết bằng VHDL với kiến trúc quản lý dữ liệu sử dụng UART kết hợp RAM để nhận và lưu trữ dữ liệu trong quá trình suy luận. Mô hình được huấn luyện và kiểm tra trên tập dữ liệu Bonn EEG. Tuy nhiên, phương pháp Hjorth Descriptor là phương pháp trích xuất đặc trưng truyền thống, chưa tận dụng được khả năng học đặc

trung tự động của các mô hình học sâu, dẫn đến độ chính xác phân loại thấp hơn so với các phương pháp dựa trên học sâu.

Bên cạnh đó, nghiên cứu [6] đề xuất một kiến trúc 1-D CNN hiệu quả trên FPGA Zynq XC7Z045 sử dụng phương pháp thiết kế mức cao (HLS). Hệ thống xử lý đồng thời ba loại tín hiệu sinh học gồm ECG, EEG và EMG với kiến trúc quản lý dữ liệu sử dụng Off-chip Memory kết hợp On-chip Buffer và PE Array nhằm nâng cao hiệu quả truyền dữ liệu. Mặc dù đạt được hiệu quả tài nguyên cao nhờ sử dụng ít tham số, nghiên cứu chủ yếu tập trung vào việc xây dựng kiến trúc CNN tổng quát cho các ứng dụng y tế và chưa hướng đến bài toán phân loại tín hiệu EEG động kinh một cách chuyên biệt.

Bảng 1.1 trình bày kết quả so sánh về tập dữ liệu, mô hình, kiến trúc quản lý dữ liệu và nền tảng FPGA được sử dụng

Bảng 1.1 So sánh với các công trình liên quan

Tiêu chí	[3]	[4]	[5]	[6]	Đề tài này
Tập dữ liệu	Bonn EEG	EEG (OpenBCI)	Bonn EEG	ECG, EEG, EMG	Bonn EEG
Mô hình	CNN-LSTM	CWT-CNN	Hjorth + KNN	1-D CNN	1-D CNN (dựa trên ResNet)
Kiến trúc quản lý dữ liệu	AXI Bus + DRAM	AXI4 + DMA + DDR3	UART + RAM	Off-chip Memory + On-chip Buffer	SBA +LDM
Triển khai FPGA	Zynq-7000	Zynq-7000	Zynq-7000	Zynq-7000	Zynq UltraScale+

Như được tóm tắt ở Bảng 1.1, đề tài này đề xuất mô hình 1-D CNN dựa trên kiến trúc ResNet được triển khai trên nền tảng Kria KV260, sử dụng kiến trúc quản

lý dữ liệu PEA kết hợp SBA và LDM, nhằm nâng cao độ chính xác phân loại tín hiệu EEG động kinh trên tập dữ liệu Bonn EEG trong khi vẫn đảm bảo hiệu quả sử dụng tài nguyên phần cứng.

1.4 Mục tiêu nghiên cứu

Nhóm hướng đến việc xây dựng một hệ thống phân loại tín hiệu điện não tự động hoàn chỉnh, từ mô hình phần mềm đến triển khai phần cứng thực tế trên FPGA. Cụ thể, đề tài đặt ra các mục tiêu sau:

Nghiên cứu và xây dựng mô hình 1-D CNN theo kiến trúc ResNet với ba giai đoạn xử lý, mỗi giai đoạn kết hợp lớp tích chập giảm chiều và Residual Block, có khả năng phân loại tín hiệu EEG thành ba nhóm: người bình thường, người mắc động kinh trong giai đoạn không lên cơn và người đang trong cơn động kinh. Mô hình được huấn luyện và đánh giá trên tập dữ liệu chuẩn Bonn University với các chỉ số Accuracy, Sensitivity, Specificity và Positive Predictive Value

Về mặt thiết kế kiến trúc phần cứng, thiết kế bộ xử lý cho mô hình 1-D CNN trên FPGA bao gồm các thành phần chính: khối Controller điều khiển toàn bộ luồng hoạt động của hệ thống; Processing Element (PE) tích hợp bốn bộ nhớ dữ liệu cục bộ (LDM) và khối xử lý đại số (ALU); Processing Element Array (PEA) kết hợp với Sharing Buffer Allocator (SBA); và khối Weight/Bias Memory lưu trữ toàn bộ trọng số và tham số cấu hình của từng lớp mạng.

Tích hợp và kiểm chứng toàn bộ hệ thống ở mức SoC trên board AMD Xilinx Kria KV260 Vision AI Starter Kit.

1.5 Giới hạn đề tài

Mặc dù đề tài đã đạt được mục tiêu phân loại tín hiệu EEG và triển khai bộ gia tốc trên nền tảng FPGA, nghiên cứu vẫn tồn tại một số giới hạn nhất định.

Thứ nhất, đề tài chỉ thực hiện đánh giá trên bộ dữ liệu Bonn EEG. Đây là bộ dữ liệu đã được tiền xử lý và có số lượng mẫu tương đối hạn chế so với các cơ sở dữ liệu EEG lâm sàng thực tế. Do đó, khả năng tổng quát hóa của mô hình đối với các

bộ dữ liệu EEG lớn hơn hoặc dữ liệu thu thập trong môi trường thực tế chưa được đánh giá đầy đủ.

Thứ hai, nghiên cứu tập trung vào bài toán phân loại tín hiệu EEG phục vụ phát hiện động kinh và chưa xem xét các ứng dụng khác của EEG như giao diện não – máy tính (BCI), nhận dạng trạng thái cảm xúc hoặc phát hiện rối loạn giấc ngủ.

Thứ ba, việc đánh giá phần cứng chủ yếu tập trung vào các chỉ tiêu như, độ trễ suy luận và độ chính xác. Các tiêu chí khác như độ tin cậy dài hạn, khả năng hoạt động trên thiết bị y tế thực tế hoặc mức tiêu thụ năng lượng trong môi trường triển khai thực tế chưa được khảo sát đầy đủ.

Thứ tư, do giới hạn về thời gian và tài nguyên nghiên cứu, đề tài chưa xây dựng hệ thống thu nhận tín hiệu EEG trực tiếp từ thiết bị phần cứng mà chỉ sử dụng dữ liệu đã được thu thập sẵn từ cơ sở dữ liệu công khai.

1.6 Phương pháp nghiên cứu

Để thực hiện đề tài này, trước tiên nhóm sẽ tiến hành nghiên cứu và tổng hợp các tài liệu, bài báo khoa học liên quan đến đặc trưng của tín hiệu điện não đồ (EEG) và các kiến trúc học sâu phù hợp cho các hệ thống tính toán biên (Edge AI). Công việc này bao gồm phân tích tập dữ liệu EEG của Đại học Bonn [2], tìm hiểu nguyên lý hoạt động của mạng nơ-ron tích chập một chiều (1-D CNN) và cơ chế kết nối tắt (Skip Connection) trong mạng thặng dư (ResNet) nhằm nâng cao khả năng học đặc trưng sâu mà không làm gia tăng đáng kể độ phức tạp tính toán.

Tiếp theo, nhóm sẽ thực hiện giai đoạn tiền xử lý dữ liệu và thiết kế mô hình phân loại. Tập dữ liệu EEG được tổ chức lại thành ba lớp trạng thái sinh lý gồm: trạng thái bình thường, trạng thái giữa các cơn động kinh và trạng thái trong cơn động kinh. Kiến trúc mạng đề xuất được xây dựng dựa trên mô hình ResNet tối giản, bao gồm các khối thặng dư (Residual Blocks), các lớp tích chập một chiều (1D Convolution), các lớp gộp cực đại (Max Pooling) và kết thúc bằng lớp gộp trung bình toàn cục (Global Average Pooling). Cuối cùng, lớp Softmax được sử dụng để thực hiện phân loại tín hiệu EEG thành ba lớp tương ứng.

Đối với việc triển khai phần cứng, nhóm nghiên cứu thiết kế bộ điều khiển (Controller), phần tử xử lý (PE) và mảng phần tử xử lý (PEA) nhằm thực hiện các phép toán tích. Bên cạnh đó, các khối Weight Memory và Bias Memory được xây dựng để lưu trữ các tham số trọng số và độ lệch của mô hình, phục vụ quá trình suy luận trên phần cứng.

Quá trình kiểm chứng hệ thống bao gồm mô phỏng chức năng và đánh giá độ chính xác của thuật toán trên môi trường phần mềm. Sau đó, thiết kế phần cứng sẽ được tổng hợp bằng các công cụ AMD Xilinx Vivado. Cuối cùng, hệ thống được triển khai và đánh giá thực nghiệm trên nền tảng System-on-Module AMD Xilinx Kria KV260 nhằm xác minh tính khả thi của giải pháp đề xuất trong các ứng dụng phát hiện động kinh thời gian thực.

1.7 Bố cục đề tài

Báo cáo được chia thành các chương như sau:

Chương 1: Giới thiệu

Chương 2: Cơ sở lý thuyết

Chương 3: Thiết kế hệ thống

Chương 4: Thực nghiệm đánh giá

Chương 5: Kết luận và hướng phát triển

CHƯƠNG 2 CƠ SỞ LÝ THUYẾT

2.1 Board SK-KV260-G Kria KV260 Vision AI Starter Kit

SK-KV260-G (hay còn gọi là Kria KV260 Vision AI Starter Kit) là một bộ phát triển hệ thống nhúng tích hợp SoC FPGA/CPU mạnh mẽ được thiết kế hướng tới các ứng dụng xử lý thị giác máy (vision AI), trí tuệ nhân tạo nhúng và xử lý tín hiệu thời gian thực. Bộ kit này được phát triển dựa trên module AMD Kria™ K26 System-on-Module (SOM), mang trên mình chip Zynq® UltraScale+™ MPSoC, một SoC không đồng nhất tích hợp vi xử lý ARM và mảng logic FPGA.

Kria KV260 không chỉ là một board FPGA thông thường mà còn là một nền tảng “vision-ready”, hỗ trợ đa dạng các giao tiếp hình ảnh và cảm biến, cùng với khả năng chạy hệ điều hành nhúng (như PetaLinux, Ubuntu) và các framework phát triển phần mềm/phần cứng (như Vitis, Vivado). Vì vậy, đây là một lựa chọn lý tưởng để nghiên cứu, phát triển thuật toán và đánh giá các thiết kế phần cứng tăng tốc, đặc biệt trong các hệ thống cạnh (edge) hoặc thiết bị nhúng tiêu thụ năng lượng thấp.

2.1.1 Giới thiệu

SK-KV260-G Kria KV260 Vision AI Starter Kit là một nền tảng phát triển hệ thống nhúng hiệu năng cao, được thiết kế hướng tới các ứng dụng trí tuệ nhân tạo (AI) và học máy (ML) chạy trực tiếp trên thiết bị biên (edge devices). Board đóng vai trò như một cầu nối giữa nghiên cứu thuật toán và triển khai thực tế, cho phép người dùng phát triển, kiểm thử và đánh giá các hệ thống AI nhúng trên một nền tảng phần cứng hoàn chỉnh.

Để giới thiệu nền tảng phần cứng được sử dụng trong đề tài, Hình 2.1 trình bày board SK-KV260-G Kria KV260 Vision AI Starter Kit.



Hình 2.1 Board SK-KV260-G Kria KV260 Vision AI Starter Kit

Hình 2.1 thể hiện hình ảnh tổng quan của. Điểm cốt lõi của Kria KV260 nằm ở kiến trúc SoC FPGA không đồng nhất, tích hợp đồng thời vi xử lý ARM và mảng logic lập trình được (FPGA) trên cùng một chip. Nhờ đó, hệ thống có khả năng kết hợp linh hoạt giữa xử lý phần mềm và tăng tốc phần cứng: các tác vụ điều khiển, quản lý hệ thống và giao tiếp được thực hiện hiệu quả trên CPU, trong khi các khối tính toán nặng như xử lý tín hiệu số, xử lý ảnh và các mô hình học sâu có thể được ánh xạ xuống FPGA để đạt hiệu năng cao và tiêu thụ năng lượng thấp.

Board được định hướng mạnh mẽ cho các ứng dụng thị giác máy và AI thời gian thực. Việc tích hợp các giao tiếp tốc độ cao như Ethernet, USB 3.0, HDMI/DisplayPort cùng các cổng MIPI CSI dành cho camera giúp KV260 dễ dàng kết nối với cảm biến hình ảnh, thiết bị ngoại vi và hạ tầng mạng. Điều này tạo điều kiện thuận lợi cho việc xây dựng các hệ thống hoàn chỉnh như nhận dạng hình ảnh, phân tích video, theo dõi đối tượng, hoặc phân loại tín hiệu sinh học ngay trên thiết bị.

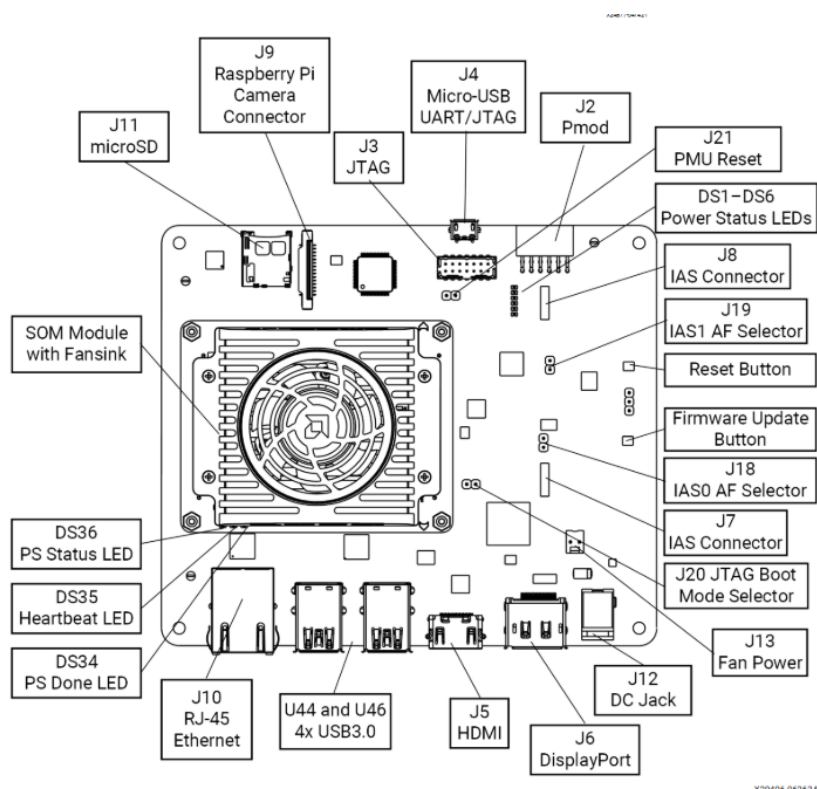
Bên cạnh phần cứng, Kria KV260 còn được hỗ trợ mạnh mẽ về mặt công cụ phát triển. Nền tảng cho phép người dùng tiếp cận đồng thời cả luồng thiết kế phần mềm và phần cứng thông qua các bộ công cụ tiêu chuẩn, từ phát triển ứng dụng chạy trên Linux đến thiết kế các khối xử lý phần cứng trên FPGA. Cách tiếp cận này giúp giảm đáng kể độ phức tạp khi triển khai các hệ thống AI nhúng, đặc biệt đối với những người không chuyên sâu về thiết kế mạch số.

Một ưu điểm quan trọng khác của SK-KV260-G là khả năng mở rộng và định hướng sản phẩm. Board không chỉ phục vụ mục đích học tập và nghiên cứu mà còn được thiết kế theo triết lý gần với sản phẩm thương mại, cho phép đánh giá sớm các yếu tố như hiệu năng, độ trễ, mức tiêu thụ năng lượng và khả năng tích hợp hệ thống. Điều này đặc biệt có ý nghĩa trong các lĩnh vực yêu cầu cao về độ tin cậy và hiệu quả như robot tự hành, IoT công nghiệp, thiết bị y tế thông minh, thị giác máy và các hệ thống AI nhúng thời gian thực.

Nhờ sự kết hợp hài hòa giữa kiến trúc phần cứng linh hoạt, hệ sinh thái phần mềm hoàn chỉnh và định hướng ứng dụng rõ ràng, SK-KV260-G Kria KV260 Vision AI Starter Kit được xem là một nền tảng phát triển toàn diện, phù hợp cho cả nghiên cứu học thuật lẫn phát triển các hệ thống nhúng AI trong thực tế.

2.1.2 Cấu trúc vi điều khiển

Để làm rõ cấu tạo của bo mạch Kria KV260, Hình 2.2 trình bày sơ đồ các cổng kết nối và các thành phần phần cứng chính



Hình 2.2 Sơ đồ bố trí các cổng kết nối và thành phần phần cứng trên board

Hình 2.2 minh họa sơ đồ các cổng kết nối và các thành phần phần cứng chính của bo mạch. Chức năng của từng thành phần được giới thiệu như sau:

- SOM Module with Heatsink: Module Kria SOM tích hợp SoC FPGA UltraScale+ MPSoC (ARM + FPGA). Đây là “bộ não” của board, thực hiện toàn bộ xử lý phần mềm và tăng tốc phần cứng. Khối tản nhiệt giúp đảm bảo hoạt động ổn định khi chạy tác vụ AI nặng.
- J10 – RJ-45 Ethernet: Cổng mạng Ethernet dùng để kết nối mạng LAN, phục vụ truyền dữ liệu, điều khiển từ xa, cập nhật phần mềm và triển khai ứng dụng AI/Edge.
- U44 & U46 – 4× USB 3.0: Cụm cổng USB 3.0 cho phép kết nối các thiết bị ngoại vi như chuột, bàn phím, USB lưu trữ, camera USB hoặc các module AI ngoại vi.
- J5 – HDMI: Cổng xuất hình HDMI, dùng để hiển thị giao diện, kết quả xử lý ảnh/AI hoặc debug trực quan.
- J6 – DisplayPort: Cổng xuất hình DisplayPort, cung cấp băng thông cao cho các ứng dụng đồ họa hoặc hiển thị độ phân giải lớn.
- J9 – Raspberry Pi Camera Connector: Cổng kết nối camera chuẩn CSI (tương thích camera Raspberry Pi), thường dùng cho các ứng dụng thị giác máy, AI Vision và xử lý ảnh thời gian thực.
- J8 & J7 – IAS Connector: Các cổng IAS (Image Acquisition Subsystem) dùng để kết nối module camera chuyên dụng, phục vụ các ứng dụng thị giác công nghiệp và AI Vision.
- J18 – IAS0 AF Selector: Công tắc chọn chế độ auto-focus cho camera kết nối qua IAS0.
- J19 – IAS1 AF Selector: Công tắc chọn chế độ auto-focus cho camera kết nối qua IAS1.
- J11 – microSD: Khe cắm thẻ microSD, dùng để lưu hệ điều hành (Linux), ứng dụng, dữ liệu và mô hình AI.

- J4 – Micro-USB UART/JTAG: Cổng micro-USB dùng cho giao tiếp UART (console debug) và JTAG, phục vụ nạp chương trình, debug phần cứng và phần mềm.
- J3 – JTAG: Cổng JTAG chuyên dụng để debug SoC FPGA, nạp bitstream và kiểm tra trạng thái phần cứng mức thấp.
- J20 – JTAG Boot Mode Selector: Công tắc chọn chế độ boot của hệ thống (JTAG, QSPI, SD...), rất quan trọng trong quá trình phát triển và debug.
- J2 – Pmod: Cổng mở rộng Pmod, cho phép kết nối các module ngoại vi như cảm biến, màn hình nhỏ, ADC/DAC, GPIO mở rộng.
- J12 – DC Jack: Cổng cấp nguồn chính cho board (nguồn DC ngoài).
- J13 – Fan Power: Cổng cấp nguồn cho quạt tản nhiệt, đảm bảo nhiệt độ ổn định khi board hoạt động liên tục.
- DS1 – DS6 Power Status LEDs: Các LED hiển thị trạng thái nguồn, giúp kiểm tra nhanh tình trạng cấp điện của board.
- DS34 – PS Done LED: LED báo trạng thái khởi tạo hoàn tất của Processing System (PS).
- DS35 – Heartbeat LED: LED nhấp nháy theo chu kỳ, cho biết hệ thống đang hoạt động bình thường.
- DS36 – PS Status LED: LED hiển thị trạng thái hoạt động của khối PS.
- Reset Button: Nút reset hệ thống, dùng để khởi động lại board trong quá trình thử nghiệm.
- Firmware Update Button: Nút kích hoạt chế độ cập nhật firmware, hỗ trợ nạp lại hệ thống khi cần.
- J21 – PMU Reset: Nút reset Power Management Unit (PMU), dùng khi gặp sự cố nguồn hoặc khởi động.

Bảng 2.1 trình bày việc phân nhóm các thành phần phần cứng của bo mạch Kria KV260 thành các khối chức năng dựa trên vai trò và chức năng tương đồng.

Bảng 2.1 Tổng hợp cấu trúc vi điều khiển của SK-KV260-G Kria KV260

Vision AI Starter Kit

Khối chức năng	Thành phần	Vai trò & chức năng trong hệ thống
Kiến trúc tổng thể	SoC FPGA không đồng nhất (ARM + FPGA trên cùng chip)	Kết hợp xử lý phần mềm linh hoạt và tăng tốc phần cứng song song trên cùng một nền tảng
Processing System (PS)	ARM Cortex-A53 (64-bit)	Chạy hệ điều hành Linux, quản lý ứng dụng, giao tiếp mạng, điều phối hệ thống
	ARM Cortex-R5	Xử lý thời gian thực, điều khiển mức thấp, đáp ứng độ trễ nhỏ
	Chức năng PS	Boot hệ thống, quản lý bộ nhớ & ngoại vi, cấu hình và điều khiển FPGA
Programmable Logic (PL)	FPGA UltraScale+	Thực hiện tăng tốc phần cứng, xử lý song song
	LUT / FF	Thực hiện logic điều khiển
	DSP Slices	Phép nhân–cộng (MAC) tốc độ cao cho CNN, xử lý tín hiệu

	BM / URAM	Lưu trữ dữ liệu cục bộ, dữ liệu trung gian
	Vai trò PL	Giảm tải CPU, tăng tốc CNN, xử lý tín hiệu & hình ảnh
Bộ nhớ hệ thống	DDR4	Bộ nhớ chính cho PS và PL, lưu dữ liệu lớn và mô hình học sâu
	QSPI Flash	Lưu bootloader, firmware
	microSD	Lưu hệ điều hành, dữ liệu ứng dụng
Giao tiếp PS–PL	AXI-GP	Truyền tín hiệu điều khiển giữa CPU và FPGA
	AXI-HP	Truyền dữ liệu tốc độ cao, phục vụ tăng tốc phần cứng
Ngoại vi và giao tiếp ngoài	USB 3.0	Kết nối thiết bị ngoại vi
	Ethernet	Truyền dữ liệu mạng
	HDMI / DisplayPort	Xuất hình ảnh
	MIPI CSI	Kết nối camera
	UART/JTAG	Debug, nạp chương trình
Nguồn và tản nhiệt	Hệ thống cấp nguồn đa mức	Cấp điện ổn định cho PS và PL

	Quạt tản nhiệt chủ động	Duy trì nhiệt độ khi xử lý tác vụ nặng
--	-------------------------	--

Như được trình bày trong Bảng 2.1, các thành phần phần cứng của bo mạch được tổ chức thành các khối chức năng như khối xử lý, khối bộ nhớ, khối giao tiếp và khối mở rộng. Cách phân nhóm này giúp làm rõ vai trò của từng khối trong kiến trúc hệ thống, đồng thời tạo thuận lợi cho việc mô tả và phân tích các thành phần ở các mục tiếp theo.

2.2 Bộ dữ liệu Bonn EEG

2.2.1 Giới thiệu

Đề tài sử dụng tập dữ liệu điện não đồ (EEG) chuẩn mực do khoa Động kinh học, Đại học Bonn (Đức) công bố [12]. Đây là một trong những bộ dữ liệu Benchmark (điểm chuẩn) được sử dụng phổ biến nhất trên toàn cầu để nghiên cứu, phát triển và đánh giá các thuật toán phát hiện, phân loại bệnh lý động kinh.

Về mặt kỹ thuật, tập dữ liệu gốc bao gồm 500 bản ghi tín hiệu EEG kênh đơn (single-channel. Mỗi bản ghi có thời lượng chính xác là 23,6 giây, tần số lấy mẫu (sampling rate) là 173,61 Hz [12], tương đương với 4097 điểm dữ liệu (data points) cho mỗi bản ghi. Dữ liệu đã được lọc thông dải (band-pass filter) trong khoảng 0,53 Hz đến 40 Hz để loại bỏ các nhiễu tần số thấp và cao không cần thiết [12].

Bảng 2.2 tổng hợp bối cảnh thu nhận mẫu và đặc điểm lâm sàng của năm tập con trong bộ dữ liệu Bonn EEG trước khi được đưa vào hệ thống phân loại.

Bảng 2.2 Phân loại tập dữ liệu Bonn EEG

Tập dữ liệu	Bối tượng lấy mẫu	Trạng thái
Z	Người khỏe mạnh	Bình thường mắt mở
O	Người khỏe mạnh	Bình thường mắt đóng
N	Bệnh nhân động kinh, giữa các cơn động kinh, ngoài vùng não khởi phát cơn động kinh	Không trong trạng thái động kinh

F	Bệnh nhân động kinh, giữa các cơn động kinh, trong vùng não khởi phát cơn động kinh	Không trong trạng thái động kinh
S	Bệnh nhân động kinh, trong cơn động kinh	Trong trạng thái động kinh

Theo Bảng 2.2, năm tập con được thu nhận từ các đối tượng và trạng thái sinh lý khác nhau, bao gồm mở mắt, nhắm mắt, không xuất hiện cơn động kinh và trong cơn động kinh.

Để xây dựng bài toán phân loại ba lớp, các tập con của bộ dữ liệu Bonn EEG được tổ chức lại theo mức độ tương đồng về trạng thái hoạt động của não. Kết quả phân nhóm được trình bày trong Bảng 2.3.

Bảng 2.3 Phân lớp cho tập dữ liệu Bonn EEG

Tập dữ liệu	Phân lớp	Trạng thái
Z, O	0	Bình thường
N, F	1	Giữa cơn động kinh
S	2	Trong trạng thái động kinh

Theo cách phân nhóm trong Bảng 2.3, các bản ghi EEG của người khỏe mạnh ở cả hai trạng thái mở mắt và nhắm mắt được gộp thành **Lớp 0 (Bình thường)**. Các tín hiệu thu nhận từ bệnh nhân động kinh trong giai đoạn giữa cơn được xếp vào **Lớp 1 (Giữa cơn động kinh)**, trong khi các mẫu ghi nhận trong thời gian xảy ra cơn được phân vào **Lớp 2 (Trong cơn động kinh)**.

2.2.2 Lịch sử phát triển của điện não học

Những nỗ lực nghiên cứu điện hoạt động của não khởi đầu từ thế kỷ thứ mười chín, với những đóng góp quan trọng từ nhiều nhà khoa học khác nhau Brazier (1961). Richard Caton (1842–1926), một bác sĩ người Anh, được công nhận là tiên phong trong lĩnh vực này khi công bố năm 1875 rằng dòng điện trong não thay đổi tùy theo vị trí điện cực. Sau đó, Adolf Beck (1863–1939) mở rộng hướng nghiên

cứu bằng cách khám phá các dao động điện có nhịp điệu từ não động vật vào năm 1890, đồng thời quan sát hiện tượng "alpha blocking" khi ánh sáng kích thích mắt. Napoleon Cybulski (1854–1919) cải tiến phương pháp ghi hình bằng kết hợp galvanometer với camera để ghi ảnh tín hiệu điện não Brazier (1961).

Bước tiến lớn tiếp theo đến từ Hans Berger, người đầu tiên ghi nhận được tín hiệu điện não từ da đầu người vào năm 1924 và công bố năm 1929, mở ra kỷ nguyên mới của EEG ứng dụng lâm sàng. Đến thập niên 1930, Frederic Gibbs và Hallowell Davis phát hiện mối liên hệ giữa các phức hợp spike-wave 3 Hz với cơn vắng ý thức ở bệnh nhân động kinh. Cùng khoảng thời gian, Alfred Lee Loomis tiên phong trong việc phân loại các giai đoạn giấc ngủ dựa trên EEG vào năm 1935, đặt nền tảng cho ngành học giấc ngủ hiện đại Brazier (1961).

2.3 Giao thức AMBA và hệ thống bus AXI4

2.3.1 Tổng quan về giao thức AMBA

Trong bối cảnh thiết kế hệ thống trên chip (System-on-Chip – SoC) ngày càng phức tạp, việc chuẩn hóa giao thức giao tiếp giữa các khối chức năng trở thành yêu cầu thiết yếu. AMBA ra đời nhằm đáp ứng nhu cầu đó. AMBA là một đặc tả giao thức kết nối trên chip (on-chip interconnect) được ARM phát triển và công bố lần đầu vào năm 1996 dưới dạng tiêu chuẩn mở, không tính phí bản quyền [1].

Kể từ khi ra mắt, AMBA đã trải qua nhiều thế hệ phát triển, mở rộng không ngừng về tính năng và phạm vi ứng dụng. Tính đến nay, AMBA đã ở thế hệ thứ 5 với nhiều giao thức khác nhau phục vụ cho các tầng kết nối khác nhau trong hệ thống SoC. Các phiên bản chính bao gồm:

- AMBA 1.0 (1996): Giới thiệu hai giao thức nền tảng là ASB (Advanced System Bus) và APB (Advanced Peripheral Bus).
- AMBA 2.0 (1999): Bổ sung AHB (Advanced High-performance Bus), giao thức hiệu năng cao dùng cho kết nối các lõi xử lý với bộ nhớ và thiết bị ngoại vi tốc độ cao.

- AMBA 3.0 (2003): Giới thiệu AXI (Advanced eXtensible Interface), đánh dấu bước ngoặt trong thiết kế bus hiệu năng cao với kiến trúc đa kênh độc lập.
- AMBA 4.0 (2010): Phát triển AXI4 với các cải tiến quan trọng, đồng thời bổ sung ACE (AXI Coherency Extensions) hỗ trợ nhất quán bộ nhớ đệm trong hệ thống đa lõi.
- AMBA 5.0 (2013): Giới thiệu CHI (Coherent Hub Interface) cho các hệ thống đa xử lý quy mô lớn và cập nhật AHB5, AXI5 với hỗ trợ bảo mật TrustZone.

Nhờ tính chất mở và chi phí bản quyền bằng không, AMBA nhanh chóng trở thành tiêu chuẩn thực tế (de facto standard) trong ngành thiết kế vi mạch. Phần lớn các hãng thiết kế IP lớn như ARM, Synopsys, Cadence đều cung cấp sản phẩm tương thích với AMBA, tạo ra một hệ sinh thái IP phong phú và có thể tái sử dụng.

2.3.2 Phân loại các giao thức trong họ AMBA

Trong hệ thống SoC hiện đại, các khối chức năng có yêu cầu truyền thông rất khác nhau: từ các thanh ghi cấu hình tốc độ thấp cho đến các luồng dữ liệu băng thông cao.

Bảng 2.4 tổng hợp các giao thức bus thuộc chuẩn AMBA sử dụng phổ biến trong thiết kế hệ thống trên chip (SoC).

Bảng 2.4 Đặc điểm các giao thức bus trong chuẩn AMBA

Giao thức	Phiên bản	Đặc điểm chính	Ứng dụng điển hình	Băng thông
APB	AMBA 2-4	Bus đơn giản, không pipeline, 2 chu kỳ/giao dịch	Timer, UART, GPIO, SPI	Thấp
AHB	AMBA 2-5	Pipeline, hỗ trợ Burst, đa Master/Slave	Bộ điều khiển DMA, SRAM nội	Trung bình

AXI3	AMBA 3	5 kênh độc lập, giao dịch không đồng bộ	IP cores hiệu năng cao	Cao
AXI4 Full	AMBA 4-5	Burst 256 beat, nhiều giao dịch đồng thời, out-of-order	DDR, CNN accelerator, DMA	Rất cao
AXI4-Lite	AMBA 4-5	Đơn giản hóa AXI4, không Burst, 1 giao dịch/lần	Thanh ghi cấu hình IP	Thấp-TB
AXI4-Stream	AMBA 4-5	Không địa chỉ, truyền một chiều liên tục	Video, audio, neural network	Rất cao

Bảng 2.4 trình bày tổng quan các giao thức bus thuộc chuẩn AMBA (Advanced Microcontroller Bus Architecture) được sử dụng phổ biến trong thiết kế SoC. Chuẩn AMBA bao gồm nhiều phiên bản từ AMBA 2 đến AMBA 5, với các giao thức được thiết kế cho từng mục đích sử dụng khác nhau [11].

Ở mức băng thông thấp, giao thức APB (Advanced Peripheral Bus) được sử dụng cho các ngoại vi đơn giản như Timer, UART và GPIO nhờ cấu trúc không pipeline và chỉ cần 2 chu kỳ mỗi giao dịch. Ở mức cao hơn, AHB (Advanced High-performance Bus) hỗ trợ pipeline và cơ chế Burst, phù hợp cho các bộ điều khiển DMA và SRAM nội với băng thông trung bình [11].

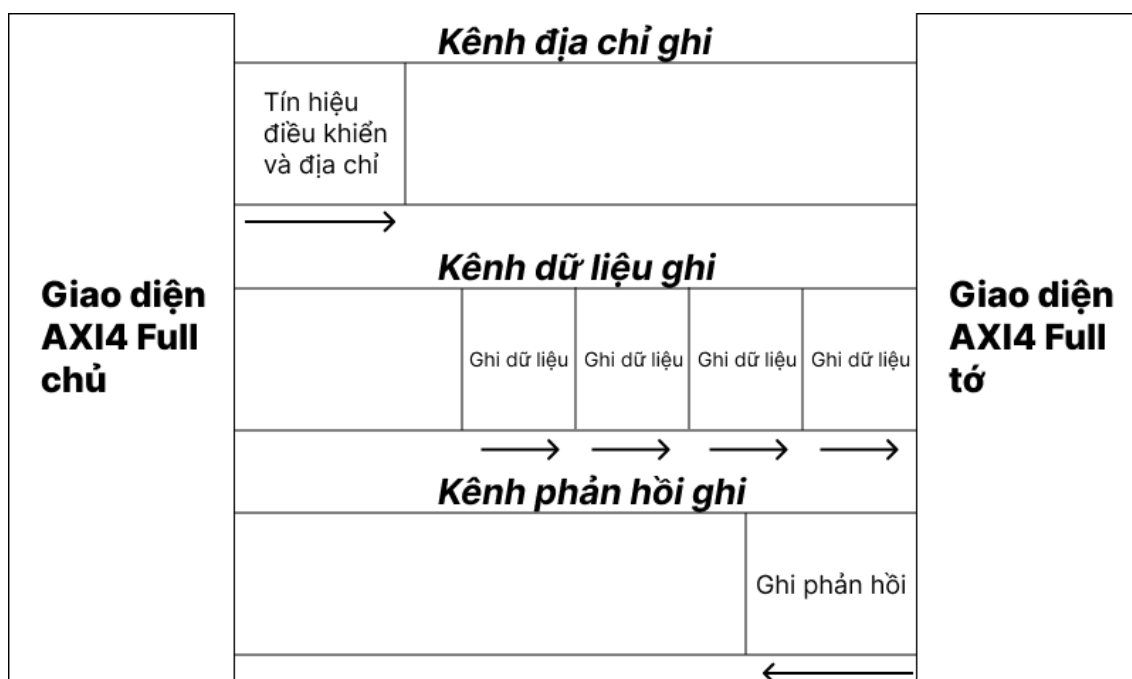
Đối với các ứng dụng yêu cầu hiệu năng cao, giao thức AXI (Advanced eXtensible Interface) nổi bật với 3 biến thể: AXI3 hỗ trợ 5 kênh độc lập với giao dịch không đồng bộ; AXI4 Full mở rộng lên Burst 256 beat, hỗ trợ nhiều giao dịch đồng thời và out-of-order, phù hợp cho DDR và CNN accelerator; AXI4-Lite là phiên bản đơn giản hóa không hỗ trợ Burst, dùng cho thanh ghi cấu hình IP. Ngoài

ra, AXI4-Stream được thiết kế riêng cho truyền dữ liệu một chiều liên tục như video, audio và mạng nơ-ron, đạt băng thông rất cao nhờ không cần địa chỉ [11].

2.3.3 Tín hiệu và kết nối AXI4 Full

Trong mục này, nội dung tập trung trình bày các tín hiệu và cách thức kết nối của giao thức AXI4-Full trong kiến trúc AMBA. AXI4-Full được thiết kế để hỗ trợ truyền dữ liệu với mức độ phức tạp cao, cho phép xử lý đồng thời nhiều giao dịch nhằm nâng cao băng thông và hiệu năng khi liên kết các khối IP trên FPGA. Giao thức này tổ chức việc truyền và nhận dữ liệu thông qua năm kênh riêng biệt, bao gồm: kênh địa chỉ ghi, kênh dữ liệu ghi, kênh phản hồi ghi, kênh địa chỉ đọc và kênh dữ liệu đọc.

Hình 2.3 minh họa quy trình ghi dữ liệu trong giao thức AXI4 Full.

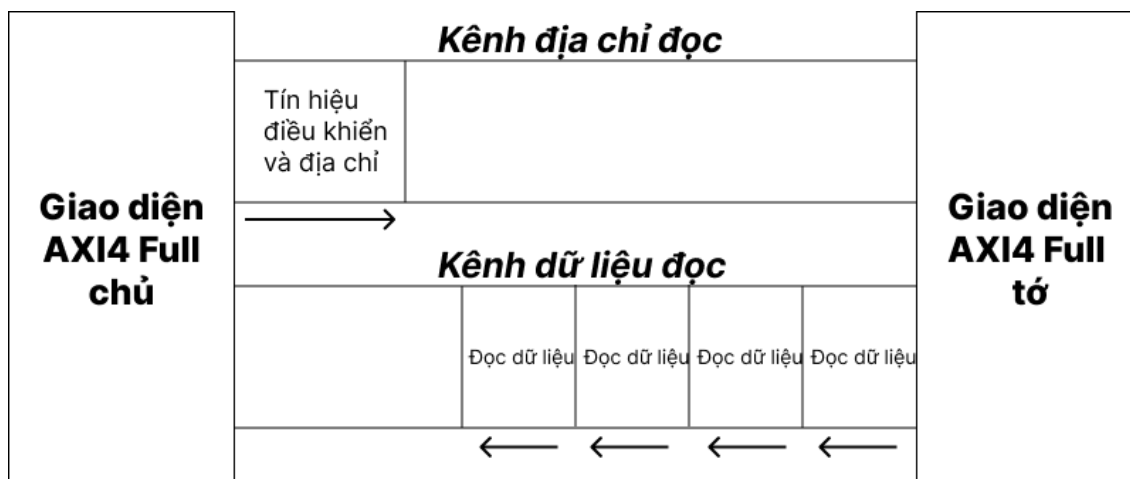


Hình 2.3 Quá trình ghi dữ liệu trên AMBA AXI4 Full sử dụng ba kênh chính: kênh địa chỉ ghi, kênh dữ liệu ghi, và kênh phản hồi ghi

Hình 2.3 trình bày chi tiết quá trình ghi dữ liệu trong giao thức AMBA AXI4, trong đó sử dụng ba kênh chính gồm: kênh địa chỉ ghi, kênh dữ liệu ghi và kênh phản hồi ghi, nhằm đảm bảo tính đồng bộ cũng như nâng cao hiệu năng hệ thống. Trong giai đoạn đầu, thiết bị chủ truyền thông tin điều khiển và địa chỉ ghi thông qua kênh địa chỉ ghi để xác định chính xác vị trí lưu trữ dữ liệu. Tiếp theo, dữ liệu

cần ghi được gửi từ chủ đến tớ qua kênh dữ liệu ghi. Sau khi hoàn tất việc ghi, thiết bị tớ sẽ phản hồi trạng thái của giao dịch thông qua kênh phản hồi ghi, cho phép chủ kiểm tra kết quả và tiếp tục các thao tác xử lý tiếp theo.

Hình 2.4 minh họa quy trình đọc dữ liệu trong giao thức AMBA AXI4.



Hình 2.4 Quá trình đọc dữ liệu trên AMBA AXI4 Full sử dụng hai kênh chính: kênh địa chỉ đọc và kênh dữ liệu đọc

Hình 2.4 minh họa quá trình đọc dữ liệu trong giao thức AXI4, trong đó sử dụng hai kênh chính là kênh địa chỉ đọc và kênh dữ liệu đọc. Ở bước đầu, thiết bị chủ truyền thông tin điều khiển cùng địa chỉ cần truy cập thông qua kênh địa chỉ đọc để xác định vị trí dữ liệu. Sau đó, thiết bị tớ sẽ gửi dữ liệu tương ứng về chủ thông qua kênh dữ liệu đọc. Nhờ cấu trúc kênh độc lập, quá trình đọc có thể diễn ra song song với các giao dịch khác, góp phần nâng cao hiệu suất và giảm độ trễ của hệ thống. Việc tách biệt các kênh trong AXI4-Full giúp cải thiện khả năng kiểm soát luồng dữ liệu và tối ưu hóa quá trình truyền tải, đặc biệt hiệu quả trong các hệ thống FPGA tích hợp nhiều khối IP có độ phức tạp cao.

Bảng 2.5 tổng hợp các tín hiệu được sử dụng trong năm kênh của giao thức AXI4 Full.

Bảng 2.5 Các tín hiệu ở 5 kênh dùng trong AXI Full

Tín hiệu	Chiều	Vai trò
ACLK	Nguồn xung nhịp→	Tín hiệu xung nhịp chung

		tắt cả các khối AXI	
	ARESETn	Nguồn reset → tắt cả các khối AXI	Tín hiệu reset mức thấp chung
Các tín hiệu kênh địa chỉ ghi	AWID	Master→Slave	ID địa chỉ ghi. Tín hiệu này là thẻ nhận dạng cho nhóm tín hiệu địa chỉ ghi.
	AWADDR	Master→Slave	Địa chỉ ghi. Địa chỉ ghi cung cấp địa chỉ của lần truyền đầu tiên trong một giao dịch Burst.
	AWLEN	Master→Slave	Độ dài Burst. Độ dài Burst cung cấp số lượng truyền chính xác trong một Burst. Thông tin này xác định số lần truyền dữ liệu liên quan đến địa chỉ.
	AWSIZE	Master→Slave	Kích thước Burst. Tín hiệu này chỉ ra kích thước của mỗi lần truyền trong
	AWBURST	Master→Slave	Loại Burst. Loại Burst và thông tin kích thước xác định cách tính địa chỉ cho mỗi lần truyền trong Burst.

	AWLOCK	Master→Slave	Loại khóa. Cung cấp thông tin bổ sung về đặc tính của lần truyền.
	AWCACHE	Master→Slave	Loại bộ nhớ. Tín hiệu này chỉ ra cách các giao dịch cần tiến hành qua hệ thống.
	AWPROT	Master→Slave	Loại bảo vệ. Tín hiệu này chỉ ra mức độ quyền và an ninh của giao dịch, và liệu giao dịch đó là truy cập dữ liệu hay lệnh.
	AWQOS	Master→Slave	Chất lượng dịch vụ (QoS). Tập định danh QoS cho mỗi lần truyền ghi. Chỉ được triển khai trong AXI4.
	AWREGION	Master→Slave	Định danh vùng. Cho phép một giao diện vật lý đơn trên tổ được sử dụng cho nhiều giao diện logic.
	AWUSER	Master→Slave	Tín hiệu người dùng. Tín hiệu do người dùng tùy chọn trong kênh địa chỉ ghi. Chỉ hỗ trợ trong AXI4.
	AWVALID	Master→Slave	Địa chỉ ghi hợp lệ. Tín hiệu này chỉ ra rằng kênh

			đang báo hiệu địa chỉ ghi hợp lệ và thông tin điều khiển.
	AWREADY	Slave→Master	Địa chỉ ghi sẵn sàng. Tín hiệu này chỉ ra rằng tớ đã sẵn sàng nhận địa chỉ ghi và thông tin điều khiển liên quan.
sdc tín hiệu kênh dữ liệu ghi	WID	Master→Slave	ID ghi. Tín hiệu này là ID của việc truyền dữ liệu ghi. Chỉ hỗ trợ AXI3.
	WDATA	Master→Slave	Dữ liệu ghi. Dữ liệu có thể điều chỉnh tùy thuộc vào yêu cầu dữ liệu của IP với độ lớn thường là 32, 64, 128, và 256 bit.
	WSTRB	Master→Slave	Strobes ghi. Tín hiệu này chỉ ra các byte nào chứa dữ liệu hợp lệ. Mỗi 8-bit của bus dữ liệu ghi có một bit strobe ghi.
	WLAST	Master→Slave	Kết thúc ghi. Tín hiệu này chỉ ra lần truyền cuối cùng trong một chuỗi ghi.
	WUSER	Master→Slave	Tín hiệu người dùng. Tín hiệu tùy chọn do người dùng định nghĩa trong kênh dữ liệu ghi.

	WVALID	Master→Slave	Ghi hợp lệ. Tín hiệu này chỉ ra rằng dữ liệu ghi hợp lệ và strobes có sẵn.
	WREADY	Slave→Master	Sẵn sàng ghi. Tín hiệu này chỉ ra rằng tớ có thể chấp nhận dữ liệu ghi.
Các tín hiệu kênh phản hồi	BID	Slave→Master	ID phản hồi. Tín hiệu này là thẻ ID của phản hồi ghi.
	BRESP	Slave→Master	Phản hồi ghi. Tín hiệu này chỉ ra trạng thái của giao dịch ghi.
	BUSER	Slave→Master	Tín hiệu người dùng. Tín hiệu tùy chọn do người dùng định nghĩa trong kênh phản hồi ghi. Chỉ hỗ trợ trong AXI4.
	BVALID	Slave→Master	Phản hồi ghi hợp lệ. Tín hiệu này chỉ ra rằng kênh đang báo hiệu một phản hồi ghi hợp lệ.
	BREADY	Master→Slave	Phản hồi sẵn sàng. Tín hiệu chỉ ra rằng chủ có thể chấp nhận phản hồi ghi.
Các	ARID	Master→Slave	ID địa chỉ đọc. Tín hiệu này là thẻ nhận dạng cho nhóm tín hiệu địa chỉ đọc.

ARADDR	Master→Slave	Địa chỉ đọc. Tín hiệu địa chỉ đọc cung cấp địa chỉ của lần chuyển đầu tiên trong một giao dịch đọc dạng Burst.
ARLEN	Master→Slave	Độ dài Burst. Tín hiệu này chỉ ra số lượng chuyển chính xác trong một Burst.
ARSIZE	Master→Slave	Kích thước Burst. Tín hiệu chỉ ra kích thước của mỗi lần chuyển trong Burst.
ARBURST	Master→Slave	Loại Burst. Loại Burst và thông tin kích thước xác định cách thức địa chỉ cho mỗi lần chuyển trong Burst được tính toán.
ARLOCK	Master→Slave	Loại khóa. Tín hiệu này cung cấp thông tin bổ sung về đặc điểm của lần chuyển.
ARCACHE	Master→Slave	Loại bộ nhớ. Tín hiệu này chỉ ra cách thức các giao dịch cần phải tiến hành trong hệ thống.
ARPROT	Master→Slave	Loại bảo vệ. Tín hiệu này chỉ ra mức độ đặc quyền

			và an ninh của giao dịch, và liệu giao dịch là truy cập dữ liệu hay truy cập lệnh.
	ARQOS	Master→Slave	Chất lượng dịch vụ (QoS). Mã định danh QoS được gửi cho mỗi giao dịch đọc. Chỉ được triển khai trong AXI4.
	ARREGION	Master→Slave	Mã định danh vùng. Cho phép một giao diện vật lý đơn lẻ trên tó được sử dụng cho nhiều giao diện logic. Chỉ được triển khai trong AXI4.
	ARUSER	Master→Slave	Tín hiệu người dùng. Tín hiệu tùy chọn do người dùng định nghĩa trong kênh địa chỉ đọc. Chỉ hỗ trợ trong AXI4.
	ARVALID	Master→Slave	Địa chỉ đọc hợp lệ. Tín hiệu này chỉ ra rằng kênh đang báo hiệu địa chỉ đọc hợp lệ và thông tin điều khiển.
	ARREADY	Slave→Master	Địa chỉ đọc sẵn sàng. Tín hiệu này chỉ ra rằng tó đã sẵn sàng chấp nhận một

			địa chỉ và các tín hiệu điều khiển liên quan.
Các tín hiệu kênh dữ liệu đọc	RID	Slave→Master	ID đọc. Tín hiệu này là thẻ nhận dạng cho nhóm tín hiệu dữ liệu đọc được tạo ra bởi tớ
	RDATA	Slave→Master	Dữ liệu đọc.
	RRESP	Slave→Master	Phản hồi đọc. Tín hiệu này chỉ ra trạng thái của lần chuyển đọc.
	RLAST	Slave→Master	Dữ liệu đọc cuối cùng. Tín hiệu này chỉ ra lần chuyển cuối cùng trong một Burst đọc.
	RUSER	Slave→Master	Tín hiệu người dùng. Tín hiệu tùy chọn do người dùng định nghĩa trong kênh dữ liệu đọc.
	RVALID	Slave→Master	Dữ liệu đọc hợp lệ. Tín hiệu này chỉ ra rằng kênh đang báo hiệu dữ liệu đọc yêu cầu.
	RREADY	Master→Slave	Dữ liệu đọc sẵn sàng. Tín hiệu này chỉ ra rằng chủ có thể chấp nhận dữ liệu đọc và thông tin phản hồi.

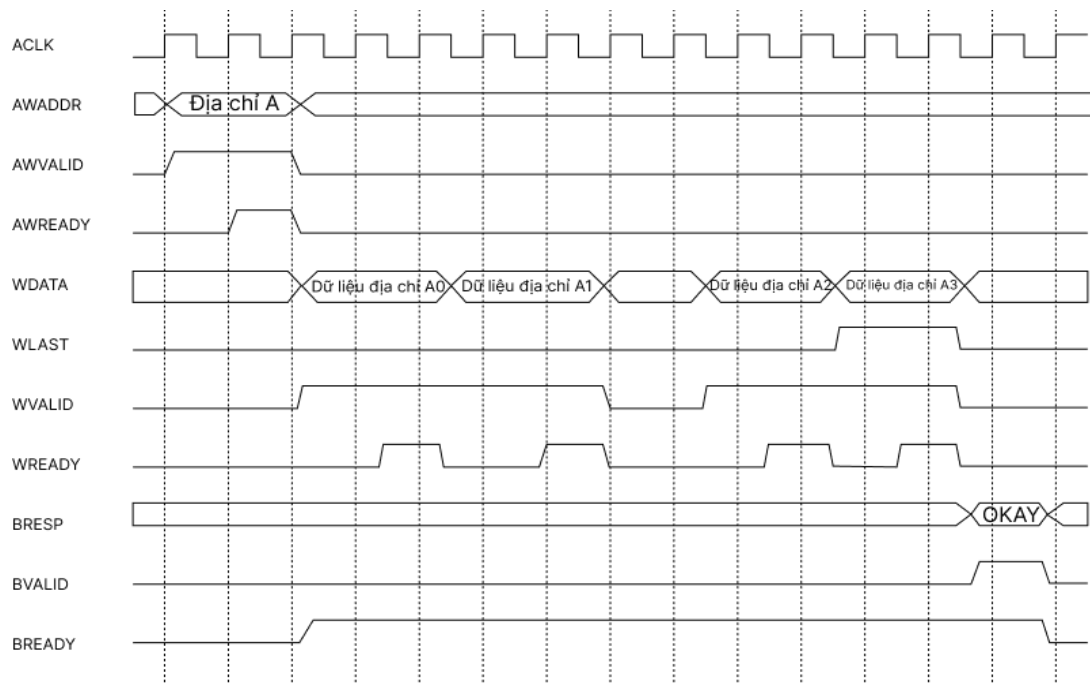
Bảng 2.5 trình bày các tín hiệu được sử dụng trong năm kênh của giao thức AXI4-Full, bao gồm: kênh địa chỉ ghi, kênh dữ liệu ghi, kênh phản hồi ghi, kênh

địa chỉ đọc và kênh dữ liệu đọc. Tổng cộng có khoảng 45 tín hiệu khác nhau được phân bổ trên các kênh này. Mỗi kênh đảm nhận một chức năng riêng, góp phần thực hiện việc truyền địa chỉ, dữ liệu cũng như phản hồi giữa thiết bị chủ và thiết bị tớ trong hệ thống.

2.3.4 Chi tiết tín hiệu của AXI4 Full trong việc đọc và ghi dữ liệu

Trong mục này, nội dung tập trung phân tích chi tiết các tín hiệu của giao thức AXI4-Full liên quan đến quá trình đọc và ghi dữ liệu trong chế độ Burst. Cơ chế Burst cho phép truyền liên tiếp nhiều dữ liệu chỉ với một lần gửi địa chỉ ban đầu, qua đó giúp nâng cao hiệu quả truyền thông và tối ưu băng thông hệ thống

Sơ đồ thời gian của quá trình ghi dữ liệu trên bus AMBA AXI4-Full trong chế độ Burst được trình bày trong Hình 2.5.



Hình 2.5 Sơ đồ thời gian chi tiết của tín hiệu chính cho đọc dữ liệu với Burst trên kênh địa chỉ ghi và kênh dữ liệu ghi

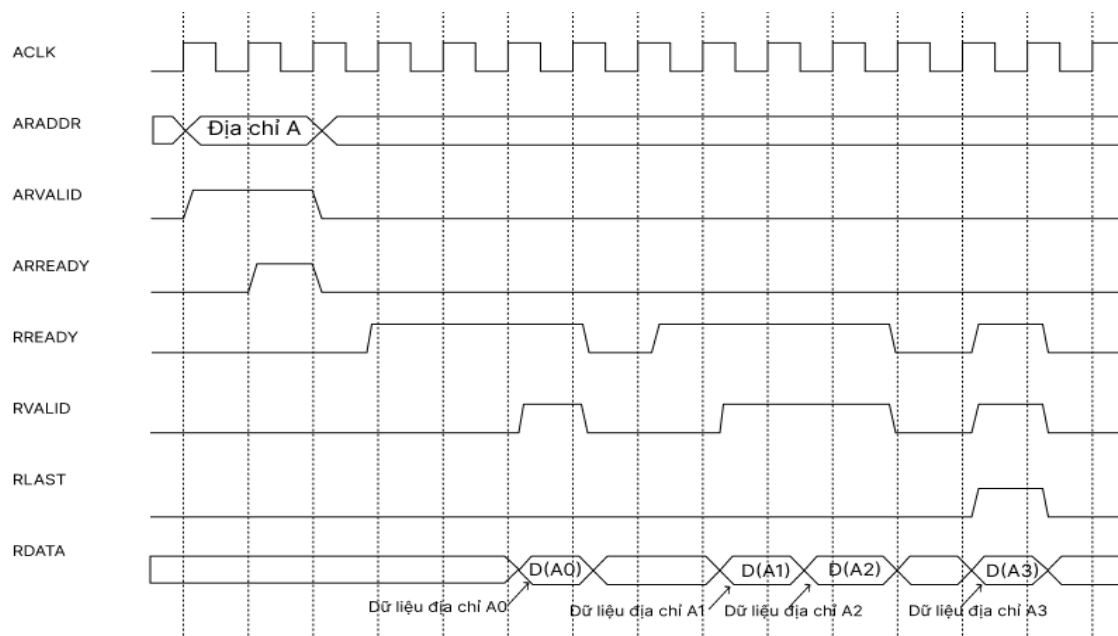
Hình 2.5 mô tả sơ đồ thời gian của một giao dịch ghi dữ liệu trên bus AXI trong chế độ Burst, sử dụng ba kênh chính gồm: kênh địa chỉ ghi, kênh dữ liệu ghi và kênh phản hồi ghi. Quá trình bắt đầu khi thiết bị chủ gửi địa chỉ cùng các tín hiệu điều khiển qua kênh địa chỉ ghi (AWADDR, AWVALID, AWREADY). Khi cả hai

tín hiệu AWVALID và AWREADY đều ở mức kích hoạt, địa chỉ được chấp nhận bởi thiết bị tớ.

Tiếp theo, dữ liệu được truyền qua kênh dữ liệu ghi (WDATA) kèm theo tín hiệu WVALID để xác nhận tính hợp lệ của dữ liệu. Tín hiệu WREADY từ phía tớ cho biết khả năng tiếp nhận dữ liệu. Trong chế độ Burst, các dữ liệu được gửi liên tục; khi đến phần tử cuối cùng, tín hiệu WLAST được kích hoạt để đánh dấu kết thúc chuỗi truyền.

Sau khi toàn bộ dữ liệu đã được tiếp nhận, thiết bị tớ gửi tín hiệu phản hồi thông qua kênh phản hồi ghi (BVALID, BREADY) nhằm thông báo trạng thái của giao dịch. Khi BVALID và BREADY đồng thời ở mức cao, phản hồi ghi (ví dụ “OKAY”) được gửi về chủ, kết thúc quá trình ghi. Sơ đồ này thể hiện rõ sự phối hợp giữa các tín hiệu điều khiển và dữ liệu trong quá trình truyền thông giữa chủ và tớ trên bus AXI.

Hình 2.6 minh họa sơ đồ thời gian của giao dịch đọc dữ liệu trên bus AXI4-Full ở chế độ Burst



Hình 2.6 Sơ đồ thời gian chi tiết của tín hiệu chính cho đọc dữ liệu với Burst trên kênh địa chỉ đọc và kênh dữ liệu đọc

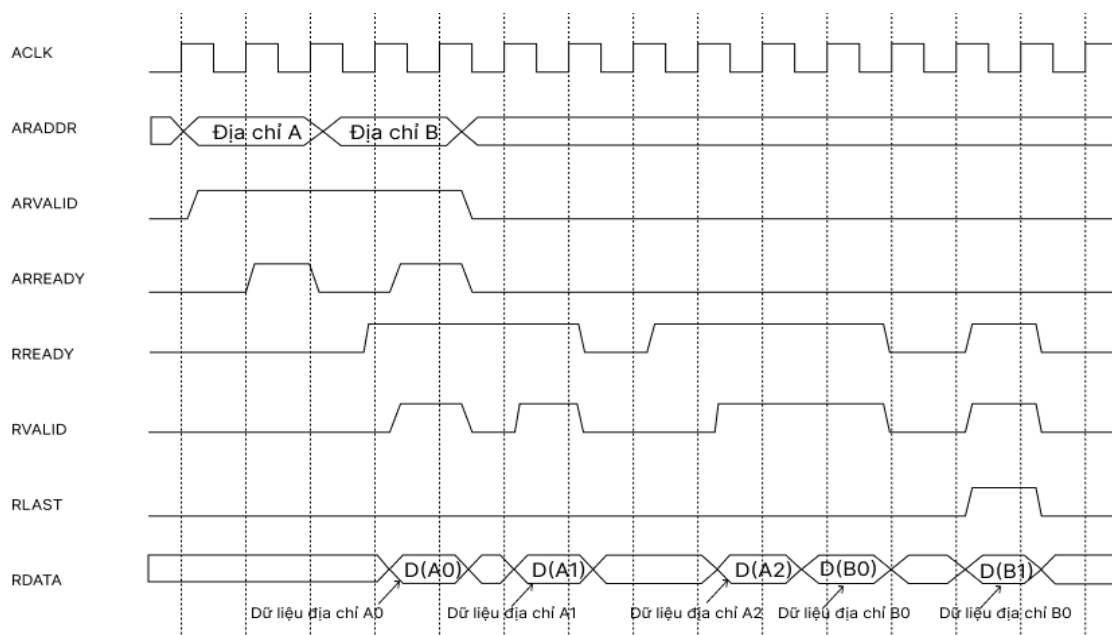
Hình 2.6 mô tả sơ đồ thời gian cho quá trình đọc dữ liệu theo chế độ Burst trên bus AXI, có cấu trúc hoạt động tương đồng với quá trình ghi đã được đề cập trước

đó. Trong ví dụ này, chủ phát địa chỉ đọc lên kênh ARADDR kèm theo các tín hiệu điều khiển mang thông tin về độ dài và kiểu Burst (được lược bỏ trong hình để tiện theo dõi). Chủ kéo tín hiệu ARVALID lên mức cao để xác nhận tính hợp lệ của địa chỉ và dữ liệu điều khiển trên bus; sau một chu kỳ, tín hiệu ARREADY từ phía tớ chuyển lên mức cao, đánh dấu việc tớ đã tiếp nhận thành công địa chỉ đó.

Tiếp theo, quá trình truyền dữ liệu diễn ra trên kênh RDATA. Tớ duy trì tín hiệu RVALID ở mức thấp cho đến khi dữ liệu sẵn sàng để phát, sau đó lần lượt chuyển dữ liệu sang chủ theo từng chu kỳ xung nhịp. Khi đến beat cuối cùng của chuỗi Burst, tớ kích hoạt tín hiệu RLAST để thông báo rằng đây là dữ liệu kết thúc giao dịch.

Nhằm nâng cao hiệu suất và tận dụng tối đa băng thông trong các hệ thống AXI4 Full, kỹ thuật đọc chồng chéo (overlapping read transactions) đóng vai trò then chốt. Theo đó, ngay sau khi tớ chấp nhận địa chỉ của một Burst, chủ có thể lập tức phát địa chỉ của Burst tiếp theo mà không cần đợi Burst hiện tại hoàn tất. Điều này cho phép tớ song song chuẩn bị dữ liệu cho giao dịch kế, từ đó rút ngắn thời gian chờ giữa các Burst và nâng cao khả năng xử lý tổng thể của hệ thống.

Cơ chế đọc chồng chéo trên bus AXI4-Full được minh họa trong Hình 2.7



Hình 2.7 Sơ đồ thời gian chi tiết của tín hiệu chính cho đọc dữ liệu chồng chéo với Burst trên kênh địa chỉ đọc và kênh dữ liệu đọc.

Hình 2.7 minh họa cơ chế đọc chồng chéo, trong đó chủ phát địa chỉ của Burst thứ hai (địa chỉ B) lên bus ngay khi địa chỉ Burst đầu tiên (địa chỉ A) vừa được tổ tiếp nhận. Cụ thể, sau khi chủ gửi địa chỉ A và tín hiệu ARREADY từ tổ chuyển lên mức cao xác nhận việc chấp nhận địa chỉ, chủ không chờ đợi mà lập tức chuyển sang phát địa chỉ B.

Song song với quá trình tổ đang truyền dữ liệu của Burst A lên kênh RDATA, tổ đồng thời tiến hành chuẩn bị dữ liệu cho Burst B. Khi dữ liệu của từng Burst sẵn sàng, tổ sử dụng cặp tín hiệu RVALID và RLAST để điều phối và đồng bộ hóa việc chuyển giao dữ liệu tương ứng cho từng giao dịch, đảm bảo chủ nhận được dữ liệu đúng thứ tự và đúng thời điểm.

2.4 Kiến trúc CNN nền tảng

2.4.1 Mạng CNN

Mạng nơ-ron tích chập (CNN) là một mô hình học sâu được phát triển dựa trên nền tảng của mạng nơ-ron nhân tạo, được thiết kế nhằm xử lý hiệu quả các dữ liệu có cấu trúc dạng lưới, đặc biệt là dữ liệu ảnh, tín hiệu và chuỗi thời gian [10], [11].

CNN sử dụng các lớp tích chập (Convolution Layer) để tự động trích xuất đặc trưng từ dữ liệu đầu vào. Nhờ cơ chế chia sẻ trọng số (Weight Sharing) và kết nối cục bộ (Local Connectivity), CNN có khả năng giảm đáng kể số lượng tham số cần huấn luyện trong khi vẫn duy trì khả năng học biểu diễn dữ liệu mạnh mẽ [10].

CNN được giới thiệu lần đầu bởi Yann LeCun và các cộng sự vào cuối những năm 1980, sau đó được phát triển thành công thông qua kiến trúc LeNet phục vụ bài toán nhận dạng chữ số viết tay [9]. Kể từ đó, CNN đã trở thành một trong những kiến trúc quan trọng nhất trong lĩnh vực học sâu và được ứng dụng rộng rãi trong nhiều lĩnh vực như nhận dạng ảnh, phát hiện đối tượng, xử lý tín hiệu sinh học, nhận dạng giọng nói và phân tích dữ liệu chuỗi thời gian [10], [11].

Mục tiêu chính của CNN là tự động học các đặc trưng từ dữ liệu đầu vào thông qua nhiều tầng xử lý khác nhau. Các tầng đầu thường học các đặc trưng đơn giản như cạnh, đường viền hoặc các mẫu cục bộ, trong khi các tầng sâu hơn có khả năng học các đặc trưng trừu tượng và mang tính phân biệt cao hơn, phục vụ cho quá trình phân loại hoặc nhận dạng [10].

Nhờ khả năng tự động trích xuất đặc trưng và hiệu quả học tập vượt trội, CNN đã trở thành nền tảng cho nhiều kiến trúc học sâu hiện đại như ResNet [12], DenseNet [13], InceptionNet [14]

CNN hoạt động dựa trên quá trình trích xuất đặc trưng từ dữ liệu đầu vào thông qua các lớp tích chập. Các bộ lọc được sử dụng để quét qua dữ liệu và phát hiện các đặc trưng quan trọng.

$$Y(i, j) = \sum_m \sum_n X(i + m, j + n) \cdot W(m, n) ; \quad (1)$$

trong đó:

- X là dữ liệu đầu vào.
- W là bộ lọc (kernel).
- Y là đặc trưng đầu ra sau phép tích chập.
- m,n là các chỉ số của bộ lọc.

Theo (1), bộ lọc sẽ di chuyển trên dữ liệu đầu vào và thực hiện phép nhân tương ứng giữa các phần tử trong vùng quan sát với các trọng số của bộ lọc. Sau đó các giá trị được cộng lại để tạo thành một đặc trưng mới. Thông qua quá trình huấn luyện, các bộ lọc sẽ tự động học các mẫu dữ liệu quan trọng nhất phục vụ cho nhiệm vụ phân loại hoặc nhận dạng.

2.4.2 Mạng 1-D CNN

Mạng tích chập một chiều (One-Dimensional Convolutional Neural Network – 1-D CNN) là một biến thể của mạng nơ-ron tích chập (Convolutional Neural Network – CNN), được thiết kế để xử lý các dữ liệu có cấu trúc dạng chuỗi hoặc tín hiệu theo thời gian [10], [11]. Khác với CNN truyền thống thường được sử

dụng cho dữ liệu ảnh hai chiều, 1-D CNN thực hiện phép tích chập trên một chiều duy nhất của dữ liệu, cho phép khai thác hiệu quả các mối quan hệ cục bộ giữa các điểm dữ liệu liên tiếp trong chuỗi tín hiệu.

Nguyên lý hoạt động của 1-D CNN dựa trên việc sử dụng các bộ lọc (filters hoặc kernels) di chuyển dọc theo tín hiệu đầu vào nhằm trích xuất các đặc trưng quan trọng [10]. Trong quá trình huấn luyện, các bộ lọc sẽ tự động học các mẫu đặc trưng từ dữ liệu mà không cần thực hiện các bước thiết kế đặc trưng thủ công như trong các phương pháp học máy truyền thống. Nhờ khả năng học đặc trưng tự động, 1-D CNN có thể phát hiện các mẫu tín hiệu ở nhiều mức độ khác nhau, từ các đặc trưng đơn giản đến các đặc trưng phức tạp phục vụ cho quá trình phân loại.

1-D CNN sử dụng cơ chế chia sẻ trọng số (Weight Sharing) và kết nối cục bộ (Local Connectivity), giúp giảm đáng kể số lượng tham số cần huấn luyện cũng như chi phí tính toán [10]. Điều này không chỉ làm tăng hiệu quả học tập của mô hình mà còn góp phần hạn chế hiện tượng quá khớp (Overfitting), đặc biệt khi làm việc với các tập dữ liệu có kích thước không quá lớn.

Đối với các bài toán xử lý dữ liệu chuỗi thời gian, 1-D CNN cho thấy hiệu quả cao trong việc khai thác các đặc trưng phụ thuộc theo thời gian nhờ khả năng học trực tiếp từ dữ liệu đầu vào [11]. Mô hình đã được ứng dụng rộng rãi trong nhiều lĩnh vực như nhận dạng giọng nói, phân tích tín hiệu cảm biến, xử lý tín hiệu y sinh và phân tích dữ liệu chuỗi thời gian.

Trong lĩnh vực phân tích tín hiệu điện não đồ (EEG), 1-D CNN đặc biệt phù hợp vì tín hiệu EEG được biểu diễn dưới dạng chuỗi các giá trị điện thế thay đổi theo thời gian [9]. Các lớp tích chập một chiều có khả năng phát hiện các mẫu dao động đặc trưng của tín hiệu não bộ, từ đó hỗ trợ hiệu quả cho các bài toán như phát hiện động kinh, phân loại trạng thái não, giao diện não – máy tính (Brain-Computer Interface) và nhận dạng các bất thường thần kinh [24]. Nhờ khả năng tự động trích xuất đặc trưng và hiệu quả tính toán cao, 1-D CNN đã trở thành nền tảng quan trọng cho nhiều kiến trúc học sâu hiện đại trong xử lý tín hiệu EEG.

Trong nghiên cứu này, 1-D CNN được sử dụng làm nền tảng cho quá trình trích xuất đặc trưng của tín hiệu EEG. Các lớp tích chập một chiều có nhiệm vụ học các đặc trưng theo miền thời gian từ dữ liệu đầu vào, đồng thời được kết hợp với cơ chế học phần dư của ResNet nhằm nâng cao khả năng biểu diễn đặc trưng và cải thiện hiệu quả phân loại.

Nguyên lý hoạt động của 1-D CNN dựa trên phép tích chập giữa tín hiệu đầu vào và các bộ lọc học được trong quá trình huấn luyện. Mỗi bộ lọc có nhiệm vụ phát hiện một loại đặc trưng cụ thể của tín hiệu như các đỉnh sóng, biên độ dao động hoặc các mẫu biến đổi theo thời gian.

$$Y(i) = \sum_m X(i + m) \cdot W(m) ; \quad (2)$$

trong đó:

- $x(t)$ là giá trị của tín hiệu đầu vào tại thời điểm t .
- $w(i)$ là trọng số của bộ lọc.
- k là kích thước của bộ lọc (kernel size).
- $y(t)$ là giá trị đầu ra sau khi thực hiện phép tích chập.

Theo (2), bộ lọc sẽ lần lượt quét qua toàn bộ tín hiệu đầu vào. Tại mỗi vị trí, các phần tử của tín hiệu trong vùng quan sát sẽ được nhân với các trọng số tương ứng của bộ lọc, sau đó cộng lại để tạo thành một giá trị mới. Kết quả thu được là một bản đồ đặc trưng (feature map), phản ánh những đặc điểm mà bộ lọc đã học được từ dữ liệu.

2.4.3 Mạng ResNet

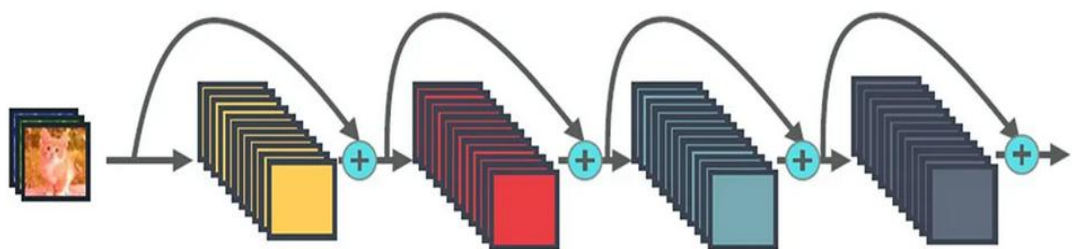
Mạng Residual Network (ResNet) là một kiến trúc mạng nơ-ron tích chập sâu (Deep Convolutional Neural Network – CNN) được đề xuất bởi Kaiming He và các cộng sự nhằm khắc phục những hạn chế của các mạng CNN có độ sâu lớn [12]. Công trình này được giới thiệu lần đầu vào năm 2015 và chính thức công bố tại Hội nghị IEEE Conference on Computer Vision and Pattern Recognition

(CVPR) năm 2016, đánh dấu một bước tiến quan trọng trong lĩnh vực học sâu [12].

Trước khi ResNet được phát triển, việc xây dựng các mạng nơ-ron với số lượng lớp lớn thường gặp nhiều khó khăn trong quá trình huấn luyện. Khi độ sâu của mạng tăng lên, mô hình có thể gặp phải hiện tượng mất gradient (vanishing gradient) hoặc suy giảm hiệu suất (degradation problem), khiến độ chính xác không những không được cải thiện mà còn có xu hướng giảm xuống [9], [12]. Điều này đặt ra thách thức lớn trong việc thiết kế các mạng học sâu có khả năng học được những đặc trưng phức tạp từ dữ liệu.

Để giải quyết vấn đề trên, ResNet đưa ra cơ chế học phần dư (Residual Learning) thông qua các kết nối tắt (Shortcut Connection hoặc Skip Connection). Thay vì học trực tiếp ánh xạ mong muốn từ đầu vào đến đầu ra, mô hình chỉ cần học phần sai khác (residual) giữa chúng. Đầu vào của một khối mạng được truyền trực tiếp đến đầu ra thông qua nhánh shortcut và được cộng với kết quả từ nhánh tích chập, giúp thông tin và gradient có thể lan truyền hiệu quả qua nhiều tầng của mạng [12].

Hình 2.8 minh họa cấu trúc khối Residual Block trong kiến trúc ResNet



Hình 2.8 Tổng quan về mô hình ResNet [17]

Quan sát Hình 2.8 cho thấy đầu vào (x) được truyền đồng thời qua hai đường khác nhau. Ở nhánh chính, dữ liệu lần lượt đi qua các lớp trọng số (weight layer) và hàm kích hoạt ReLU để học hàm phần dư ($F(x)$). Song song với đó, đầu vào được truyền trực tiếp qua nhánh shortcut (identity mapping) mà không trải qua quá trình biến đổi.

2.4.4 Các thành phần sử dụng trong mô hình

Để xây dựng mô hình phân loại tín hiệu EEG, nghiên cứu sử dụng nhiều thành phần khác nhau của mạng học sâu nhằm thực hiện quá trình trích xuất đặc trưng và phân loại dữ liệu. Các thành phần chính bao gồm lớp tích chập một chiều (Conv1D), hàm kích hoạt ReLU, lớp gộp cực đại (MaxPooling1D), khối phần dư (Residual Block), lớp Global Average Pooling và hàm Softmax. Mỗi thành phần đảm nhận một vai trò riêng trong quá trình xử lý dữ liệu.

2.4.4.1 Lớp tích chập một chiều (Conv1D)

Lớp tích chập một chiều (Conv1D) là thành phần cơ bản trong các mô hình xử lý dữ liệu chuỗi thời gian và tín hiệu sinh học. Conv1D sử dụng các bộ lọc (filters) di chuyển dọc theo tín hiệu đầu vào để phát hiện các đặc trưng cục bộ quan trọng như đỉnh sóng, dao động hoặc các mẫu biến đổi theo thời gian

$$Y(i) = \sum_m X(i + m) \cdot W(m) ; \quad (3)$$

trong đó:

- $x(t)$ là giá trị của tín hiệu đầu vào tại thời điểm t .
- $w(i)$ là trọng số của bộ lọc.
- k là kích thước của bộ lọc (kernel size).
- $y(t)$ là giá trị đầu ra sau khi thực hiện phép tích chập.

Theo (3), bộ lọc sẽ lần lượt quét qua toàn bộ tín hiệu đầu vào. Tại mỗi vị trí, các phần tử của tín hiệu trong vùng quan sát sẽ được nhân với các trọng số tương ứng của bộ lọc, sau đó cộng lại để tạo thành một giá trị mới. Kết quả thu được là một bản đồ đặc trưng (feature map), phản ánh những đặc điểm mà bộ lọc đã học được từ dữ liệu.

2.4.4.2 Hàm ReLU

Sau mỗi lớp tích chập, đầu ra thường được đưa qua hàm kích hoạt phi tuyến nhằm tăng khả năng biểu diễn của mô hình.

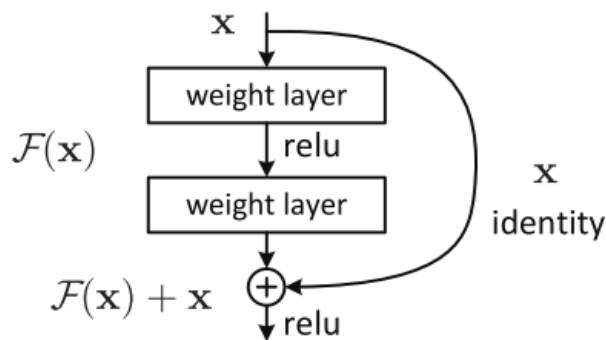
$$f(x) = \max(0, x) ; \quad (4)$$

Theo công thức (4), các giá trị âm sẽ được đưa về 0 trong khi các giá trị dương được giữ nguyên. Điều này giúp tăng tính phi tuyến của mạng và giảm hiện tượng mất gradient trong quá trình huấn luyện.

2.4.4.3 Khối Residual Block

Residual Block là thành phần cốt lõi của kiến trúc ResNet, bao gồm nhánh chính dùng để học đặc trưng thông qua các lớp tích chập và nhánh tắt (Shortcut Connection) dùng để truyền trực tiếp thông tin từ đầu vào đến đầu ra của khối. Hai nhánh được kết hợp thông qua phép cộng phần tử để tạo thành đầu ra cuối cùng của Residual Block. Cấu trúc này giúp cải thiện khả năng lan truyền gradient và cho phép xây dựng các mạng có độ sâu lớn mà vẫn đảm bảo hiệu quả huấn luyện.

Hình 2.9 minh họa cấu trúc tổng quát của một Residual Block trong kiến trúc ResNet.



Hình 2.9 Tổng quan về khối Residual Block

Một Residual Block thường bao gồm các lớp tích chập (Convolution Layer), các hàm kích hoạt phi tuyến và một nhánh kết nối tắt (Shortcut Connection) như Hình 2.9. Trong cấu trúc này, dữ liệu đầu vào được truyền qua hai đường khác nhau. Đường thứ nhất là nhánh chính (Main Path), nơi dữ liệu được xử lý qua các lớp tích chập để học đặc trưng. Đường thứ hai là nhánh shortcut, cho phép đầu vào được truyền trực tiếp đến đầu ra của khối mà không phải đi qua các phép biến đổi phức tạp. Đầu ra của Residual Block được xác định theo công thức:

$$H(x) = F(x) + x ; \quad (5)$$

trong đó:

- (x) là dữ liệu đầu vào.
- $F(x)$ là hàm phân dư được học bởi các lớp tích chập.
- $H(x)$ là đầu ra của khối.

Theo công thức (5), đầu ra của khối được tạo thành bằng cách cộng trực tiếp đặc trưng học được từ nhánh chính với dữ liệu đầu vào ban đầu. Cơ chế này giúp thông tin được truyền xuyên suốt qua nhiều tầng mạng, đồng thời tạo điều kiện thuận lợi cho quá trình lan truyền gradient trong giai đoạn huấn luyện.

2.4.4.4 Lớp MaxPooling1D

Lớp MaxPooling1D là một kỹ thuật giảm mẫu thường được sử dụng trong các mạng tích chập một chiều nhằm giảm kích thước của dữ liệu đặc trưng sau quá trình tích chập. Mục tiêu của lớp này là giữ lại những đặc trưng quan trọng nhất trong khi giảm số lượng dữ liệu cần xử lý ở các tầng tiếp theo [10].

Nguyên lý hoạt động của MaxPooling1D dựa trên việc chia dữ liệu đầu vào thành các cửa sổ có kích thước xác định trước (pool size). Trong mỗi cửa sổ, giá trị lớn nhất sẽ được lựa chọn làm đầu ra đại diện cho vùng dữ liệu đó. Nhờ vậy, các đặc trưng nổi bật của tín hiệu được bảo toàn, trong khi các thông tin ít quan trọng hơn được loại bỏ.

Giá trị đầu ra của phép Max Pooling được xác định theo công thức:

$$y = \max(x_1, x_2, \dots, x_n); \quad (6)$$

trong đó:

- $x_1, x_2, \dots, x_n$ là các giá trị trong cửa sổ pooling
- y là giá trị đầu ra sau khi thực hiện phép Max Pooling

Theo công thức (6), giá trị lớn nhất trong vùng quan sát sẽ được giữ lại để đại diện cho toàn bộ cửa sổ. Điều này giúp mô hình tập trung vào các đặc trưng nổi bật nhất của dữ liệu.

Trong các bài toán xử lý tín hiệu EEG, MaxPooling1D giúp giảm chiều dữ liệu đặc trưng sau các lớp Conv1D, từ đó giảm chi phí tính toán và số lượng tham số cần huấn luyện. Đồng thời, việc giữ lại các giá trị cực đại còn giúp mô hình duy trì được những đặc trưng quan trọng của tín hiệu não bộ, góp phần nâng cao hiệu quả phân loại.

2.4.4.5 Lớp Global Average Pooling

Global Average Pooling (GAP) là một kỹ thuật tổng hợp đặc trưng thường được sử dụng ở giai đoạn cuối của các mạng tích chập nhằm giảm kích thước dữ liệu trước khi thực hiện phân loại. GAP thực hiện phép lấy giá trị trung bình trên toàn bộ chiều không gian hoặc chiều thời gian của từng kênh đặc trưng, từ đó tạo ra một vector đặc trưng có kích thước nhỏ gọn hơn

Đối với dữ liệu chuỗi thời gian và tín hiệu EEG, lớp GlobalAveragePooling1D thực hiện phép tính trung bình trên chiều thời gian của mỗi kênh đặc trưng. Giá trị đầu ra của mỗi kênh được xác định theo công thức:

$$G_k = \frac{1}{N} \sum_{i=1}^N x_{i,k} ; \quad (7)$$

trong đó:

- N là số phần tử trên chiều thời gian của kênh đặc trưng.
- $X_{i,k}$ là giá trị tại vị trí thứ i của kênh đặc trưng thứ k .
- G_k là giá trị trung bình của kênh đặc trưng thứ k .

Theo công thức (7), mỗi kênh đặc trưng sau lớp tích chập sẽ được biểu diễn bằng một giá trị trung bình duy nhất. Nhờ đó, dữ liệu đầu ra được rút gọn đáng kể nhưng vẫn giữ được thông tin tổng quát của các đặc trưng đã học.

2.4.4.6 Hàm Softmax

Softmax là một hàm kích hoạt thường được sử dụng tại lớp đầu ra của các mô hình học sâu trong các bài toán phân loại đa lớp (Multi-class Classification). Chức năng chính của hàm Softmax là chuyển đổi các giá trị đầu ra của mạng nơ-ron

thành các xác suất tương ứng với từng lớp, từ đó hỗ trợ quá trình đưa ra quyết định phân loại

$$\sigma(z)_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} ; \quad (8)$$

trong đó:

- Z_i là giá trị đầu ra (logit) của lớp thứ i trước khi áp dụng hàm Softmax
- Z_j là giá trị logit thứ j trong vector đầu vào
- K là số lớp phân loại
- e^{z_i} hàm mũ cơ số e , đảm bảo giá trị luôn dương
- $\sum_{j=1}^K e^{z_j}$ là tổng toàn bộ giá trị mũ dùng để chuẩn hóa

Theo công thức (8), mỗi giá trị đầu ra z_i trước tiên được chuyển đổi sang dạng hàm mũ e^{z_i} . Sau đó, giá trị này được chia cho tổng các giá trị hàm mũ của toàn bộ các lớp. Quá trình chuẩn hóa này giúp các đầu ra của mô hình được chuyển thành các giá trị nằm trong khoảng từ 0 đến 1 và có tổng bằng 1.

CHƯƠNG 3 THIẾT KẾ HỆ THỐNG

3.1 Yêu cầu hệ thống

3.1.1 Yêu cầu về kiến trúc mô hình 1-D CNN

Mục tiêu của hệ thống là xây dựng mô hình 1-D CNN có khả năng phân loại chính xác tín hiệu EEG từ tập dữ liệu Bonn EEG, đồng thời phù hợp để triển khai trên nền tảng FPGA. Để đáp ứng yêu cầu này, kiến trúc mạng cần thỏa mãn các tiêu chí sau:

- Độ chính xác cao: Mô hình phải trích xuất hiệu quả các đặc trưng của tín hiệu EEG để phân biệt chính xác các trạng thái hoạt động của não bộ. Độ chính xác cao giúp nâng cao độ tin cậy của hệ thống và đảm bảo hiệu quả phân loại trong thực tế.
- Tổng số lượng tham số thấp: Kiến trúc mạng cần được tối ưu nhằm giảm số lượng tham số và phép toán tính toán. Điều này giúp tiết kiệm tài nguyên phần cứng, giảm dung lượng lưu trữ trọng số và tăng tốc độ tính toán khi triển khai trên FPGA.

3.1.2 Yêu cầu về phần cứng

Mục tiêu của hệ thống là xây dựng kiến trúc CNN có khả năng xử lý tín hiệu EEG hiệu quả trên nền tảng SoC FPGA. Để đáp ứng yêu cầu triển khai thực tế, phần cứng cần thỏa mãn các tiêu chí sau:

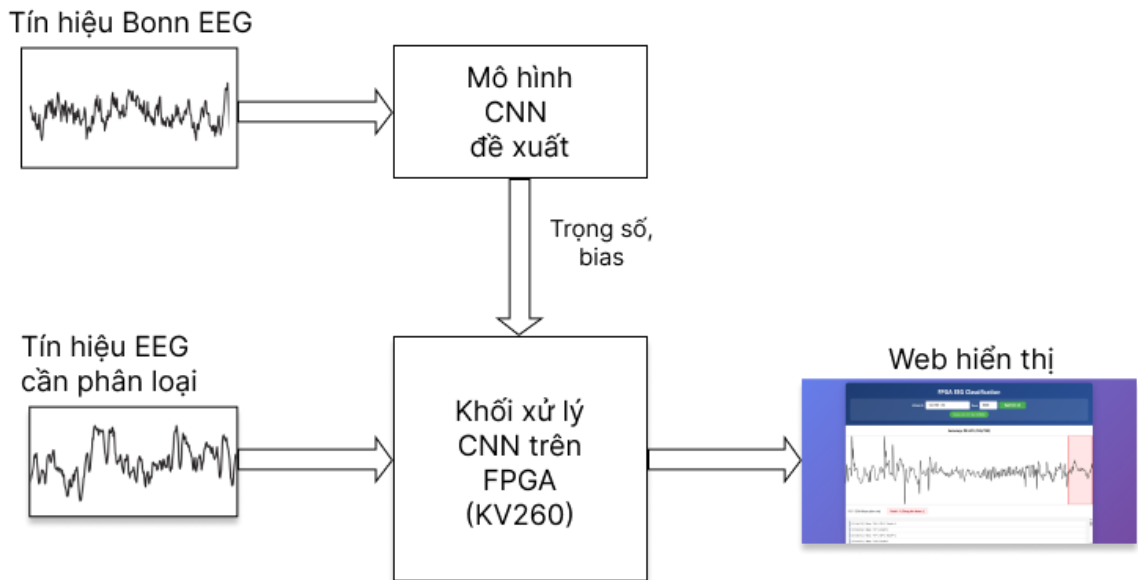
- Năng lượng thấp: Kiến trúc phần cứng cần được thiết kế nhằm giảm mức tiêu thụ năng lượng trong quá trình xử lý. Điều này giúp hệ thống hoạt động ổn định trong thời gian dài và phù hợp với các thiết bị nhúng.
- Kích thước nhỏ: Bộ gia tốc cần sử dụng hiệu quả các tài nguyên phần cứng như LUT, FF, ... để giảm diện tích chiếm dụng trên FPGA. Điều này cho phép triển khai trên các thiết bị có tài nguyên hạn chế và giảm chi phí hệ thống.
- Khả năng thay đổi cấu hình mạng CNN: Kiến trúc phần cứng cần hỗ trợ thay đổi các tham số của mạng CNN như số lớp, số kênh đầu vào/đầu ra,

kích thước kernel hoặc kiểu lớp xử lý mà không cần sửa đổi lớn trong thiết kế phần cứng. Điều này giúp tăng tính linh hoạt và khả năng tái sử dụng của bộ gia tốc đối với nhiều mô hình CNN khác nhau.

3.2 Thiết kế hệ thống

3.2.1 Thiết kế tổng quát

Hình 3.1 thể hiện thiết kế tổng quát của toàn bộ hệ thống.



Hình 3.1 Thiết kế tổng quát của hệ thống

Hình 3.1 trình bày kiến trúc tổng quan của hệ thống phân loại tín hiệu EEG động kinh trên nền tảng FPGA. Tín hiệu EEG đầu vào được đưa vào mô hình CNN đã được thiết kế và sau đó huấn luyện trên máy tính. Sau khi hoàn tất quá trình huấn luyện, các tham số của mạng (trọng số và bias) được triển khai lên khối xử lý CNN trên FPGA Kria KV260. Khối xử lý này thực hiện quá trình tính toán trực tiếp trên dữ liệu EEG đầu vào để phân loại trạng thái tín hiệu. Hệ thống được triển khai theo mô hình SoC FPGA, trong đó phần mềm trên Processing System (PS) phối hợp với phần cứng trên Programmable Logic (PL) để thực hiện mạng 1-D CNN. PS (CPU) chạy hệ điều hành Linux đảm nhiệm các tác vụ điều phối như: đọc dữ liệu từ tập dữ liệu, tiền xử lý/cắt tín hiệu theo nhịp, nạp trọng số và tham số cấu hình, kích hoạt bộ xử lý, và cuối cùng đọc kết quả phân loại từ bộ xử lý để đánh giá. Về phía phần cứng, các phép tính nặng (đặc biệt là Conv1D, MaxPooling, ReLU và

cộng residual) được thực thi trong PL. Sau khi hoàn thành các lớp trích xuất đặc trưng trên bộ xử lý, dữ liệu đầu ra được chuyển về PS để thực hiện lớp Global Average Pooling (GAP). Vector đặc trưng thu được sau GAP được sử dụng để thực hiện bước phân loại cuối cùng, từ đó phân loại tín hiệu EEG đầu vào. Kết quả nhận dạng sau đó được truyền đến giao diện web để hiển thị tín hiệu cùng với kết quả phân loại theo thời gian thực.

3.2.2 Mô hình Deep Learning

3.2.2.1 Tập dữ liệu

Nhóm lựa chọn tập dữ liệu Bonn EEG Dataset (còn được gọi là Bonn University EEG Database) làm nguồn dữ liệu chính cho bài toán phân loại tín hiệu điện não. Đây là một trong những bộ dữ liệu EEG công khai được sử dụng rộng rãi trong các nghiên cứu về phát hiện động kinh và phân tích tín hiệu não nhờ tính chuẩn hóa cao, dữ liệu tương đối sạch và hệ thống phân chia trạng thái não rõ ràng. Bộ dữ liệu được xây dựng bởi nhóm nghiên cứu của University of Bonn và được giới thiệu trong nghiên cứu của Ralph G. Andrzejak cùng cộng sự. Mục tiêu chính của nghiên cứu là khảo sát liệu tín hiệu EEG của não người có tồn tại các đặc tính phi tuyến và mang tính xác định (deterministic) hay không, thay vì chỉ là tín hiệu ngẫu nhiên tuyến tính. Đồng thời, nghiên cứu cũng phân tích sự khác biệt về đặc trưng động học của tín hiệu EEG giữa các trạng thái hoạt động khác nhau của não bộ, đặc biệt là giữa trạng thái bình thường và trạng thái động kinh.

Tập dữ liệu Bonn bao gồm 5 nhóm tín hiệu EEG riêng biệt ký hiệu Z, O, N, F và S; mỗi nhóm chứa 100 đoạn tín hiệu EEG đơn kênh với thời lượng khoảng 23.6 giây. Các tín hiệu được lấy mẫu ở tần số 173.61 Hz và đã được lựa chọn nhằm giảm thiểu ảnh hưởng của nhiễu và artifact. Trong đó, tập Z và O được thu từ người khỏe mạnh trong trạng thái mở mắt và nhắm mắt; tập N và F được ghi từ bệnh nhân động kinh trong giai đoạn không lên cơn, tương ứng với vùng ngoài và trong vùng sinh động kinh; tập S chứa tín hiệu EEG ghi nhận trong cơn động kinh (seizure). Việc xây dựng dữ liệu theo nhiều trạng thái sinh lý và bệnh lý khác nhau

giúp bộ dữ liệu phản ánh rõ sự thay đổi về đặc trưng động học của tín hiệu điện não trong các điều kiện hoạt động khác nhau của não bộ.

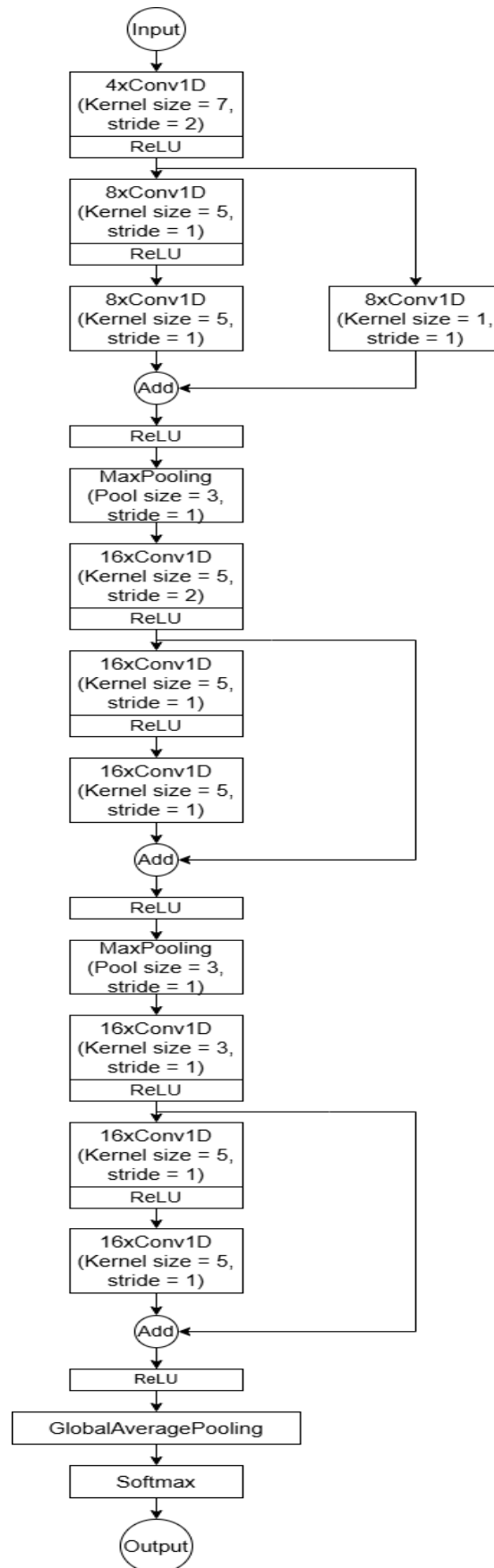
Trong quá trình tiền xử lý dữ liệu, nhóm thực hiện phân chia dữ liệu theo tỉ lệ 70% cho huấn luyện, 15% cho validation và 15% cho kiểm thử. Việc phân chia được thực hiện ở mức file nhằm đảm bảo các đoạn tín hiệu sinh ra từ cùng một bản ghi EEG chỉ xuất hiện trong duy nhất một tập dữ liệu. Cách tiếp cận này giúp hạn chế hiện tượng rò rỉ dữ liệu giữa các tập và phản ánh chính xác hơn khả năng tổng quát hóa của mô hình trong quá trình đánh giá.

Sau bước phân chia dữ liệu, mỗi tín hiệu EEG tiếp tục được xử lý bằng kỹ thuật sliding window để tạo ra các đoạn tín hiệu có độ dài cố định phục vụ cho việc huấn luyện mạng học sâu. Trong nghiên cứu này, nhóm sử dụng kích thước cửa sổ 320 mẫu. Với tần số lấy mẫu 173.61 Hz của bộ dữ liệu Bonn, mỗi đoạn tín hiệu tương ứng khoảng 1.84 giây hoạt động điện não. Việc chia tín hiệu thành các đoạn ngắn giúp mô hình tập trung khai thác các đặc trưng quan trọng theo miền thời gian như sự thay đổi biên độ, nhịp dao động và các mẫu hoạt động bất thường liên quan đến động kinh, đồng thời làm tăng số lượng mẫu dữ liệu phục vụ quá trình huấn luyện.

Để xây dựng bài toán phân loại phù hợp với mục tiêu nghiên cứu, nhóm tiến hành gộp 5 tập dữ liệu gốc thành 3 lớp chính. Cụ thể, các tập Z và O được gán vào lớp tín hiệu EEG bình thường của người khỏe mạnh; các tập N và F đại diện cho tín hiệu EEG của bệnh nhân động kinh trong giai đoạn không lên cơn; trong khi tập S đại diện cho tín hiệu EEG trong cơn động kinh. Việc ánh xạ dữ liệu theo hướng này giúp bài toán tập trung hơn vào khả năng phân biệt giữa trạng thái bình thường, trạng thái liên quan đến động kinh nhưng chưa xuất hiện cơn giật và trạng thái cơn giật thực sự, đồng thời giảm độ phức tạp của bài toán so với thiết lập phân loại 5 lớp ban đầu.

3.2.2.2 Tổng quan kiến trúc mô hình

Mô hình được xây dựng dựa trên mạng ResNet với cấu trúc được thể hiện ở Hình 3.2



Hình 3.2 Cấu trúc mô hình Deep Learning

Hình 3.2 cho thấy mô hình bao gồm các lớp:

1. Input layer: Lớp đầu vào là tín hiệu EEG theo từng nhịp có kích thước (320×1) .
2. Lớp Conv đầu tiên: Sử dụng Conv1D với 4 kernel, kích thước kernel = 7 và stride = 2, sau đó qua hàm kích hoạt ReLU nhằm trích xuất đặc trưng ban đầu và giảm chiều dài tín hiệu. Ngõ ra thu được có kích thước (160×4) .
3. Residual Block 1: Khối residual đầu tiên làm việc trên tensor đầu vào (160×4) .
 - Nhánh chính gồm:
 - + Conv1D với 8 kernel, kích thước kernel = 5, stride = 1, ReLU.
 - + Conv1D với 8 kernel, kích thước kernel = 5, stride = 1.
 - Nhánh shortcut sử dụng Conv1D 1×1 để điều chỉnh số lượng kênh từ 4 lên 8 nhằm phù hợp với nhánh chính trước khi thực hiện phép cộng.
 - Sau phép cộng, kết quả được đưa qua ReLU.
 - Ngõ ra của Residual Block 1 có kích thước (160×8) .
4. Lớp MaxPooling thứ nhất: Sử dụng MaxPooling1D với pool size = 3, stride = 1 nhằm tăng các đặc trưng nổi bật nhưng vẫn giữ nguyên kích thước tensor. Ngõ ra vẫn là (160×8) .
5. Lớp Conv thứ 2: Sử dụng Conv1D với 16 kernel, kích thước kernel = 5 và stride = 2, sau đó qua ReLU để tiếp tục giảm chiều dài tín hiệu và tăng số lượng đặc trưng. Ngõ ra thu được có kích thước xấp xỉ (80×16) .
6. Residual Block 2: Khối residual thứ hai hoạt động trên tensor (80×16) .
 - Nhánh chính gồm hai lớp Conv1D liên tiếp với:
 - + 16 kernel, kích thước kernel = 5, stride = 1.
 - + Sau lớp Conv thứ nhất là hàm kích hoạt ReLU.
 - Do số lượng kênh đầu vào đã phù hợp nên nhánh shortcut được giữ nguyên.
 - Sau phép cộng, dữ liệu được đưa qua ReLU.

- Ngõ ra của Residual Block 2 giữ nguyên kích thước (80×16).
7. Lớp MaxPooling thứ hai: Tương tự như lớp Maxpooling thứ nhất, sử dụng MaxPooling1D với pool size = 3, stride = 1 nhằm làm nổi bật đặc trưng quan trọng nhưng không thay đổi kích thước dữ liệu. Ngõ ra vẫn là (80×16).
 8. Lớp Conv thứ 3: Sử dụng Conv1D với 16 kernel, kích thước kernel = 3, stride = 1, sau đó qua ReLU để tiếp tục trích xuất đặc trưng cục bộ của tín hiệu ECG. Kích thước ngõ ra vẫn là (80×16).
 9. Residual Block 3: Khối residual cuối cùng tiếp tục xử lý tensor (80×16).
 - Nhánh chính gồm hai lớp Conv1D liên tiếp với:
 - + 16 kernel, kích thước kernel = 5, stride = 1.
 - + Sau lớp Conv đầu tiên là hàm ReLU.
 - Shortcut được giữ nguyên do không thay đổi kích thước dữ liệu.
 - Sau phép cộng và ReLU, ngõ ra của block vẫn giữ nguyên kích thước (80×16).
 10. Lớp GlobalAveragePooling1D: Thực hiện phép lấy trung bình theo chiều thời gian trên từng kênh đặc trưng, chuyển tensor từ kích thước (80×16) thành vector đặc trưng kích thước (1×16). Lớp này giúp giảm số lượng tham số đáng kể so với fully-connected truyền thống và phù hợp cho triển khai trên hệ thống nhúng hoặc FPGA.
 11. Lớp Dense ngõ ra: Lớp cuối cùng là Dense với số nơ-ron bằng số lớp cần phân loại và sử dụng hàm kích hoạt Softmax để đưa ra xác suất dự đoán cho từng lớp.

Tổng số lượng tham số của mô hình: 7243

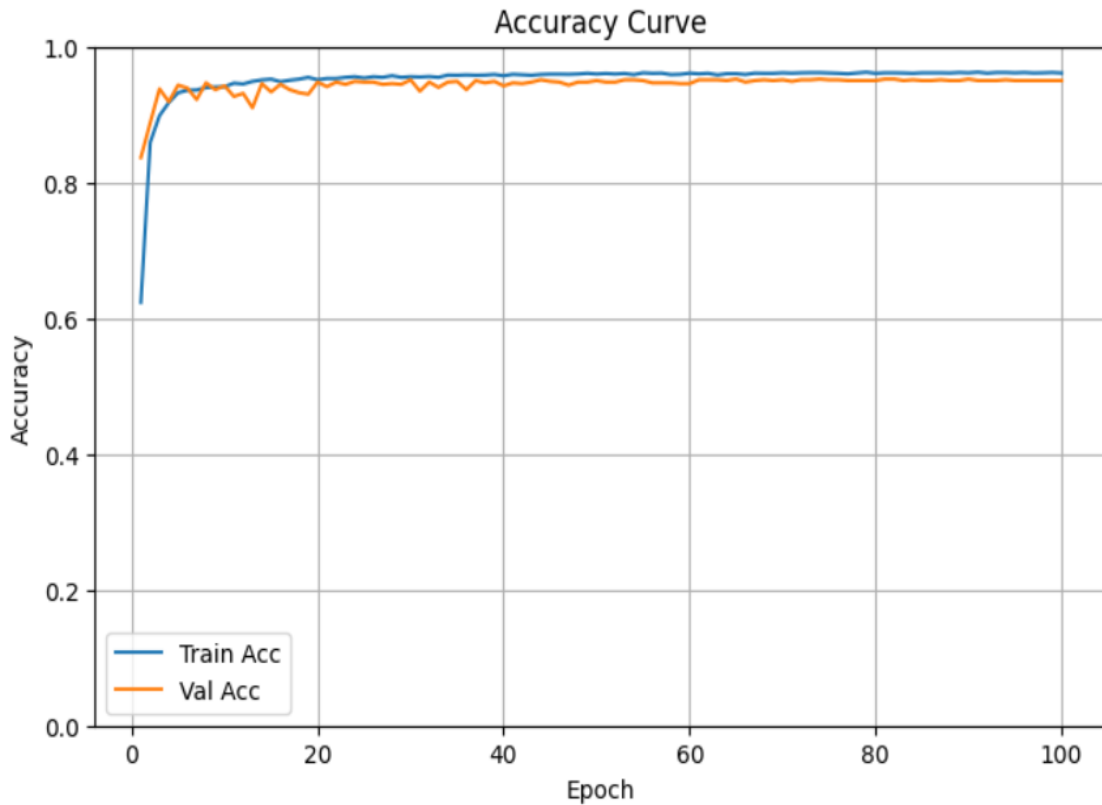
3.2.2.3 Huấn luyện mô hình

Quá trình huấn luyện mô hình được thực hiện với các thông số như sau:

- Batch size: 8
- Hàm tối ưu: Adam
- Hàm mất mát: Cross-Entropy

- Epoch: 100

Kết quả sau khi huấn luyện mô hình và Hình 3.3 biểu diễn sự thay đổi về độ chính xác của mô hình theo epoch trong quá trình huấn luyện



Hình 3.3 Biểu đồ sự thay đổi độ chính xác trên tập huấn luyện và đánh giá theo epoch

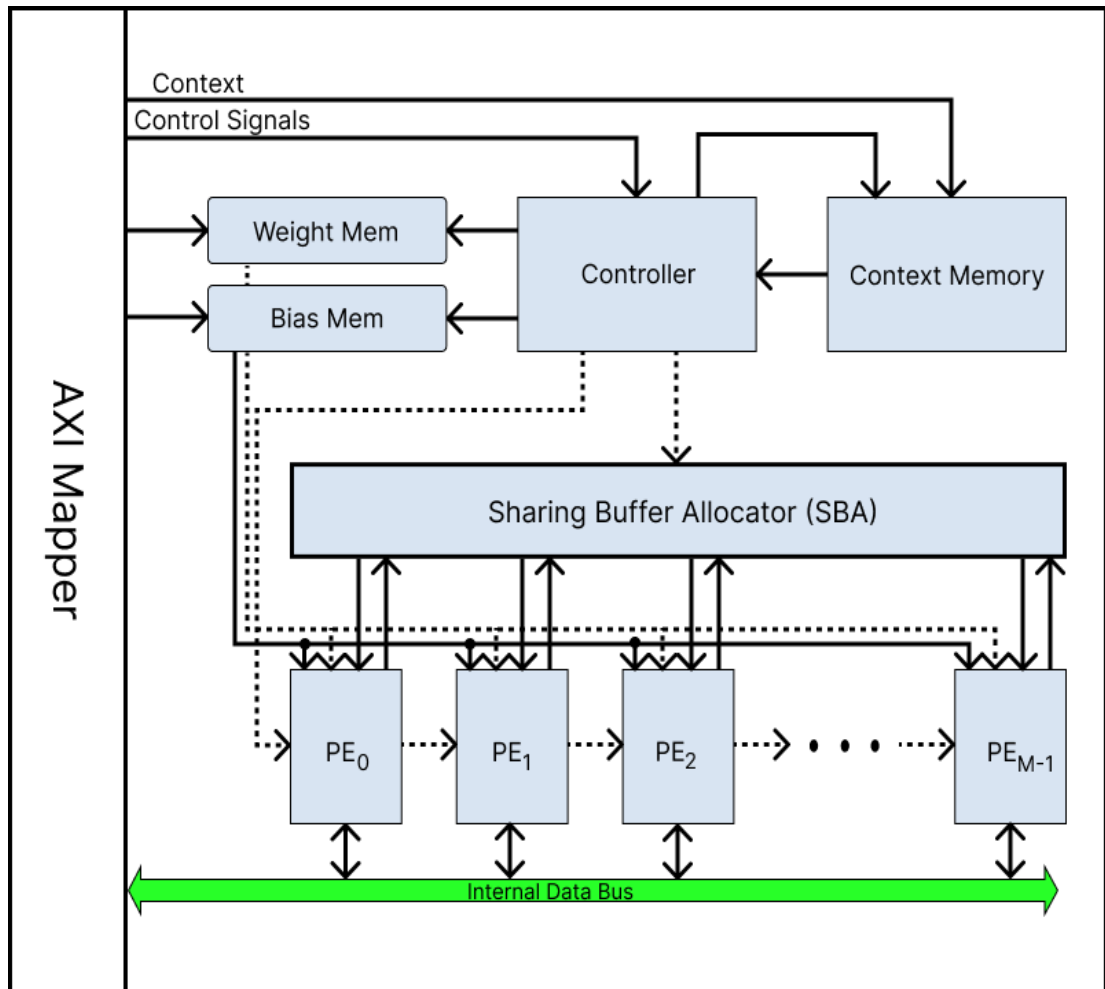
Theo Hình 3.3, mô hình đạt độ chính xác 96,12% trên tập huấn luyện và 95,00% trên tập đánh giá. Mặc dù tồn tại sự chênh lệch giữa hai giá trị này, mức chênh lệch không đáng kể, cho thấy mô hình chưa xuất hiện hiện tượng overfitting. Tuy nhiên, các kết quả này chỉ phản ánh hiệu suất của mô hình trên tập dữ liệu huấn luyện và tập đánh giá, chưa đủ để khẳng định khả năng tổng quát hóa hoặc hiệu quả khi áp dụng trên các dữ liệu thực tế chưa từng được quan sát.

Sau khi mô hình được huấn luyện, các trọng số và bias được trích xuất và lưu trữ trong bộ nhớ của FPGA nhằm phục vụ cho quá trình tính toán trong giai đoạn suy luận.

3.2.3 Kiến trúc phần cứng trên FPGA

3.2.3.1 Tổng quan về kiến trúc

Sơ đồ khối của kiến trúc trên FPGA như Hình 3.4



Hình 3.4 Kiến trúc của bộ xử lý trên FPGA

Dựa trên Hình 3.4, chức năng của từng thành phần trong kiến trúc được mô tả như sau:

- **Weight/Bias Memory:** Lưu trữ trọng số và bias của mô hình, cung cấp dữ liệu tham số cho các PE trong quá trình thực hiện các phép tính (đặc biệt là MAC trong Conv1D).
- **Controller:** Đóng vai trò bộ điều phối trung tâm, quản lý trình tự thực thi theo từng lớp/nhánh, phát tín hiệu điều khiển và đồng bộ hoạt động giữa các khối (bộ nhớ, SBA, PE).

- Context Memory: Lưu trữ tham số cấu hình của từng lớp CNN (ví dụ: kernel size, stride, số kênh vào/ra, chế độ tính toán...), giúp phần cứng thực thi linh hoạt nhiều lớp khác nhau bằng cách nạp/đọc cấu hình tương ứng.
- Sharing Buffer Allocator (SBA): Thực hiện phân phối và định tuyến dữ liệu giữa các PE, đảm bảo mỗi PE nhận đúng mẫu đầu vào cần thiết theo cơ chế cửa sổ trượt của Conv1D, đồng thời giảm truy cập bộ nhớ ngoài và tránh nghẽn dữ liệu.
- Processing Elements (PEs): Các khối xử lý chính thực hiện phép toán của mạng (MAC, cộng residual, ReLU, MaxPooling...), tạo ra feature map trung gian và kết quả cuối theo cấu hình được điều khiển bởi Controller. Kiến trúc sẽ có 20 PE thực thi xong xong cùng lúc.

3.2.3.2 Khối Controller

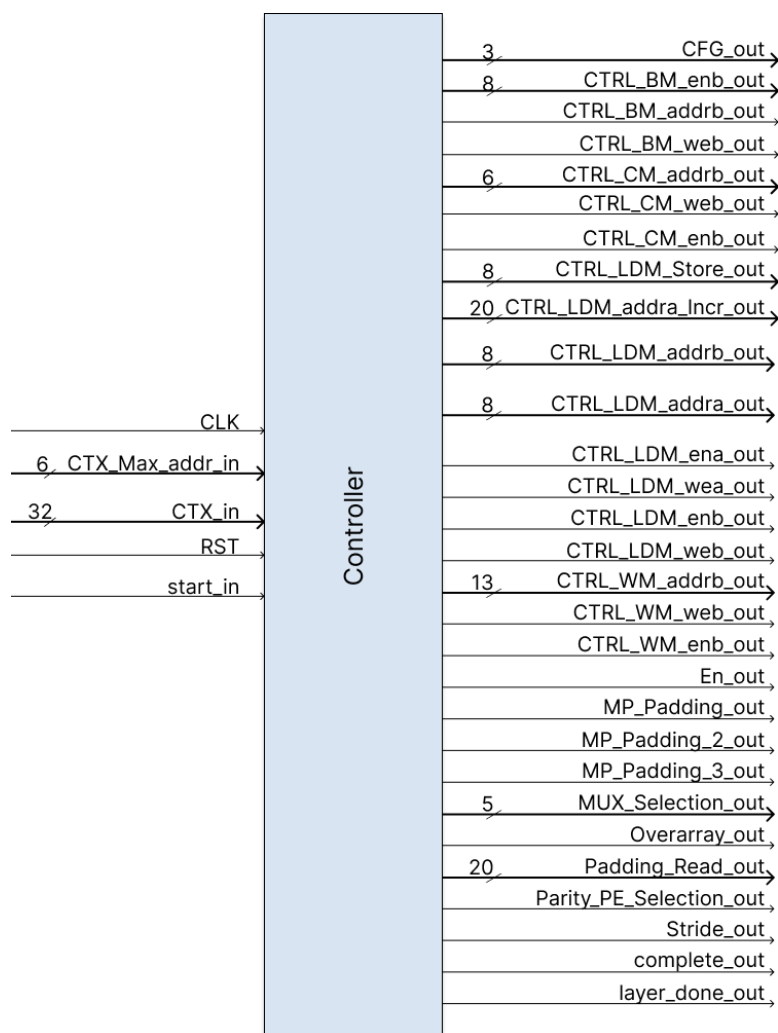
A. Chức năng

Khối Controller là thành phần quan trọng điều khiển hoạt động của hệ thống:

1. Lấy context từ CM: Nhận dữ liệu cấu hình lớp CNN từ CM.
2. Gửi các tín hiệu điều khiển đến PE: Điều khiển PE thực thi đúng với cấu hình được nhận.
3. Truyền địa chỉ dữ liệu đến WM và BM: Truyền địa chỉ phù hợp để WM và BM gửi trọng số và bias phù hợp đến các PE
4. Điều khiển SBA: Truyền các tín hiệu điều khiển để SBA phân phối dữ liệu đến các PE.

B. Sơ đồ khối

Sơ đồ khối Controller thể hiện như Hình 3.5



Hình 3.5 Sơ đồ khối Controller

Controller có nhiều output được thể hiện ở Hình 3.5 để phục vụ cho việc điều khiển các khối khác trong hệ thống. Cụ thể các tín hiệu inout và output được thể hiện trong Bảng 3.1 và Bảng 3.2

Bảng 3.1 Tín hiệu input của controller

Tín hiệu	Độ rộng	Nguồn	Mô tả
CLK	1	Hệ thống	Tín hiệu xung clock

RST	1	Hệ thống	Tín hiệu reset
CTX_in	32	CM	Dữ liệu cấu hình lớp hiện tại
CTX_Max_addr_in	6	Hệ thống	Cho biết khi nào kết thúc mô hình
Start in	1	Hệ thống	Tín hiệu bắt đầu thực thi

Bảng 3.2 Tín hiệu output của controller

Tín hiệu	Độ rộng	Đích	Mô tả
CFG_out	3	PEs; SBA	Phép toán cần thực thi ở lớp hiện tại
CTRL_BM_addrb_out	8	BM	Địa chỉ lưu trữ của bias trong BM
CTRL_BM_enb_out	1	BM	Tín hiệu cho phép truy cập BM
CTRL_BM_web_out	1	BM	Tín hiệu cho phép ghi vào BM
CTRL_CM_addrb_out	6	CM	Địa chỉ lưu trữ của context trong CM
CTRL_CM_enb_out	1	CM	Tín hiệu cho phép truy cập CM
CTRL_CM_web_out	1	CM	Tín hiệu cho phép ghi vào CM
CTRL_WM_addrb_out	13	WM	Địa chỉ lưu trữ weight trong WM

CTRL_WM_enb_out	1	WM	Tín hiệu cho phép truy cập WM
CTRL_WM_web_out	1	WM	Tín hiệu cho phép ghi vào WM
CTRL_LDM_Store_out	8	PEs	Cho biết nơi lưu trữ giá trị trong LDM
CTRL_LDM_addra_Incr_out	20	PEs	Tăng địa chỉ trong LDM của các PE được chọn
CTRL_LDM_addra_out	8	PEs	Địa chỉ của Pixel trong LDM (Port a)
CTRL_LDM_addrb_out	8	PEs	Địa chỉ của Pixel trong LDM (Port b)
CTRL_LDM_ena_out	1	PEs	Tín hiệu cho phép truy cập LDM (Port a)
CTRL_LDM_enb_out	1	PEs	Tín hiệu cho phép truy cập LDM (Port b)
CTRL_LDM_wea_out	1	PEs	Tín hiệu cho phép ghi dữ liệu vào LDM (Port a)
CTRL_LDM_web_out	1	PEs	Tín hiệu cho phép ghi dữ liệu vào LDM (Port b)
En_out	1	PEs; SBA	Tín hiệu bắt đầu PE và SBA
MP_Padding_out	1	SBA	Tín hiệu giúp phân phối dữ liệu khi có padding

MP_Padding_2_out	1	SBA	Tín hiệu giúp phân phối dữ liệu khi có padding ở biên phải
MP_Padding_3_out	1	SBA	Tín hiệu giúp phân phối dữ liệu khi có padding ở biên phải
Overarray_out	1	PE (last)	Tín hiệu báo vượt ngưỡng dữ liệu
MUX_Selection_out	5	SBA	Tín hiệu điều khiển phân bổ dữ liệu
Padding_read_out	20	PEs	Tín hiệu cho biết PE nào cần thêm padding
Parity_PE_Selection_out	1	PEs; SBA	Tín hiệu giúp chọn PE
Stride_out	1	PEs; SBA	Tín hiệu cho biết stride của lớp hiện tại
Layer_done_out	1	PEs	Tín hiệu cho biết hoàn thành một lớp trong mạng
Complete_out	1	Hệ thống	Tín hiệu thông báo việc tính toán hoàn tất

Bảng 3.2 và Bảng 3.1 trình bày chức năng của các chân input và output của khối Controller

C. Hoạt động của khối Controller

Khởi động: Khi $RST = 0$, Controller đưa các bộ đếm, thanh ghi địa chỉ và tín hiệu điều khiển về trạng thái khởi tạo. Khi nhận được tín hiệu `start_in`, hệ thống chuyển sang trạng thái chờ và bắt đầu đọc context từ CM.

Trong quá trình thực thi:

- Controller giải mã các tham số cấu hình của layer như loại phép toán, kích thước kernel, stride, padding và địa chỉ bộ nhớ. Dựa trên các thông tin này, Controller tạo các tín hiệu điều khiển để điều phối hoạt động của các khối phần cứng trong hệ thống.
- Đối với lớp Convolution, Controller điều khiển việc đọc dữ liệu từ LDM, truy xuất trọng số từ WM và bias từ BM, đồng thời kích hoạt các PE thực hiện phép tính MAC.
- Đối với lớp MaxPooling hoặc cộng, Controller cấu hình lại chế độ hoạt động của PE để thực hiện phép toán tương ứng. Bên cạnh đó, Controller cũng quản lý quá trình phân phối dữ liệu thông qua SBA và điều khiển việc ghi kết quả trở lại LDM.

Khi hoàn thành một lớp mạng: Controller cập nhật địa chỉ context để đọc cấu hình của lớp tiếp theo. Quá trình này được lặp lại cho đến khi toàn bộ mạng CNN được xử lý xong. Sau khi lớp cuối cùng hoàn tất, Controller phát tín hiệu kết thúc.

3.2.3.3 Khối Sharing Buffer Allocator (SBA)

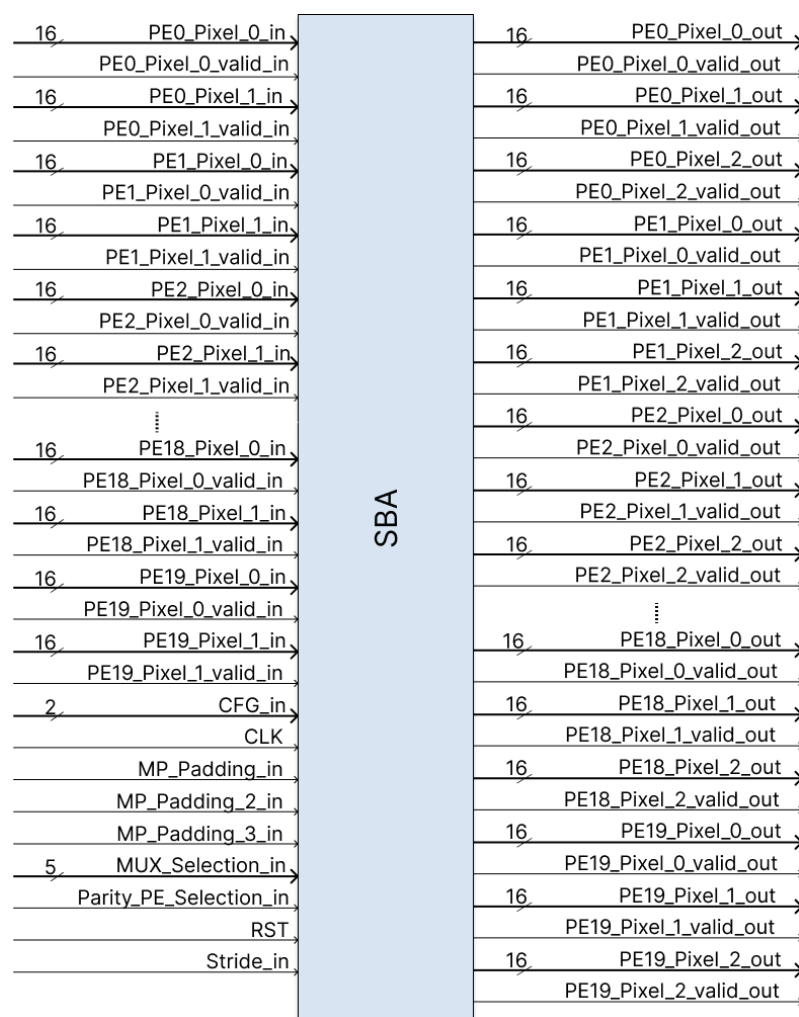
A. Chức năng

SBA là có nhiệm vụ phân phối dữ liệu giữa các PE:

1. Nhận các tín hiệu điều khiển từ Controller: Nhận và xử lý các tín hiệu điều khiển từ Controller để phân phối dữ liệu phù hợp đến các PE.
2. Nhận các dữ liệu được trả về từ PE: Truyền các dữ liệu được trả về từ PE lên bus chung để tái sử dụng cho lớp tiếp theo.
3. Phân phối dữ liệu đến các PE: Từ các tín hiệu điều khiển, phân phối các dữ liệu từ bus chung đến các PE tương ứng.

B. Sơ đồ khối

Sơ đồ khối SBA thể hiện như Hình 3.6



Hình 3.6 Sơ đồ khối của SBA

Hình 3.6 miêu tả chi tiết khối SBA gồm nhiều input và output. Chúng được mô tả chi tiết trong Bảng 3.3 và Bảng 3.4

Bảng 3.3 Các tín hiệu input của SBA

Tín hiệu	Độ rộng	Nguồn	Mô tả
CLK	1	Hệ thống	Tín hiệu xing clock
RST	1	Hệ thống	Tín hiệu reset
En_in	1	Controller	Tín hiệu bắt đầu thực thi

CFG_in	2	Controller	Phép toán cần thực thi ở lớp hiện tại
MP_Padding_in	1	Controller	Tín hiệu giúp phân phối dữ liệu khi có padding
MP_Padding_2_in	1	Controller	Tín hiệu giúp phân phối dữ liệu khi có padding ở biên phải
MP_Padding_3_in	1	Controller	Tín hiệu giúp phân phối dữ liệu khi có padding ở biên phải
MUX_Selection_in	5	Controller	Dữ liệu điều khiển việc phân phối
Parity_Selection_in	1	Controller	Tín hiệu chọn PE
Stride_in	1	Controller	Tín hiệu cho biết stride của lớp hiện tại
PE(0→19)_Pixel_0_in	16	PEs	Dữ liệu trả về từ PE
PE(0→19)_Pixel_0_valid_in	1	PEs	Tín hiệu báo có dữ liệu trả về
PE(0→19)_Pixel_1_in	16	PEs	Dữ liệu trả về từ PE
PE(0→19)_Pixel_1_valid_in	1	PEs	Tín hiệu báo có dữ liệu trả về

Bảng 3.4 Các tín hiệu output của SBA

Tín hiệu	Độ rộng	Đích	Mô tả
PE(0→19)_Pixel_0_out	16	PEs	Dữ liệu được phân phối đến PE

PE(0→19)_Pixel_0_valid_out	1	PEs	Tín hiệu báo có dữ liệu
PE(0→19)_Pixel_1_out	16	PEs	Dữ liệu được phân phối đến PE
PE(0→19)_Pixel_1_valid_out	1	PEs	Tín hiệu báo có dữ liệu
PE(0→19)_Pixel_2_out	16	PEs	Dữ liệu được phân phối đến PE
PE(0→19)_Pixel_2_valid_out	1	PEs	Tín hiệu báo có dữ liệu

Bảng 3.3 và Bảng 3.4 trình bày chức năng của các chân input và output của khối SBA

C. Hoạt động của khối SBA

Khởi động: Khi $RST = 0$, tất cả các tín hiệu output được đặt về 0. Khi $En_in = 1$, SBA bắt đầu trả dữ liệu

Hoạt động bình thường: Ở mỗi chu kì xung clock, SBA sẽ:

- Nhận và xử lí các tín hiệu điều khiển từ Controller
- Truyền dữ liệu cần xử lí đến các PE tương ứng: PEx_Pixel_0_out sẽ truyền các dữ liệu để tính toán ở các lớp Conv trong khi PEx_Pixel_1_out và PEx_Pixel_2_out sẽ cung cấp dữ liệu khi thực hiện tính toán ở các lớp MaxPooling và cộng cho Residual.
- Sau khi PE tính toán SBA sẽ nhận lại các giá trị tính toán đó và gửi lại lên bus chung để tái sử dụng cho lớp kế tiếp

3.2.3.4 Khối Processing Element (PE)

A. Chức năng

PE đảm nhận nhiệm vụ tính toán chính của hệ thống, mỗi PE tính toán độc lập một phần tử và các PE sẽ thực hiện song song:

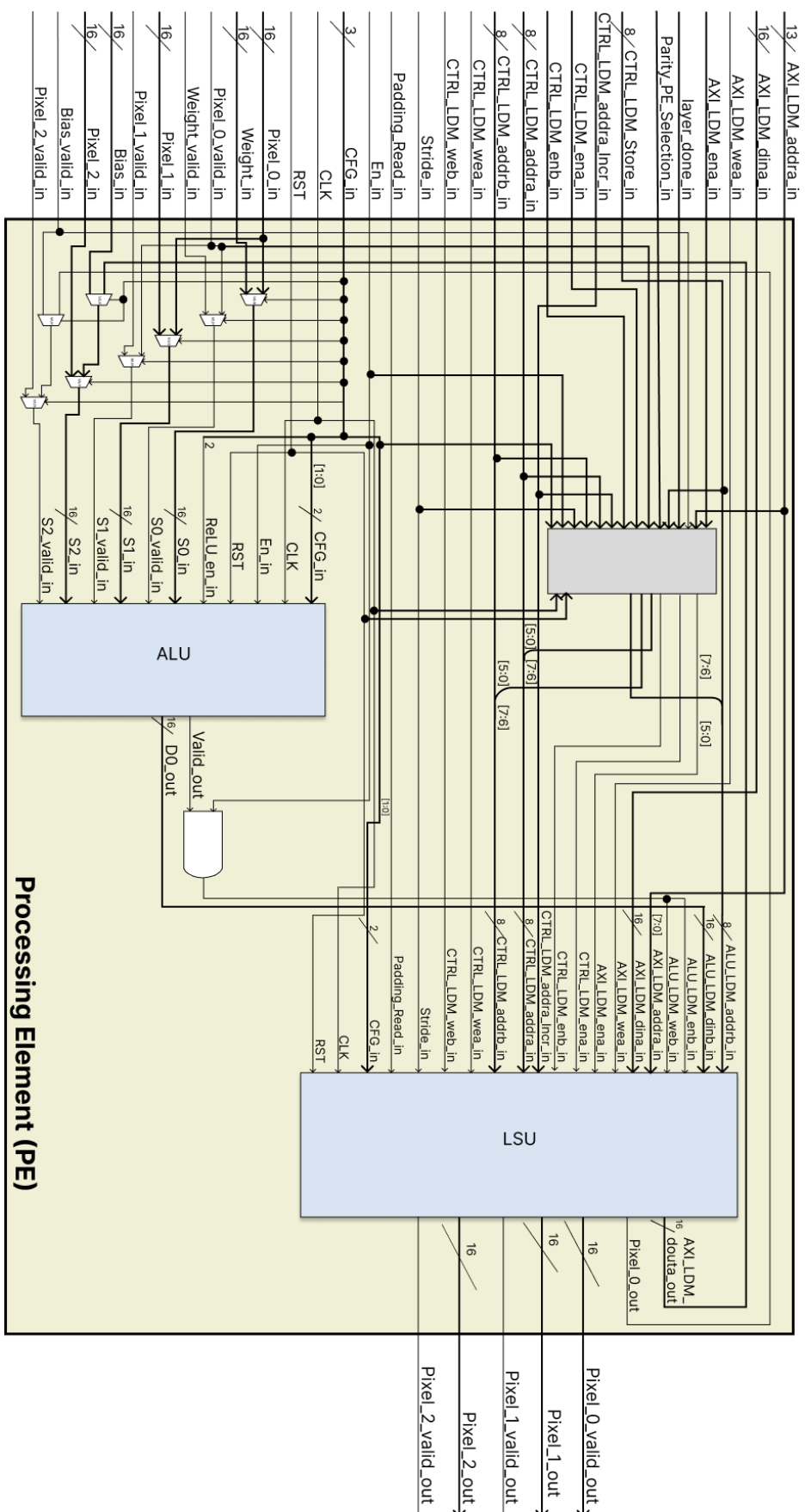
1. Nhận trọng số và bias: Nhận các trọng số và bias của mô hình từ WM và BM

2. Tính toán: Thực hiện tính toán ở từng lớp tương ứng và được điều khiển bởi Controller
3. Trả kết quả: Khi tính toán xong từng lớp thì đưa kết quả lên lại SBA hoặc trả kết quả ra hệ thống khi việc suy luận kết thúc.

Kết quả output sẽ được lưu trữ theo cơ chế round-robin trong các PE

B. Sơ đồ khối

Hình 3.7 thể hiện sơ đồ khối của PE



Hình 3.7 Sơ đồ khối của PE

Từ Hình 3.7, ta có thể thấy trong PE có hai khối chính đảm nhận chức năng tính toán và lưu trữ như sau:

1. ALU: khối tính toán chính của PE, thực hiện các phép toán MAC, ReLU, MaxPooling và cộng residual
2. LSU: khối chứa các LDM, có nhiệm vụ lưu trữ các output sau khi tính toán, việc lưu trữ trong các bộ nhớ nội bộ giúp giảm tình trạng tắc cổ chai khi lưu trong các bộ nhớ ngoài.

Các chân input và output của PE được nêu ở Bảng 3.5 và Bảng 3.6.

Bảng 3.5 Tín hiệu input của PE

Tín hiệu	Độ rộng	Nguồn	Mô tả
CLK	1	Hệ thống	Tín hiệu xung clock
RST	1	Hệ thống	Tín hiệu reset
EN_in	1	Controller	Tín hiệu bắt đầu thực thi
AXI_LDM_addra_in	13	Hệ thống	Địa chỉ dữ liệu input được hệ thống đưa vào
AXI_LDM_dina_in	16	Hệ thống	Dữ liệu input được hệ thống đưa vào
AXI_LDM_ena_in	1	Hệ thống	Tín hiệu cho phép truy cập bộ nhớ
AXI_LDM_wea_in	1	Hệ thống	Tín hiệu cho phép ghi
CTRL_LDM_Store_in	8	Controller	Địa chỉ lưu trữ kết quả trong LDM
CTRL_LDM_addra_Incr_in	1	Controller	Tăng địa chỉ trong LDM của các PE được chọn

CTRL_LDM_addr_a_in	8	Controller	Địa chỉ LDM cần đọc dữ liệu (port a)
CTRL_LDM_addr_b_in	8	Controller	Địa chỉ LDM cần đọc dữ liệu (port b)
CTRL_LDM_ena_in	1	Controller	Tín hiệu cho phép truy cập (port a)
CTRL_LDM_enb_in	1	Controller	Tín hiệu cho phép truy cập (port b)
CTRL_LDM_wea_in	1	Controller	Tín hiệu cho phép ghi (port a)
CTRL_LDM_web_in	1	Controller	Tín hiệu cho phép ghi (port b)
Bias_in	16	BM	Bias đi vào
Bias_valid_in	1	Controller	Tín hiệu báo có bias
Weight_in	16	WM	Trọng số đi vào
Weight_valid_in	1	Controller	Tín hiệu báo có trọng số
CFG_in	3	Controller	Phép toán cần thực thi ở lớp hiện tại
Padding_Read_in	1	Controller	PE nào cần thêm padding
Parity_PE_Selection_in	1	Controller	Tín hiệu giúp chọn PE
Stride_in	1	Controller	Tín hiệu cho biết stride của lớp hiện tại
Pixel_0_in	16	SBA	Dữ liệu đầu vào của lớp

Pixel_0_valid_in	1	SBA	Tín hiệu báo có dữ liệu
Pixel_1_in	16	SBA	Dữ liệu đầu vào của lớp
Pixel_1_valid_in	1	SBA	Tín hiệu báo có dữ liệu
Pixel_2_in	16	SBA	Dữ liệu đầu vào của lớp
Pixel_2_valid_in	1	SBA	Tín hiệu báo có dữ liệu
Layer_done_in	1	Controller	Tín hiệu báo kết thúc lớp

Bảng 3.6 Tín hiệu output của PE

Tín hiệu	Độ rộng	Đích	Mô tả
Pixel_0_out	16	SBA	Dữ liệu trả về SBA
Pixel_0_valid_out	1	SBA	Tín hiệu báo có dữ liệu
Pixel_1_out	16	SBA	Dữ liệu trả về SBA
Pixel_1_valid_out	1	SBA	Tín hiệu báo có dữ liệu
AXI_LDM_douta_out	16	Hệ thống	Dữ liệu trả về hệ thống khi quá trình tính toán kết thúc

Bảng 3.5 và Bảng 3.6 trình bày về các chân output, input cung với đó là mô tả của từng chân khối PE

C. Hoạt động của khối PE

Khởi động: Khi nhận được tín hiệu En_in từ Controller thì PE bắt đầu tính toán.

Khi hoạt động bình thường:

- Ở chế độ tính toán MAC, PE nhận được trọng số và bias theo trình tự nhất định từ WM và BM, cùng lúc cũng nhận được dữ liệu đầu vào được phân phối từ SBA để tính toán. Kết quả sẽ được lưu trữ trong LDM nhất định và đưa dữ liệu lên lại SBA để tái sử dụng

- Ở chế độ tính MaxPooling, PE nhận được các giá trị trong cửa sổ cần tính MaxPooling và tìm giá trị lớn nhất. Kết quả cũng sẽ được lưu trữ trong LDM và gửi lên SBA
- Ở chế độ thực hiện cộng residual, PE sẽ nhận được giá trị đầu vào từ SBA, đồng thời giá trị của lớp skip connection sẽ được lưu trong LDM3 sẽ được đưa ra để thực hiện phép cộng. Kết quả thu được sẽ được lưu trữ trong LDM khác và truyền lên SBA để tái sử dụng.

Khi kết thúc tính toán: PE sẽ trả kết quả cuối cùng về cho hệ thống để tiến hành các bước phân tích cuối cùng.

3.2.3.5 Khối Weight Memory (WM) và Bias Memory (BM)

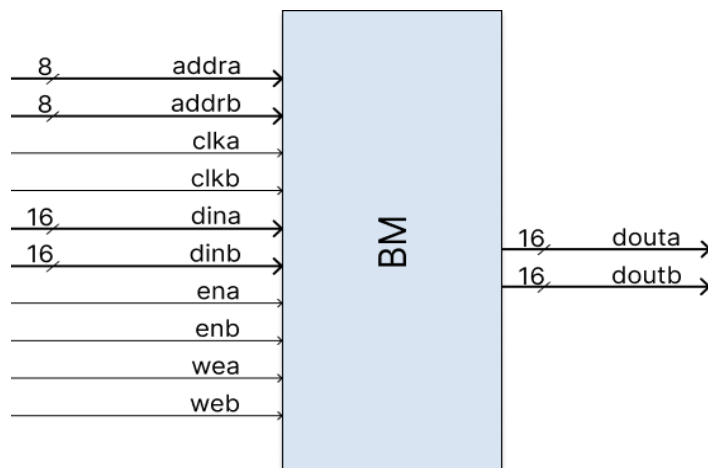
A. Chức năng

WM có chức năng lưu trữ những trọng số của mô hình CNN, đưa các trọng số đến PE khi nhận được tín hiệu từ Controller

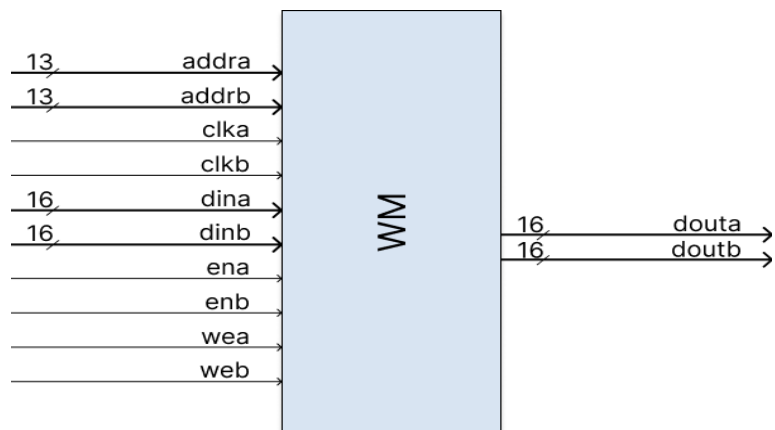
BM lưu trữ các bias của mô hình CNN và truyền bias đến PE khi có tín hiệu từ Controller

B. Sơ đồ khối

Sơ đồ khối của WM và BM như Hình 3.8



(a)



(b)

Hình 3.8 Sơ đồ khối của (a) BM và (b) WM

Hình 3.8 cho thấy WM/BM có hai port độc lập giúp cho việc truy cập thuận tiện hơn.

Bảng 3.7 miêu tả chi tiết cho các tín hiệu input của WM và BM.

Bảng 3.7 Tín hiệu input của WM/BM

Tín hiệu	Độ rộng	Nguồn	Mô tả
clka, clkb	1	Hệ thống	Tín hiệu xung clock
addra	8 (13)	Hệ thống	Địa chỉ vào (port a)
dina	16	Hệ thống	Dữ liệu vào (port a)
ena	1	Hệ thống	Tín hiện cho phép truy cập WM/BM (port a)
wea	1	Hệ thống	Tín hiệu cho phép ghi (port a)
addrb	8 (13)	Controller	Địa chỉ vào (port b)
dinb	16	0	Dữ liệu vào (port b)
enb	1	Controller	Tín hiện cho phép truy cập WM/BM (port b)
web	1	Controler	Tín hiệu cho phép ghi (port b)

Bảng 3.7 cho biết chức năng của chân input của hai khối WM và BM

Tín hiệu output của WM/BM: doutb có độ rộng 16-bit và được đưa tới các PE khi có tín hiệu từ Controller.

C. Hoạt động của khối BM/WM

Lúc khởi động lần đầu, các trọng số và bias của mô hình sẽ được hệ thống đưa vào WM/BM để lưu trữ thông qua port a.

Ở các lớp cần thực hiện tính toán MAC, WM và BM nhận được các địa chỉ dữ liệu và tín hiệu enable từ Controller thông qua port b và sẽ gửi lại các giá trị lưu trữ đến PE để thực hiện tính toán

3.2.3.6 Khối Context Memory (CM)

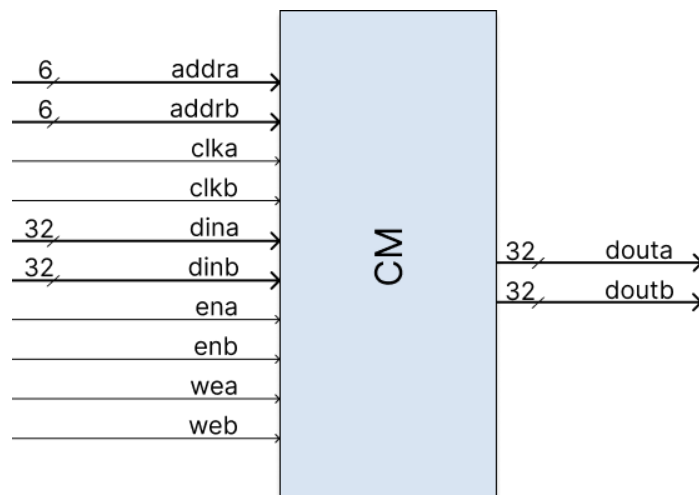
A. Chức năng

CM có chức năng lưu trữ các giá trị cấu hình của mô hình 1-D CNN, được gọi là context và truyền đến Controller khi nhận được tín hiệu. Các context này quy định các tham số cấu hình của lớp mạng, địa chỉ LDM để đọc/ghi dữ liệu.

Các giá trị này tạo hình một chuỗi 32-bit và nó đại diện cho một lớp trong mô hình mạng.

B. Sơ đồ khối

Hình 3.9 thể hiện sơ đồ khối CM



Hình 3.9 Sơ đồ khối của CM

Hình 3.9 cho thấy CM có hai port độc lập giúp cho việc truy cập thuận tiện hơn.

Bảng 3.8 miêu tả chi tiết cho các tín hiệu input của CM.

Bảng 3.8 Tín hiệu input của CM

Tín hiệu	Độ rộng	Nguồn	Mô tả
clka, clkb	1	Hệ thống	Tín hiệu xung clock
addra	6	Hệ thống	Địa chỉ vào (port a)
dina	32	Hệ thống	Dữ liệu vào (port a)
ena	1	Hệ thống	Tín hiệu cho phép truy cập CM (port a)
wea	1	Hệ thống	Tín hiệu cho phép ghi (port a)
addrb	6	Controller	Địa chỉ vào (port b)
dinb	32	0	Dữ liệu vào (port b)
enb	1	Controller	Tín hiệu cho phép truy cập CM (port b)
web	1	Controller	Tín hiệu cho phép ghi (port b)

Bảng 3.8 cho biết chức năng của chân input của hai khối CM

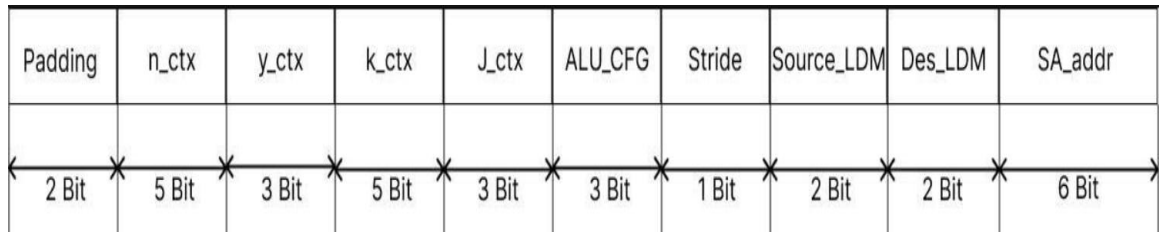
Tín hiệu output của CM: doutb có độ rộng 32-bit và được đưa tới các Controller khi có tín hiệu từ Controller.

C. Hoạt động của khối CM

Khi khởi động hệ thống lần đầu, các dữ liệu context sẽ được truyền vào CM từ hệ thống thông qua port a.

Trong quá trình hoạt động của hệ thống, khi bắt đầu một lớp mạng mới, Controller sẽ gửi tín hiệu truy cập và địa chỉ cần truy cập đến CM, khi nhận được tín hiệu thì CM sẽ trả dữ liệu về cho Controller để giải mã.

Chuỗi context này có 32-bit và chứa các thông tin của lớp mạng như Hình 3.10



Hình 3.10 Các thông tin chứa trong một dữ liệu context

Chuỗi context 32-bit này gồm:

- Padding [31:30]: Chứa dữ liệu cho biết lớp này có padding là bao nhiêu.
- n_ctx [29:25]: Số lượng kernel.
- y_ctx [24:22]: Là thương là số lượng output chia cho số lượng PE.
- k_ctx [21:17]: Số lượng channel ngõ vào.
- J_ctx [16:14]: Kích thước kernel.
- ALU_CFG [13:11]: Phép toán cần thực hiện.
- Stride [10]: Bước trượt của kernel.
- Source_LDM [9:8]: LDM nơi lưu input.
- Des_LDM [7:6]: LDM nơi lưu output.
- SA_addr [5:0]: Địa chỉ bắt đầu lưu output trong LDM.

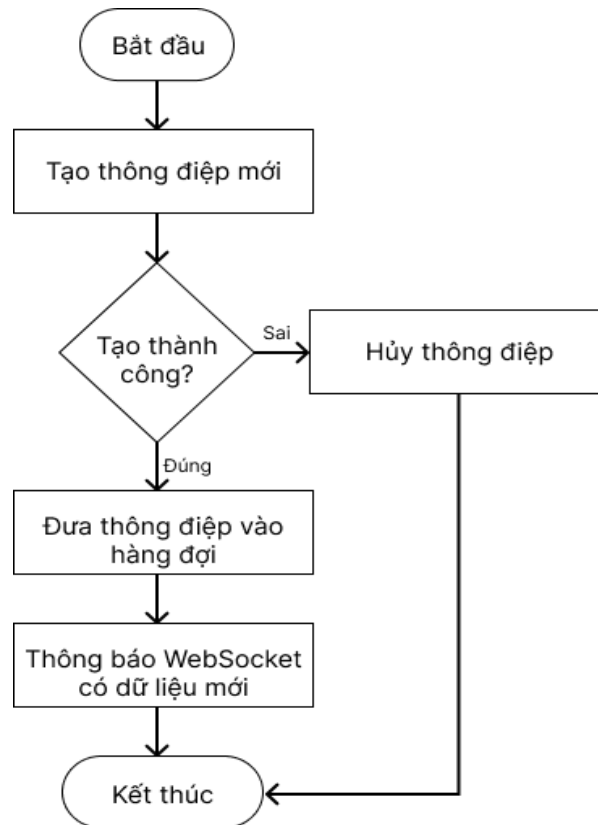
3.2.4 Phần mềm trên FPGA

Chương trình C thực hiện điều khiển toàn bộ quá trình xử lý và giao tiếp với bộ xử lý CNN trên FPGA. Chương trình chịu trách nhiệm nạp các tham số mô hình, cung cấp dữ liệu đầu vào, kích hoạt quá trình tính toán và thu nhận kết quả từ phần cứng. Sau khi nhận được dữ liệu đầu ra từ FPGA, chương trình thực hiện các bước hậu xử lý cần thiết để tạo ra kết quả phân loại cuối cùng. Đồng thời, chương trình còn đảm nhiệm việc trao đổi dữ liệu với giao diện web thông qua cơ chế WebSocket, cho phép truyền tín hiệu đầu vào và kết quả dự đoán theo thời gian thực. Nhờ đó, người dùng có thể theo dõi trực quan quá trình hoạt động của hệ thống trên giao diện web trong khi vẫn đảm bảo hiệu năng xử lý.

3.2.4.1 Hàm ws_enqueue()

Nhiệm vụ: Dùng để lưu các thông điệp cần gửi vào hàng đợi WebSocket và kích hoạt quá trình gửi dữ liệu tới client.

Lưu đồ giải thuật của hàm ws_enqueue() được biểu diễn như Hình 3.11.



Hình 3.11 Lưu đồ giải thuật hàm ws_enqueue()

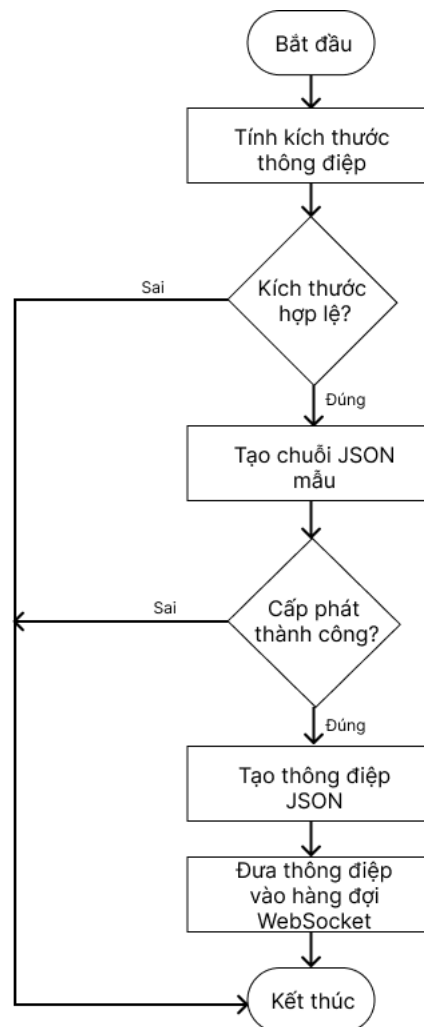
Lưu đồ Hình 3.11 mô tả quá trình tạo và đưa một thông điệp mới vào hàng đợi WebSocket để chuẩn bị gửi đến web. Khi hàm được gọi, hệ thống bắt đầu bằng việc tạo một thông điệp mới, bao gồm nội dung cần truyền và các thông tin cần thiết để quản lý thông điệp trong hàng đợi. Sau đó, hệ thống kiểm tra quá trình tạo thông điệp có thành công hay không. Nếu việc tạo thông điệp thất bại, thông điệp sẽ được hủy để giải phóng tài nguyên và quá trình kết thúc. Ngược lại, nếu thông điệp được tạo thành công, nó sẽ được đưa vào hàng đợi chờ gửi. Sau khi thông điệp đã được lưu vào hàng đợi, hệ thống thông báo cho WebSocket rằng có dữ liệu mới cần truyền. Khi kết nối WebSocket sẵn sàng gửi, thông điệp trong hàng đợi sẽ được lấy ra và truyền đến web. Cơ chế này giúp tách biệt quá trình tạo dữ liệu và

quá trình gửi dữ liệu, đảm bảo việc truyền thông tin diễn ra ổn định và phù hợp với cơ chế xử lý sự kiện của WebSocket.

3.2.4.2 Hàm send_data()

Nhiệm vụ: Hàm send_data() có nhiệm vụ tạo một thông điệp JSON theo định dạng WebSocket từ loại dữ liệu và nội dung, sau đó đưa thông điệp này vào hàng đợi.

Hình 3.12 miêu tả hoạt động của hàm send_data():



Hình 3.12 Lưu đồ giải thuật hàm send_data()

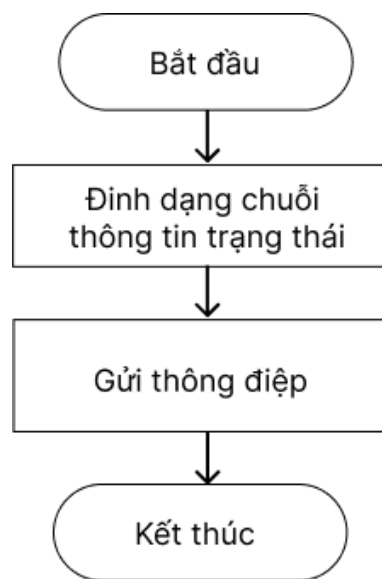
Lưu đồ Hình 3.12 mô tả quá trình tạo và đưa một thông điệp JSON vào hàng đợi WebSocket để phục vụ việc truyền dữ liệu đến web. Đầu tiên, hệ thống tính toán kích thước cần thiết của thông điệp nhằm xác định dung lượng bộ nhớ cần cấp phát. Sau đó, kích thước này được kiểm tra để đảm bảo tính hợp lệ trước khi

tiếp tục xử lý. Nếu kích thước không hợp lệ, quá trình sẽ dừng và kết thúc. Khi kích thước hợp lệ, hệ thống tiến hành khởi tạo cấu trúc JSON và thực hiện cấp phát bộ nhớ động để lưu trữ thông điệp. Trường hợp cấp phát bộ nhớ thất bại, quá trình cũng sẽ kết thúc để tránh phát sinh lỗi trong quá trình thực thi. Nếu việc cấp phát thành công, nội dung JSON sẽ được tạo dựa trên dữ liệu cần truyền, sau đó thông điệp được đưa vào hàng đợi WebSocket. Việc sử dụng hàng đợi giúp tách biệt quá trình tạo dữ liệu và quá trình truyền dữ liệu, đảm bảo các thông điệp được lưu trữ tạm thời và gửi đến web khi kết nối WebSocket sẵn sàng, từ đó nâng cao tính ổn định và độ tin cậy của hệ thống.

3.2.4.3 Hàm `send_status()`

Nhiệm vụ: Gửi thông báo trạng thái (khởi tạo, lỗi, hoàn tất...).

Hình 3.13 miêu tả lưu đồ của hàm `send_status()` như sau:



Hình 3.13 Lưu đồ giải thuật hàm `send_status()`

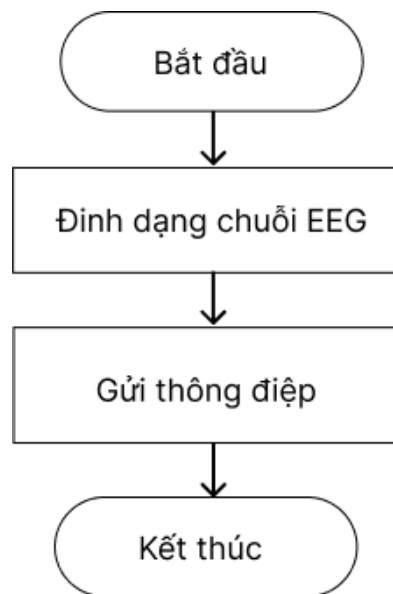
Hình 3.13 mô tả quá trình tạo và gửi một thông điệp chứa thông tin trạng thái của hệ thống. Khi hàm được gọi, hệ thống tiến hành định dạng các thông tin trạng thái thành một chuỗi dữ liệu theo cấu trúc ở dạng JSON. Chuỗi này chứa các thông tin cần truyền đến. Sau khi hoàn tất quá trình định dạng, thông điệp được gửi thông qua kết nối WebSocket đến web đang kết nối. Quá trình kết thúc sau khi yêu cầu

gửi dữ liệu được thực hiện, cho phép web nhận được thông tin trạng thái mới nhất của hệ thống một cách kịp thời

3.2.4.4 Hàm `send_sample()`

Nhiệm vụ: Gửi từng mẫu EEG lên web để vẽ đồ thị theo thời gian thực.

Lưu đồ giải thuật của hàm `send_sample` như Hình 3.14



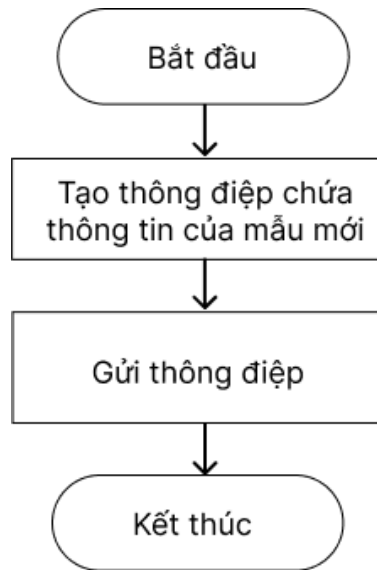
Hình 3.14 Lưu đồ giải thuật hàm `send_sample()`

Hình 3.14 mô tả quá trình tạo và gửi dữ liệu tín hiệu EEG đến web thông qua kết nối WebSocket. Khi hàm được gọi, các mẫu dữ liệu EEG sẽ được chuyển đổi và định dạng thành một chuỗi dữ liệu. Quá trình định dạng bao gồm việc sắp xếp dữ liệu và đóng gói các giá trị EEG thành một thông điệp hoàn chỉnh. Sau khi hoàn tất, thông điệp được gửi đến web đang kết nối thông qua WebSocket, cho phép dữ liệu EEG được truyền theo thời gian thực để phục vụ mục đích giám sát và hiển thị. Quá trình kết thúc sau khi yêu cầu gửi dữ liệu được thực hiện thành công.

3.2.4.5 Hàm `send_beat_start()`

Nhiệm vụ: Hàm `send_beat_start()` có nhiệm vụ tạo thông điệp chứa thông tin khởi đầu của một mẫu, bao gồm mã số beat, vị trí bắt đầu, nhãn thực tế và trạng thái dự đoán ban đầu, sau đó chuyển thông điệp này cho hàm `send_data()`.

Hình 3.15 thể hiện lưu đồ giải thuật của hàm `send_beat_start()`



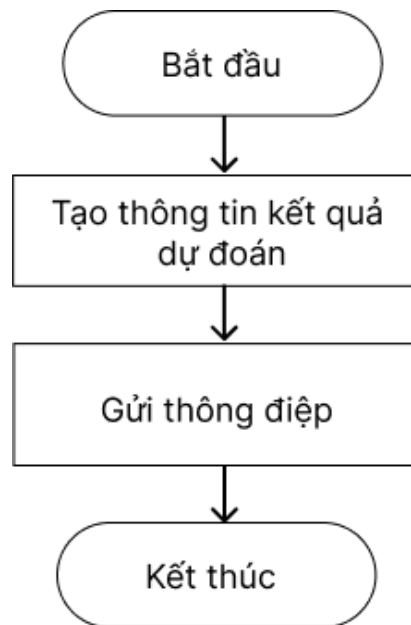
Hình 3.15 Lưu đồ giải thuật của hàm `send_beat_start()`

Hình 3.15 mô tả quá trình tạo và gửi thông điệp chứa thông tin của một mẫu dữ liệu mới đến web thông qua kết nối WebSocket. Khi hàm được gọi, hệ thống tiến hành tạo một thông điệp chứa các thông tin của mẫu dữ liệu vừa được thu thập và xử lý. Thông điệp này được xây dựng theo định dạng quy định, bao gồm các trường dữ liệu cần thiết để phía web có thể nhận biết và xử lý chính xác nội dung được truyền. Sau khi hoàn tất quá trình tạo thông điệp, dữ liệu được gửi đến web đang kết nối thông qua WebSocket. Quá trình kết thúc sau khi yêu cầu gửi thông điệp được thực hiện.

3.2.4.6 Hàm `send_beat_pred()`

Nhiệm vụ: Hàm `send_beat_pred()` có nhiệm vụ tạo thông điệp chứa kết quả dự đoán của một beat sau khi quá trình suy luận hoàn tất, bao gồm nhãn dự đoán, tên nhãn, độ chính xác hiện tại và thống kê số mẫu dự đoán đúng, sau đó chuyển thông điệp này cho hàm `send_data()`.

Lưu đồ giải thuật của hàm `send_beat_pred()` như Hình 3.16



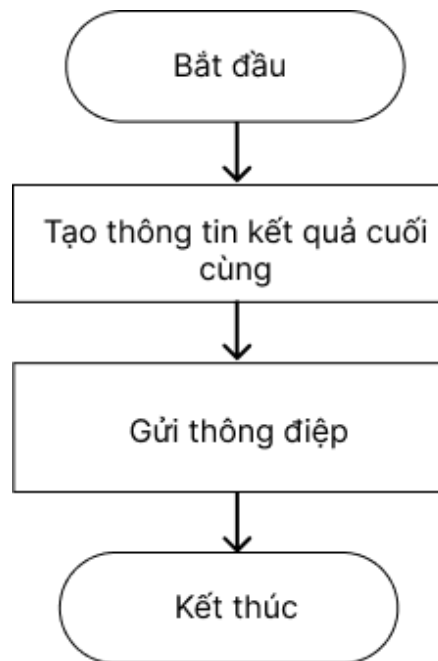
Hình 3.16 Lưu đồ giải thuật của hàm `send_beat_pred()`

Hình 3.16 mô tả quá trình tạo và gửi thông điệp chứa kết quả dự đoán của mẫu dữ liệu hiện tại đến web thông qua kết nối WebSocket. Khi hàm được gọi, hệ thống tiến hành tạo thông tin kết quả dự đoán dựa trên mẫu dữ liệu vừa được xử lý. Sau khi thông điệp được tạo theo định dạng quy định, dữ liệu sẽ được gửi đến web đang kết nối thông qua WebSocket. Nhờ đó, kết quả dự đoán có thể được cập nhật và hiển thị theo thời gian thực trên giao diện người dùng, giúp quá trình giám sát và đánh giá tín hiệu được thực hiện liên tục. Quá trình kết thúc sau khi yêu cầu gửi thông điệp được thực hiện.

3.2.4.7 Hàm `send_result()`

Nhiệm vụ: Hàm `send_result()` có nhiệm vụ tạo thông điệp chứa kết quả tổng hợp sau khi hoàn thành quá trình xử lý toàn bộ dữ liệu, bao gồm tổng số mẫu, số mẫu dự đoán đúng, độ chính xác và thời gian thực thi, sau đó chuyển thông điệp này cho hàm `send_data()`

Hình 3.17 thể hiện lưu đồ giải thuật của hàm `send_result()`

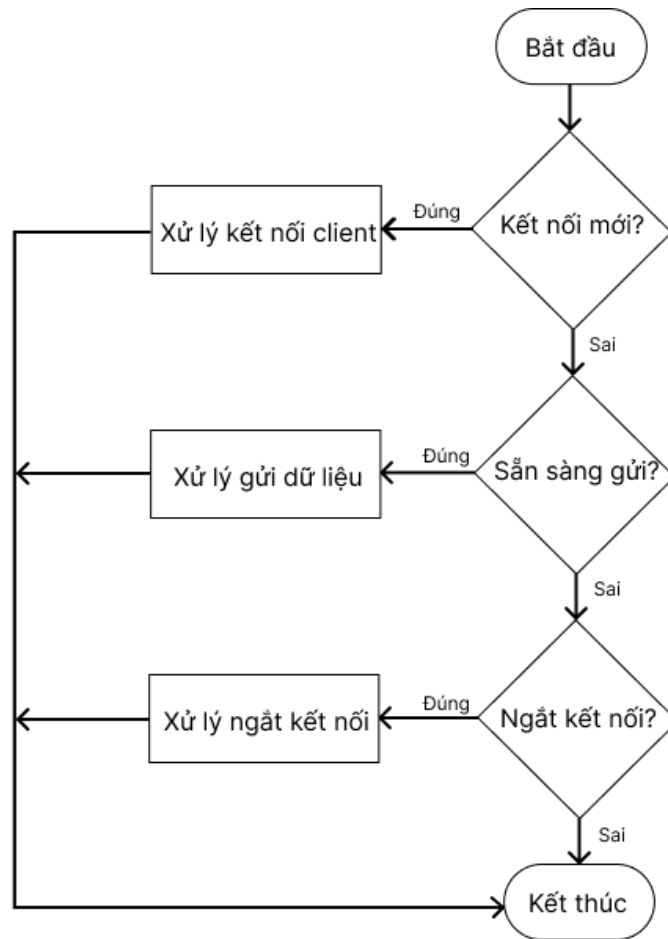


Hình 3.17 Lưu đồ giải thuật của hàm `send_result()`

Hình 3.17 mô tả quá trình tạo và gửi thông điệp chứa kết quả cuối cùng của quá trình xử lý đến web thông qua kết nối WebSocket. Khi hàm được gọi, hệ thống tổng hợp các thông tin cần thiết sau khi quá trình xử lý hoặc phân loại dữ liệu đã hoàn tất. Sau khi dữ liệu được định dạng thành thông điệp theo cấu trúc quy định, hệ thống thực hiện gửi thông điệp đến web đang kết nối. Cơ chế này giúp phía web nhận được kết quả tổng kết cuối cùng một cách đầy đủ và kịp thời, phục vụ cho việc hiển thị, theo dõi và đánh giá kết quả xử lý của hệ thống. Quá trình kết thúc sau khi yêu cầu gửi thông điệp được thực hiện.

3.2.4.8 Hàm `callback_eeg()`

Nhiệm vụ: Hàm `callback_eeg()` có nhiệm vụ xử lý các sự kiện WebSocket như thiết lập kết nối, gửi dữ liệu đến client và đóng kết nối như biểu diễn trong lưu đồ ở Hình 3.18.

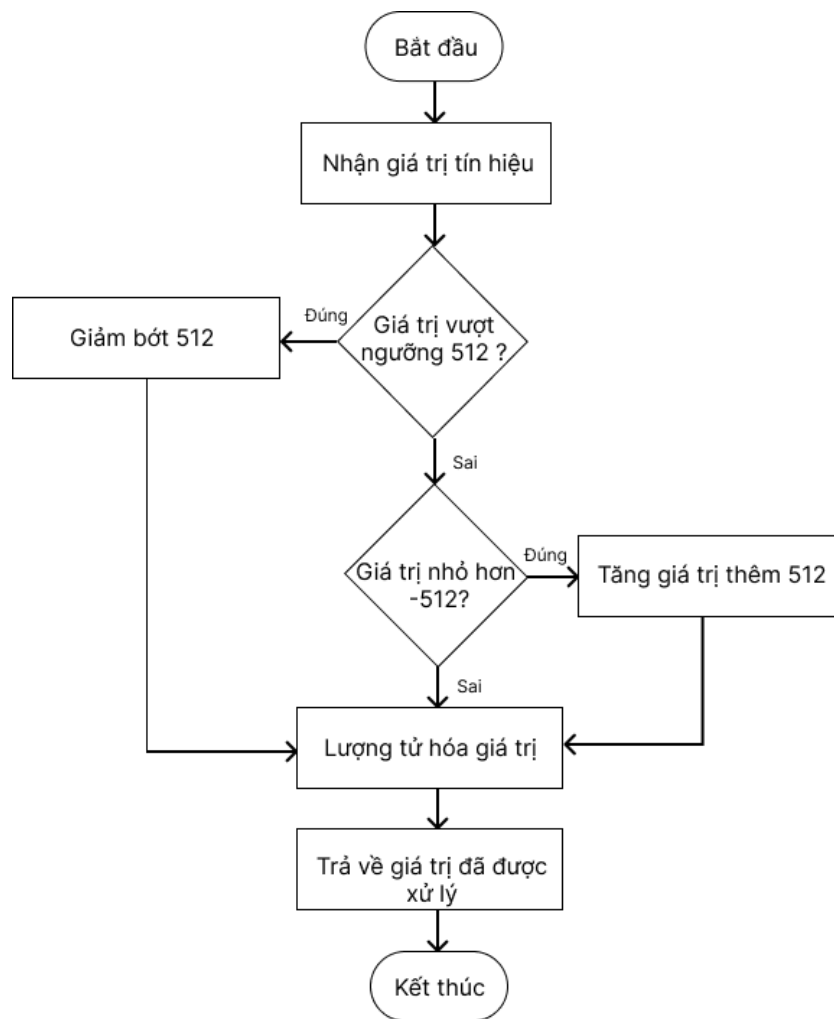


Hình 3.18 Lưu đồ giải thuật của hàm `callback_eeg()`

Lưu đồ Hình 3.18 mô tả quá trình xử lý các sự kiện chính trong hàm `callback` của WebSocket. Khi hàm được gọi, hệ thống lần lượt kiểm tra loại sự kiện nhận được. Trước tiên, nếu sự kiện là kết nối mới từ client, hệ thống sẽ thực hiện xử lý kết nối client, chẳng hạn như lưu thông tin client hoặc cập nhật trạng thái kết nối. Nếu không phải sự kiện kết nối mới, hệ thống tiếp tục kiểm tra trạng thái sẵn sàng gửi dữ liệu. Khi WebSocket sẵn sàng gửi, hệ thống sẽ xử lý việc truyền dữ liệu, thường là lấy thông điệp từ hàng đợi và gửi đến client. Nếu sự kiện không phải sẵn sàng gửi, hệ thống tiếp tục kiểm tra sự kiện ngắt kết nối. Trong trường hợp client ngắt kết nối, hệ thống thực hiện các thao tác dọn dẹp cần thiết như xóa thông tin client và cập nhật trạng thái kết nối. Nếu sự kiện không thuộc các trường hợp trên, hàm sẽ kết thúc mà không thực hiện thêm xử lý. Cách tổ chức này giúp hàm `callback` phản hồi đúng từng loại sự kiện WebSocket và đảm bảo quá trình kết nối, truyền dữ liệu và ngắt kết nối được quản lý rõ ràng.

3.2.4.9 Hàm fx_preprocess()

Nhiệm vụ; Tiền xử lý dữ liệu đầu vào bằng cách giới hạn giá trị trong miền biểu diễn của fixed-point và lượng tử hóa dữ liệu về độ phân giải 1/64 trước khi chuyển sang định dạng fixed-point. Quá trình thực hiện hàm fx_preprocess() như lưu đồ Hình 3.19.



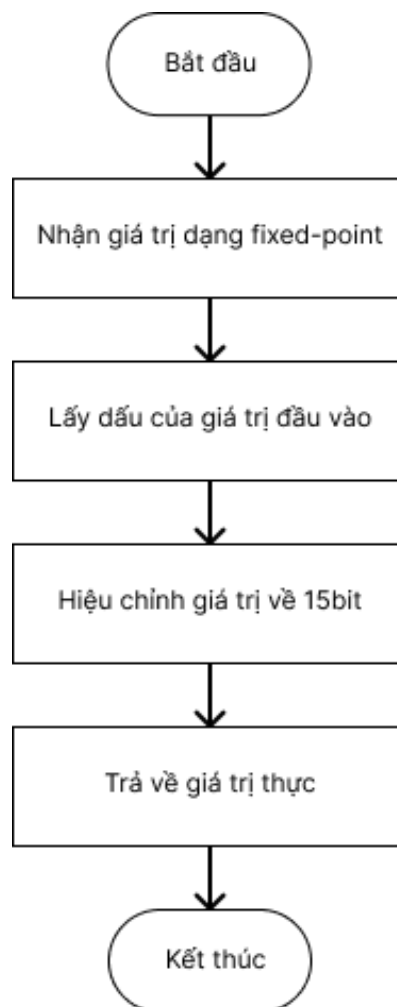
Hình 3.19 Lưu đồ giải thuật hàm fx_preprocess()

Hình 3.19 mô tả quá trình tiền xử lý và lượng tử hóa giá trị tín hiệu đầu vào trước khi đưa vào các bước xử lý tiếp theo. Khi hàm được gọi, hệ thống nhận một giá trị tín hiệu cần xử lý và tiến hành kiểm tra biên độ của giá trị này. Nếu giá trị tín hiệu vượt ngưỡng 512, hệ thống sẽ giảm giá trị đi 512 nhằm đưa dữ liệu về khoảng biểu diễn phù hợp. Ngược lại, nếu giá trị nhỏ hơn -512, hệ thống sẽ cộng thêm 512 để hiệu chỉnh lại giá trị. Sau khi hoàn tất bước kiểm tra và hiệu chỉnh biên độ, giá trị được đưa qua bước lượng tử hóa nhằm chuyển đổi về dạng fixel-

point 16-bit với độ phân giải xác định. Cuối cùng, hàm trả về giá trị tín hiệu đã được xử lý, đảm bảo dữ liệu đầu vào nằm trong miền giá trị phù hợp và có định dạng nhất quán cho quá trình tính toán trên phần cứng.

3.2.4.10 Hàm `fx_to_float()`

Nhiệm vụ: Chuyển đổi dữ liệu fixed-point 16-bit từ PL sang giá trị số thực để PS có thể tiếp tục xử lý. Quá trình thực hiện hàm `fx_to_float()` theo như lưu đồ ở Hình 3.20.



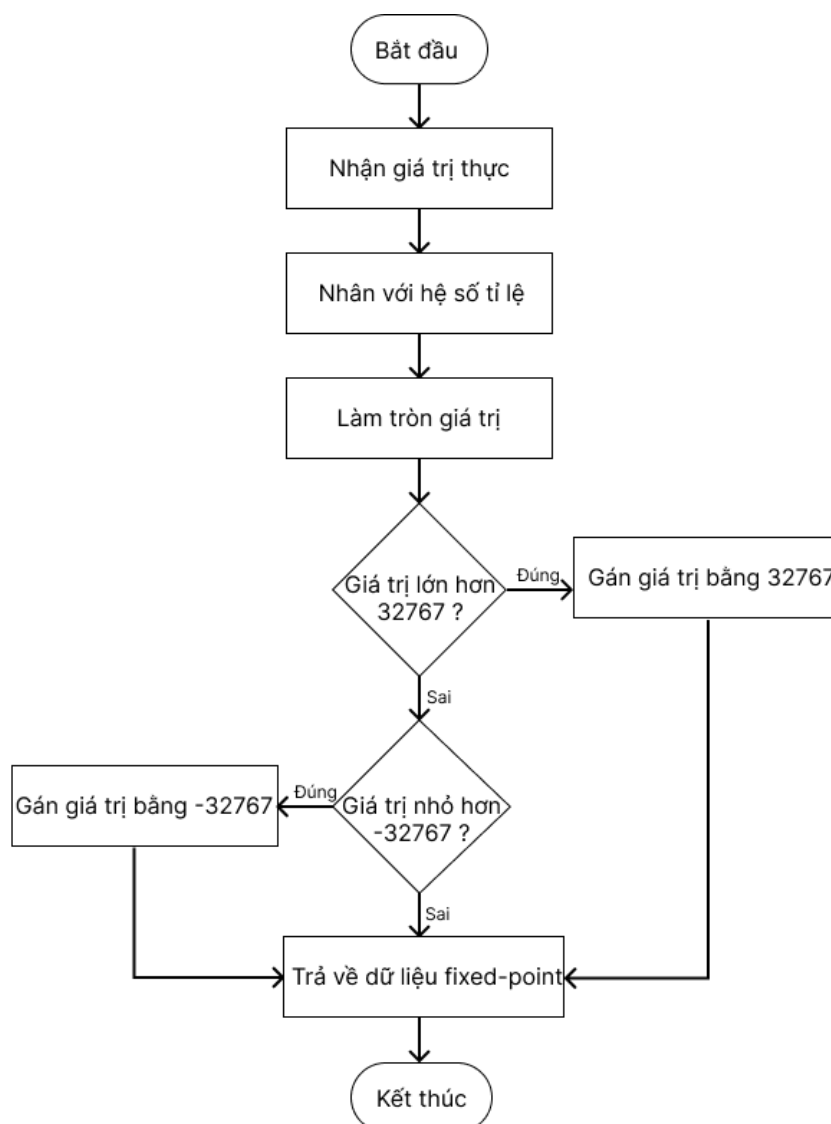
Hình 3.20 Lưu đồ giải thuật hàm `fx_to_float()`

Hình 3.20 mô tả quá trình chuyển đổi một giá trị ở dạng fixed-point sang giá trị thực để phục vụ cho việc kiểm tra và hiển thị kết quả tính toán. Khi hàm được gọi, hệ thống nhận vào một giá trị dữ liệu dạng fixed-point 16-bit. Tiếp theo, dấu của giá trị đầu vào được xác định nhằm phân biệt số dương và số âm. Sau đó, giá trị

được hiệu chỉnh về miền biểu diễn 15-bit. Cuối cùng, giá trị sau khi hiệu chỉnh được quy đổi về dạng số thực dựa trên hệ số tỉ lệ của định dạng fixed-point dạng Q10.6 thì giá trị được chia cho ($2^6 = 64$). Hàm trả về giá trị thực sau chuyển đổi, giúp dữ liệu phần cứng có thể được đọc, phân tích và so sánh thuận tiện ở phía phần mềm.

3.2.4.11 Hàm `fx_from_float()`

Nhiệm vụ: Chuyển đổi dữ liệu số thực sang định dạng fixed-point 16-bit để ghi xuống FPGA. Lưu đồ hình 3.21 thể hiện hàm `fx_from_float()`.

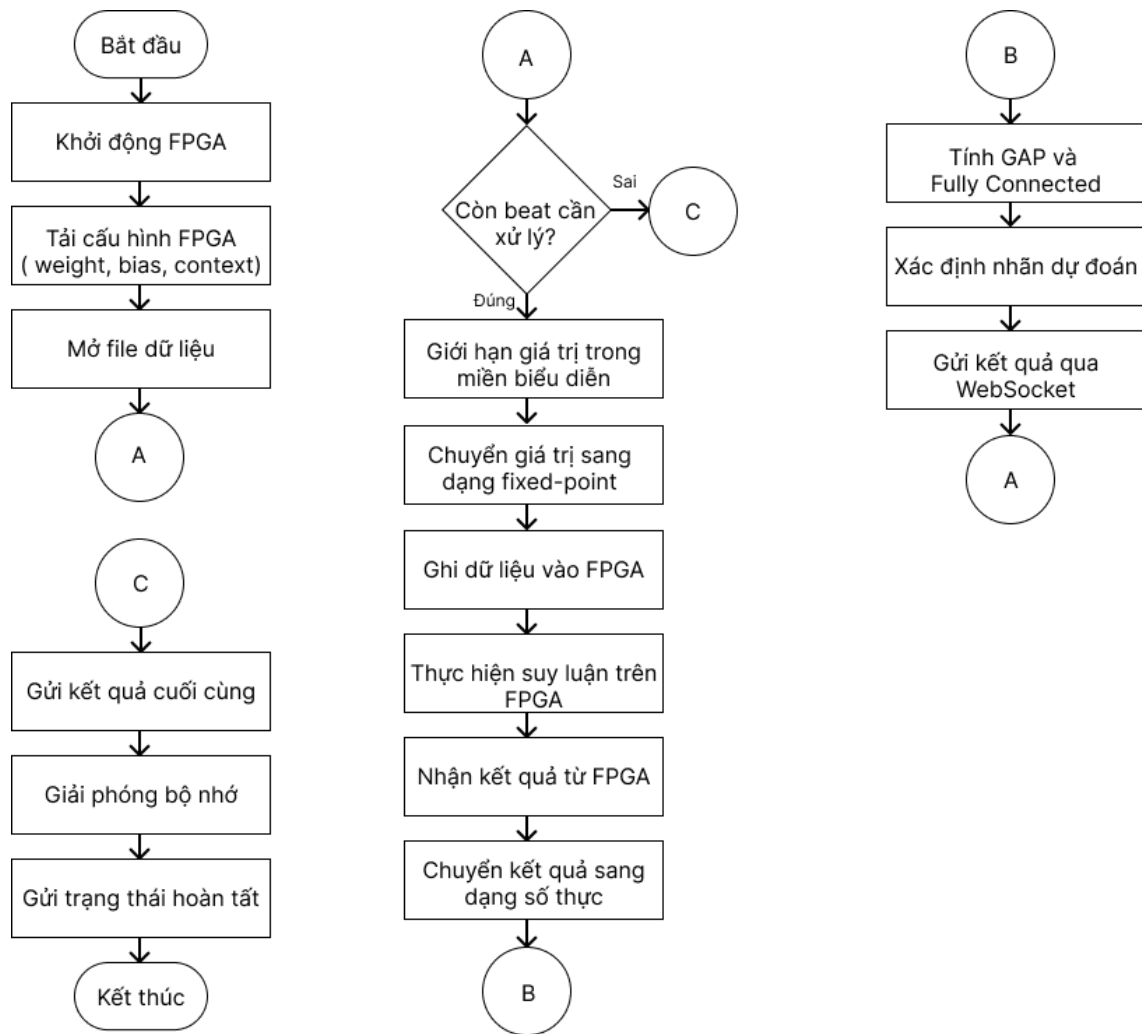


Hình 3.21 Lưu đồ giải thuật hàm `fx_from_float()`

Hình 3.21 mô tả quá trình chuyển đổi giá trị thực sang dữ liệu fixed-point để phục vụ cho việc tính toán trên phần cứng. Khi hàm được gọi, hệ thống nhận vào một giá trị thực cần chuyển đổi, sau đó nhân giá trị này với hệ số tỉ lệ tương ứng với định dạng fixed-point được sử dụng. Với định dạng Q10.6, hệ số tỉ lệ là $(2^6 = 64)$, nhằm chuyển phần thập phân sang dạng số nguyên có thể biểu diễn trên phần cứng. Sau bước nhân tỉ lệ, giá trị được làm tròn. Tiếp theo, hệ thống kiểm tra giới hạn biểu diễn của dữ liệu fixed-point. Nếu giá trị sau chuyển đổi lớn hơn 32767, giá trị sẽ được gán bằng 32767; nếu nhỏ hơn -32767, giá trị sẽ được gán bằng -32767. Cơ chế giới hạn này giúp tránh hiện tượng tràn số trong quá trình biểu diễn. Cuối cùng, hàm trả về dữ liệu fixed-point đã được xử lý, đảm bảo giá trị nằm trong miền biểu diễn hợp lệ trước khi đưa vào hệ thống phần cứng.

3.2.4.12 Hàm fpga_thread()

Nhiệm vụ: Hàm `fpga_thread()` có nhiệm vụ khởi tạo và cấu hình FPGA, nạp dữ liệu và tham số của mô hình CNN, thực hiện quá trình suy luận cho từng beat tín hiệu EEG, gửi dữ liệu và kết quả dự đoán tới client thông qua WebSocket. Chi tiết quá trình thực thi hàm `fpga_thread()` được thể hiện trong lưu đồ Hình 3.22



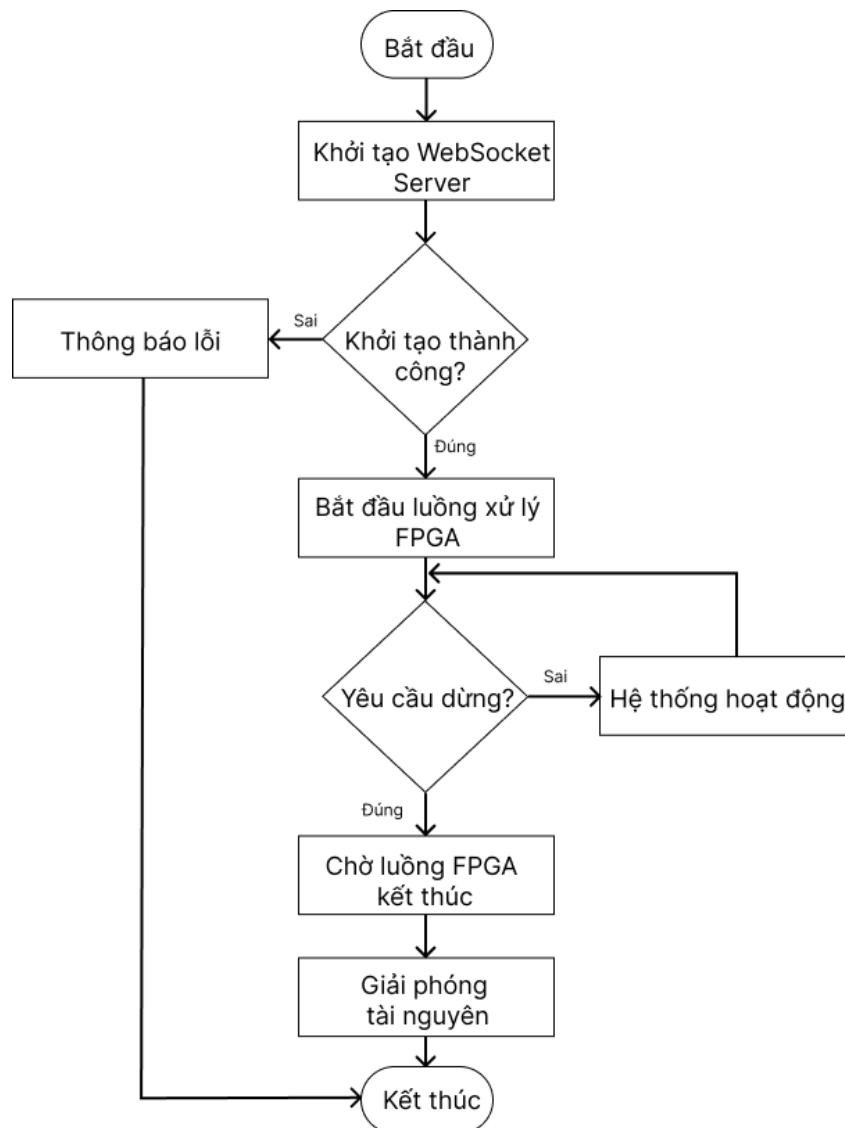
Hình 3.22 Lưu đồ giải thuật hàm `fpga_thread()`

Lưu đồ Hình 3.22 mô tả quá trình xử lý chính của hệ thống khi thực hiện xử lý tín hiệu EEG trên FPGA. Ban đầu, hệ thống tiến hành khởi động và cấu hình FPGA, bao gồm việc nạp các tham số cần thiết như trọng số, bias và context điều khiển vào các vùng nhớ tương ứng. Sau đó, chương trình mở tệp dữ liệu EEG và bắt đầu vòng lặp xử lý từng mẫu tín hiệu. Với mỗi mẫu cần xử lý, dữ liệu đầu vào được giới hạn về miền biểu diễn phù hợp, sau đó chuyển đổi sang dạng fixed-point để tương thích với kiến trúc tính toán trên phần cứng. Dữ liệu sau khi định dạng được ghi vào FPGA. Khi FPGA hoàn tất xử lý, kết quả đầu ra được đọc về phía phần mềm và chuyển đổi lại sang dạng số thực để phục vụ các bước xử lý tiếp theo. Phần mềm tiếp tục thực hiện các lớp còn lại như Global Average Pooling và Fully Connected, sau đó xác định nhãn dự đoán của mẫu hiện tại. Kết quả dự đoán được gửi đến giao diện người dùng thông qua WebSocket để hiển thị theo thời gian

thực. Quá trình này được lặp lại cho đến khi không còn mẫu dữ liệu cần xử lý. Khi toàn bộ dữ liệu đã được xử lý xong, hệ thống gửi kết quả cuối cùng, giải phóng bộ nhớ đã sử dụng và gửi trạng thái hoàn tất trước khi kết thúc chương trình.

3.2.4.13 Hàm main()

Nhiệm vụ: Hàm main() là hàm điều khiển chính của chương trình, có nhiệm vụ khởi tạo WebSocket Server, tạo luồng xử lý FPGA, duy trì hoạt động của WebSocket trong suốt thời gian hệ thống chạy, sau đó giải phóng tài nguyên và kết thúc chương trình khi nhận yêu cầu dừng. Chi tiết về hàm main() được miêu tả theo lưu đồ ở Hình 3.23.



Hình 3.23 Lưu đồ giải thuật của hàm main()

Lưu đồ Hình 3.23 mô tả quá trình hoạt động chính của chương trình điều khiển hệ thống WebSocket kết hợp xử lý FPGA. Khi chương trình bắt đầu, hệ thống tiến hành khởi tạo WebSocket Server để tạo kênh giao tiếp với các client phục vụ việc truyền dữ liệu và hiển thị kết quả. Sau đó, chương trình kiểm tra quá trình khởi tạo có thành công hay không. Nếu khởi tạo thất bại, hệ thống sẽ thông báo lỗi và kết thúc chương trình. Ngược lại, nếu WebSocket Server được khởi tạo thành công, chương trình sẽ tạo và bắt đầu luồng xử lý FPGA. Luồng này chịu trách nhiệm thực hiện các tác vụ liên quan đến nạp dữ liệu, kích hoạt phần cứng, nhận kết quả suy luận và gửi thông tin về phía giao diện. Trong quá trình hoạt động, chương trình liên tục kiểm tra yêu cầu dừng hệ thống. Nếu chưa có yêu cầu dừng, hệ thống tiếp tục duy trì hoạt động và xử lý các sự kiện WebSocket cũng như dữ liệu từ FPGA. Khi có yêu cầu dừng, chương trình sẽ chờ luồng FPGA kết thúc hoàn toàn, sau đó giải phóng các tài nguyên đã sử dụng như bộ nhớ, kết nối và ngữ cảnh WebSocket. Cuối cùng, chương trình kết thúc sau khi toàn bộ tài nguyên được thu hồi an toàn.

3.2.5 Web hiển thị

3.2.5.1 Mục tiêu và vai trò

Giao diện Web được xây dựng nhằm phục vụ quá trình giám sát và trực quan hóa kết quả phân loại tín hiệu EEG trên hệ thống FPGA. Trong hệ thống này, FPGA đảm nhiệm vai trò xử lý và tính toán mô hình CNN, trong khi giao diện Web đóng vai trò là lớp hiển thị trực quan cho người dùng. Thông qua trình duyệt Web, người dùng có thể quan sát tín hiệu EEG đang được truyền lên theo thời gian thực, theo dõi kết quả phân loại của từng đoạn tín hiệu và đánh giá độ chính xác của hệ thống trong quá trình chạy.

Giao diện thể hiện rõ chức năng của hệ thống là phân loại tín hiệu EEG bằng nền tảng FPGA. Các thông tin quan trọng được hiển thị bao gồm dạng sóng EEG, nhãn thật, nhãn dự đoán, độ chính xác hiện tại và nhật ký hoạt động của hệ thống. Nhờ đó, người dùng không cần theo dõi trực tiếp từ terminal hoặc chương trình

backend mà vẫn có thể quan sát toàn bộ quá trình xử lý thông qua một giao diện trực quan.

3.2.5.2 Cơ chế kết nối và truyền dữ liệu qua WebSocket

Để truyền dữ liệu từ backend lên trình duyệt theo thời gian thực, giao diện Web sử dụng cơ chế giao tiếp WebSocket. Đây là cơ chế phù hợp cho các ứng dụng cần cập nhật dữ liệu liên tục vì kết nối giữa trình duyệt và server được duy trì trong suốt quá trình hoạt động. WebSocket cho phép backend chủ động gửi dữ liệu mới lên frontend ngay khi có mẫu tín hiệu hoặc kết quả phân loại.

Trên giao diện, người dùng có thể nhập địa chỉ IP của board FPGA. Trong thiết kế hiện tại, địa chỉ IP mặc định được đặt là 192.168.1.50. Sau khi nhấn nút Kết nối, trình duyệt sẽ tạo kết nối tới backend. Trạng thái kết nối được hiển thị ngay trên giao diện để người dùng dễ dàng theo dõi.

Dữ liệu truyền từ backend lên frontend được tổ chức dưới dạng các bản tin JSON. Mỗi bản tin có trường type để xác định loại dữ liệu và trường data chứa nội dung cần xử lý. Trong giao diện hiện tại, các loại bản tin chính bao gồm status, beat_start, sample, beat_pred và result. Dữ liệu truyền từ backend lên frontend được tổ chức dưới dạng các thông điệp JSON.

3.2.5.3 Giao diện hiển thị tín hiệu EEG và kết quả phân loại

Giao diện Web được bố trí thành các khu vực chính gồm vùng thiết lập kết nối, vùng hiển thị độ chính xác, vùng vẽ tín hiệu EEG, vùng hiển thị nhãn và vùng nhật ký hoạt động. Cách bố trí này giúp người dùng có thể quan sát đầy đủ quá trình xử lý của hệ thống trên một màn hình duy nhất.

Vùng thiết lập kết nối nằm ở phía trên cùng của giao diện, cho phép người dùng nhập địa chỉ IP của FPGA và cổng WebSocket. Bên dưới khu vực này là trạng thái kết nối, giúp người dùng biết hệ thống đang ở trạng thái chờ, đang kết nối hay đã kết nối thành công. Ngay bên dưới phần tiêu đề là thanh hiển thị độ chính xác, trong đó giá trị accuracy được cập nhật theo dạng phần trăm kèm theo số mẫu phân

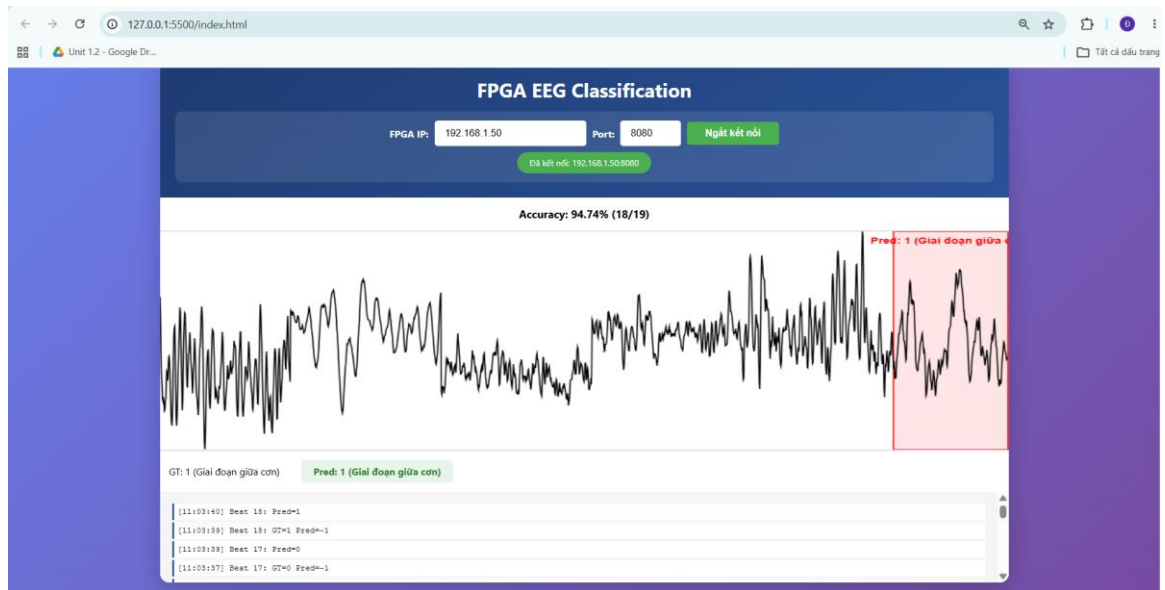
loại đúng trên tổng số mẫu đã xử lý. Ban đầu, khi chưa có dữ liệu, giá trị này được hiển thị là Accuracy: 0.00% (0/0).

Khu vực trung tâm của giao diện là vùng vẽ tín hiệu EEG sử dụng thẻ canvas. Tín hiệu EEG được cập nhật liên tục khi frontend nhận được các mẫu mới từ backend. Để phục vụ hiển thị thời gian thực, chương trình sử dụng bộ đệm vòng để lưu các mẫu tín hiệu trong một cửa sổ quan sát có độ dài 1-2 giây.

Trong quá trình vẽ tín hiệu, chương trình tự động xác định giá trị lớn nhất và nhỏ nhất của dữ liệu hiện có để co giãn trục biên độ cho phù hợp với vùng hiển thị. Nhờ vậy, dạng sóng EEG có thể được quan sát rõ ràng ngay cả khi biên độ tín hiệu thay đổi. Ngoài ra, giao diện còn đánh dấu đoạn tín hiệu hiện tại đang được xử lý bằng một khung màu đỏ. Cơ chế này giúp người dùng nhận biết đoạn dữ liệu nào đang được đưa vào mô hình để suy luận, từ đó dễ dàng liên hệ giữa tín hiệu đầu vào và kết quả phân loại.

Bên dưới vùng đồ thị là khu vực hiển thị nhãn thật và nhãn dự đoán. Nhãn thật được hiển thị tại trường GT, trong khi nhãn do mô hình dự đoán được hiển thị tại trường Pred. Khi backend gửi kết quả dự đoán, giao diện sẽ cập nhật nhãn dự đoán tương ứng. Nếu kết quả dự đoán trùng với nhãn thật, vùng hiển thị dự đoán được đổi sang màu xanh để thể hiện phân loại đúng. Ngược lại, nếu dự đoán sai, vùng này được hiển thị bằng màu đỏ nhằm giúp người dùng dễ dàng nhận biết lỗi phân loại.

Ngoài các vùng hiển thị chính, giao diện còn có cửa sổ nhật ký hoạt động ở phía dưới. Cửa sổ này ghi lại các thông tin quan trọng trong quá trình chạy như trạng thái sẵn sàng, địa chỉ kết nối, kết nối thành công, mất kết nối, lỗi kết nối và kết quả phân loại của từng đoạn tín hiệu. Các thông báo được gán thời gian cụ thể, giúp người dùng dễ dàng theo dõi trình tự hoạt động của hệ thống. Đối với các thông báo lỗi, giao diện sử dụng màu sắc riêng để làm nổi bật, hỗ trợ quá trình kiểm tra và sửa lỗi khi hệ thống hoạt động không đúng mong muốn. Giao diện hiển thị được biểu diễn ở Hình 3.24.



Hình 3.24 Giao diện hiển thị web

Hình 3.24 minh họa giao diện Web được xây dựng cho hệ thống phân loại tín hiệu EEG trên FPGA. Giao diện bao gồm khu vực nhập địa chỉ IP và cổng kết nối, trạng thái WebSocket, vùng hiển thị độ chính xác, vùng vẽ dạng sóng EEG, khu vực hiển thị nhãn thật và nhãn dự đoán, cùng cửa sổ log ghi nhận quá trình hoạt động. Nhờ giao diện này, quá trình giám sát hệ thống FPGA trở nên trực quan hơn, hỗ trợ tốt cho việc kiểm thử, đánh giá và trình diễn kết quả phân loại tín hiệu EEG theo thời gian thực.

CHƯƠNG 4 KẾT QUẢ VÀ ĐÁNH GIÁ

4.1 Giới thiệu nội dung đánh giá

Chương này trình bày quá trình đánh giá và kiểm chứng bộ xử lý 1-D CNN thông qua cả mô phỏng chức năng trên môi trường Vivado và thực thi thực tế trên nền tảng FPGA. Việc đánh giá được thực hiện dựa trên các tiêu chí về tính đúng đắn của thuật toán, khả năng thực hiện chính xác các phép tính và độ tin cậy của kết quả đầu ra. Trên môi trường mô phỏng, hệ thống được kiểm tra nhằm đảm bảo các khối chức năng hoạt động đúng theo đặc tả thiết kế và cho kết quả khớp với giá trị mong đợi. Trên phần cứng FPGA, bộ xử lý được đánh giá thông qua các thử nghiệm thực tế để xác minh khả năng vận hành ổn định cũng như mức độ tương đồng giữa kết quả suy luận của hệ thống và kết quả tham chiếu từ mô hình gốc. Kết quả đánh giá là cơ sở để xác nhận tính chính xác và hiệu quả của kiến trúc được đề xuất.

4.2 Môi trường mô phỏng và phương pháp kiểm tra

4.2.1 Môi trường mô phỏng

Việc mô phỏng được thực hiện trên môi trường sau:

- Công cụ: Xilinx Vivado Design Suite 2022.2
- Simulator: Vivado Simulator (xsim)
- FPGA Target: KV260 Vision AI Starter Kit
- Tần số clock mô phỏng: 100 MHz
- Ngôn ngữ testbench: Verilog HDL

4.2.2 Phương pháp kiểm tra

Đồ án không dùng phương pháp phân tích waveform mà sẽ ghi kết quả nhận được ra thành tệp kết quả (output file) và dùng nó để so sánh với tệp kết quả tham chiếu (expected output file) được ghi ra trên phần mềm nhằm so sánh tín đúng đắn khi thực hiện tính toán trên phần cứng.

*Nguyên lý kiểm tra:

Tạo ra tệp kết quả tham chiếu bằng cách chạy mô hình trên phần mềm bằng ngôn ngữ C++.

Tiến hành chạy mô phỏng trên Vivado và ghi kết quả ngõ ra vào tệp kết quả.

Tiến hành so sánh hai tệp này với nhau, nếu các giá trị đầu ra trùng khớp hoặc sai lệch trong giới hạn cho phép, có thể kết luận rằng bộ xử lý đã thực hiện các phép tính chính xác và hoạt động đúng theo thiết kế.

4.3 Định nghĩa testcase và kết quả mô phỏng

4.3.1 Định nghĩa testcase

*Tính năng kiểm tra: Kiểm tra khả năng tính toán đúng đắn của bộ xử lý 1-D CNN

*Miêu tả testcase: Testcase thực hiện nạp dữ liệu input vào LDM. Sau đó sẽ thực nạp các dữ liệu về trọng số, bias và context vào các bộ nhớ tương ứng là WM, BM, CM kích hoạt hệ thống bằng cách bật tín hiệu start_in. Ghi kết quả nhận được ra tệp kết quả để bắt đầu so sánh.

4.3.2 Kết quả chạy mô phỏng

Sau khi chạy mô phỏng ta thu được tệp kết quả (do output có nhiều giá trị nên nhóm chỉ đưa lên các giá trị đầu và cuối). Kết quả sẽ so sánh vào tệp kết quả tham chiếu ở Bảng 4.1 (do output có nhiều giá trị nên nhóm chỉ đưa lên các giá trị đầu và cuối)

Bảng 4.1 So sánh kết mô phỏng với kết quả tham chiếu

Thứ tự output	Kết quả tệp mô phỏng	Kết quả tệp tham chiếu
1	0024	0025
2	0000	0000
3	0000	0000
4	0000	0000
5	0000	0000
6	0000	0000

7	0000	0000
8	0000	0000
9	0000	0000
10	0000	0000
11	0000	0000
12	0000	0000
13	0000	0000
14	0000	0000
15	0000	0000
⋮	⋮	⋮
1265	00d5	00d5
1266	00e6	00dd
1267	00d4	00ca
1268	00d0	00d4
1269	00c7	00ca
1270	00c6	00c7
1271	00ce	00d2
1272	00cd	00d3
1273	00d0	00cc
1274	00c4	00be
1275	00c3	00b8
1276	00ba	00af
1277	00b7	00a9
1278	00b4	00ad
1279	00a2	0099
1280	0075	006f

Bảng 4.1 cho thấy khi so sánh hai tệp cho ra kết quả khá tương đồng (một số sai số là do quá trình chuyển đổi số từ dạng float sang fixed-point khi đưa lên FPGA). Từ đây có thể thấy khả năng tính toán của hệ thống hoạt động tốt

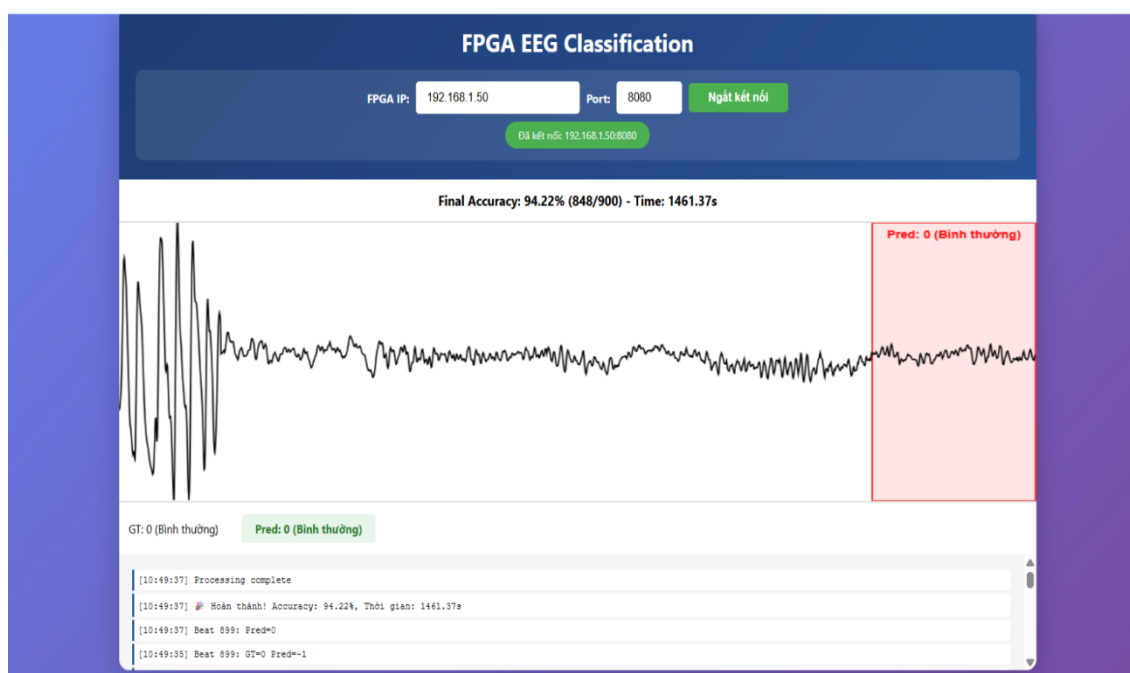
4.4 Chạy thực nghiệm trên FPGA

4.4.1 Phương pháp đánh giá

Phần này trình bày kết quả đánh giá hiệu năng của bộ xử lý 1-D CNN khi được triển khai trên nền tảng FPGA KV260. Việc đánh giá tập trung vào hai chỉ tiêu chính: độ chính xác phân loại trên toàn bộ tập dữ liệu kiểm thử và thời gian xử lý cho mỗi mẫu đầu vào. Độ chính xác được sử dụng để xác minh khả năng duy trì hiệu năng của mô hình sau khi được ánh xạ lên phần cứng, trong khi thời gian xử lý phản ánh tốc độ suy luận và hiệu quả thực thi của kiến trúc được đề xuất. Các kết quả thu được là cơ sở để đánh giá tính khả thi và hiệu quả của hệ thống trong các ứng dụng xử lý tín hiệu thời gian thực.

4.4.2 Kết quả đánh giá

Nhóm thực hiện kiểm thử 900 mẫu tín hiệu EEG và thu được kết quả như Hình 4.1.



Hình 4.1 Kết quả sau khi kiểm thử 900 mẫu EEG

Hình 4.1 cho thấy kết quả thực nghiệm trên nền tảng FPGA KV260 cho thấy hệ thống đạt độ chính xác phân loại 94,22% trên tập dữ liệu kiểm thử. Mức độ chính xác này khá cao, cho thấy kiến trúc phần cứng đã thực hiện đúng các phép tính của

mô hình 1-D CNN và duy trì được hiệu năng của mô hình sau khi triển khai thực tế. Kết quả đạt được cũng chứng minh tính khả thi của giải pháp đề xuất, đồng thời khẳng định khả năng xử lý chính xác và ổn định của bộ xử lý trong quá trình xử lý trên phần cứng.

Chi tiết kết quả thực nghiệm thể hiện dưới dạng confusion matrix ở Bảng 4.1:

Bảng 4.1 Kết quả sau quá trình kiểm thử

Thực tế \ Dự đoán	Healthy	Interictal	Seizure
Healthy	334	26	0
Interictal	17	339	4
Seizure	2	3	175

Theo Bảng 4.1, mô hình phân loại chính xác phần lớn các mẫu thuộc cả ba lớp Healthy, Interictal và Seizure. Các giá trị trên đường chéo chính lần lượt đạt 334, 339 và 175, cho thấy đa số các mẫu được dự đoán đúng. Trong khi đó, các trường hợp phân loại nhầm chủ yếu xảy ra giữa hai lớp Healthy và Interictal, còn lớp Seizure chỉ xuất hiện rất ít sai lệch. Kết quả này cho thấy mô hình có khả năng nhận diện tốt các cơn động kinh và đạt hiệu quả phân loại cao trên tập dữ liệu kiểm tra.

Các thông số về thời gian được thể hiện ở Bảng 4.2

Bảng 4.2 Các thông số về thời gian trong quá trình thực thi

Thông số đánh giá	Kết quả
Tổng thời gian thực thi trên FPGA	0,871 s
Thời gian tính toán trung bình mỗi mẫu	0,968 ms
Thông lượng	1033 mẫu/s

Dựa trên các kết quả trong Bảng 4.2, hệ thống 1-D CNN khi triển khai trên FPGA KV260 cho thấy khả năng xử lý với hiệu năng tương đối cao. Cụ thể, tổng thời gian thực thi cho toàn bộ tập dữ liệu kiểm thử là 0,871 giây, cho thấy bộ gia tốc phần cứng có khả năng hoàn thành quá trình suy luận trong thời gian ngắn. Điều này phản ánh hiệu quả của việc triển khai mô hình trên FPGA, nơi các phép toán được thực hiện song song và được tối ưu hóa ở mức phần cứng.

Thời gian tính toán trung bình cho mỗi mẫu đạt 0,968 ms, tức là nhỏ hơn 1 ms cho một lần suy luận. Kết quả này chứng minh hệ thống có độ trễ thấp, đáp ứng tốt các yêu cầu của các ứng dụng xử lý tín hiệu thời gian thực. Với thời gian xử lý dưới 1 ms, bộ xử lý hoàn toàn có khả năng xử lý liên tục các tín hiệu đầu vào mà không tạo ra hiện tượng tắc nghẽn dữ liệu hoặc làm giảm tốc độ phản hồi của hệ thống.

Bên cạnh đó, hệ thống đạt thông lượng 1033 mẫu/giây, nghĩa là trong một giây bộ xử lý có thể thực hiện suy luận cho hơn một nghìn mẫu dữ liệu. Đây là một chỉ số quan trọng cho thấy khả năng khai thác hiệu quả tài nguyên phần cứng của kiến trúc đề xuất. Mức thông lượng này không chỉ đáp ứng yêu cầu của các ứng dụng phân loại tín hiệu sinh học như ECG theo thời gian thực mà còn tạo điều kiện để mở rộng hệ thống cho các bài toán có khối lượng dữ liệu lớn hơn trong tương lai.

Tổng hợp các kết quả trên cho thấy bộ xử lý 1-D CNN được triển khai trên FPGA KV260 không chỉ đảm bảo độ chính xác của mô hình mà còn đạt được hiệu năng xử lý cao với độ trễ thấp và thông lượng lớn. Điều này khẳng định tính khả thi của kiến trúc đề xuất trong các ứng dụng nhúng thông minh và các hệ thống yêu cầu khả năng suy luận thời gian thực trên nền tảng FPGA.

4.5 Đánh giá tài nguyên phần cứng

4.5.1 Kết quả Implementation

Hình 4.2 là kết quả Utilization Report sau khi chạy Implementation trên Vivado.

Name	CLB LUTs (117120)	CLB Registers (234240)	CARRY8 (14640)	F7 Muxes (58560)	F8 Muxes (29280)	CLB (14640)	LUT as Logic (117120)	LUT as Memory (57600)	GLOBAL CLOCK BUFFERS (352)	PS8 (1)
design_1_wrapper	31892	16330	580	592	128	5511	24356	7536	2	1
design_1_i (design_1)	31892	16330	580	592	128	5511	24356	7536	2	1
axi_smc (design_1_axi_smc_0)	7073	9463	16	0	0	1463	5418	1655	0	0
CGRA_0 (design_1_CGRA_0_0)	24807	6841	564	592	128	4198	18927	5880	0	0
inst (CGRA)	24807	6841	564	592	128	4198	18927	5880	0	0
AXI_Interface (AXI_Interface)	24807	6841	564	592	128	4198	18927	5880	0	0
cnv_core (CNN_1D_Core)	24514	6558	542	592	128	4123	18634	5880	0	0
BRAM (Dual_Port_RAM__paramete	112	16	0	0	0	25	32	80	0	0
controller (Controller)	3022	98	2	5	2	1040	3022	0	0	0
CRAM (Dual_Port_RAM)	3479	32	0	0	0	1396	3439	40	0	0
pe0 (PE_RP)	514	280	27	0	0	161	354	160	0	0
pe1 (PE_RP__parameterized0)	501	280	27	0	0	168	341	160	0	0
pe2 (PE_RP__parameterized1)	515	277	27	0	0	182	355	160	0	0
pe3 (PE_RP__parameterized2)	634	300	27	0	0	212	474	160	0	0
pe4 (PE)	426	275	27	0	0	150	266	160	0	0
pe5 (PE__parameterized0)	427	278	27	0	0	141	267	160	0	0
pe6 (PE__parameterized1)	587	291	27	0	0	212	427	160	0	0
pe7 (PE__parameterized2)	428	282	27	0	0	140	268	160	0	0
pe8 (PE__parameterized3)	435	278	27	0	0	130	275	160	0	0
pe9 (PE__parameterized4)	434	275	27	0	0	147	274	160	0	0
pe10 (PE__parameterized5)	533	291	27	0	0	181	373	160	0	0
pe11 (PE__parameterized6)	427	278	27	0	0	158	267	160	0	0
pe12 (PE__parameterized7)	427	278	27	0	0	147	267	160	0	0
pe13 (PE__parameterized8)	652	298	27	0	0	270	492	160	0	0
pe14 (PE__parameterized9)	427	278	27	0	0	126	267	160	0	0
pe15 (PE__parameterized10)	570	287	27	0	0	204	410	160	0	0
pe16 (PE__parameterized11)	426	276	27	0	0	138	266	160	0	0
pe17 (PE_LP)	525	281	27	0	0	149	365	160	0	0
pe18 (PE_LP__parameterized0)	549	282	27	0	0	159	389	160	0	0
pe19 (PE_LP_Last)	503	283	27	0	0	161	343	160	0	0
sgb_inst (SGB)	4950	669	0	320	0	1476	4950	0	0	0
WRAM (Dual_Port_RAM__paramete	3125	16	0	267	126	504	565	2560	0	0

Hình 4.2 Kết quả Utilization Report sau Implementation

Hình 4.2 trình bày báo cáo Utilization Report sau khi Implementation thiết kế trên FPGA. Kết quả cho thấy thiết kế là cnv_core sử dụng 24807 CLB LUTs (khoảng 21,18% tổng số 117.120 LUTs) và 6841 CLB Registers (khoảng 2,9% tổng số 234.240 thanh ghi) của thiết bị. Ngoài ra, thiết kế sử dụng 564 khối CARRY8, 592 khối F7 MUXSES, 128 khối F8 MUXSES. Kết quả được tóm gọn lại theo Bảng 4.3

Bảng 4.3 Tài nguyên FPGA sau Implementation

Tài nguyên	Đã sử dụng	Khả dụng	Tỷ lệ
CLB LUTs	24807	117120	21,18%
CLB Registers	6841	234240	2,9%
CARRY8	564	14640	3,8%
F7 MUXES	592	58560	1%
F8 MUXES	128	29280	0,4%

Như được tổng hợp trong Bảng 4.3, kiến trúc chỉ sử dụng khoảng 21,18% số lượng CLB LUTs và dưới 4% đối với các tài nguyên còn lại. Điều này cho thấy thiết kế có hiệu quả sử dụng tài nguyên cao, đáp ứng yêu cầu triển khai trên FPGA Kria KV260 mà vẫn còn dư địa để mở rộng hoặc bổ sung các khối xử lý khác.

Bảng 4.4 thể hiện tài nguyên được phân bổ qua từng khối

Bảng 4.4 Phân bổ tài nguyên theo từng khối

Khối	CLB LUTs	CLB Registers	CARRY8	F7 MUXES	F8 MUXES
Cnn_core (top module)	24514	6558	542	592	128
Controller	3022	98	2	5	2
SBA	4950	669	0	320	0
20xPE	9940	5650	540	0	0
CM	3479	32	0	0	0
WM	3125	16	0	267	126
BM	112	16	0	0	0

Bảng 4.4 ở trên trình bày chi tiết việc phân bổ tài nguyên theo từng khối.

Phân tích tổng quan:

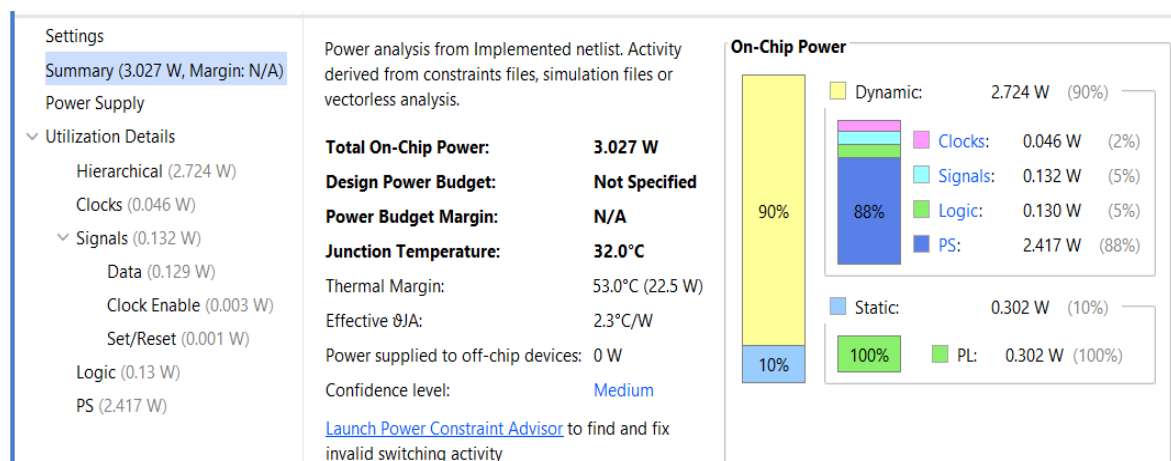
- CLB LUTs sử dụng 24514/117170 LUT, tương đương 20,9% tổng tài nguyên. Đây là loại tài nguyên được sử dụng nhiều nhất do các phép toán điều khiển, định tuyến dữ liệu và xử lý logic trong mạng CNN chủ yếu được ánh xạ lên LUT.
- CLB Registers sử dụng 6558/234240 register, tương đương 2,8%.
- CARRY8 sử dụng 542/14640 khối, tương đương 3,7%. Các khối này chủ yếu phục vụ các phép toán cộng, tích lũy và sinh địa chỉ trong bộ xử lý CNN.
- F7 MUXES và F8 MUXES lần lượt sử dụng 1,0% và 0,4%, cho thấy số lượng hàm logic lớn cần ghép nhiều LUT không quá lớn.

Phân bổ theo từng khối:

- 20×PE sử dụng 9941 CLB LUTs, 5646 CLB Registers và 540 CARRY8, là khối chiếm nhiều tài nguyên nhất trong hệ thống do đảm nhiệm các phép toán tích chập và lưu trữ của mạng CNN.
- SBA sử dụng 4951 CLB LUTs, 669 CLB Registers và 329 F7 MUXES. Tài nguyên của khối này chủ yếu phục vụ cho việc điều phối và phân phối dữ liệu giữa các phần tử xử lý.
- CM sử dụng 3483 CLB LUTs và 32 CLB Registers. Đây là khối lưu trữ thông tin cấu hình của các lớp mạng và các tham số điều khiển trong quá trình thực thi.
- WM sử dụng 3125 CLB LUTs, 16 CLB Registers, 267 F7 MUXES và 126 F8 MUXES. Khối này chịu trách nhiệm lưu trữ và cung cấp trọng số cho các PE trong quá trình suy luận.
- Controller sử dụng 3024 CLB LUTs, 98 CLB Registers, 2 CARRY8, 5 F7 MUXES và 2 F8 MUXES. Tài nguyên được sử dụng chủ yếu cho việc quản lý trạng thái và điều khiển hoạt động của toàn hệ thống.
- BM sử dụng 113 CLB LUTs và 16 CLB Registers, là khối có mức sử dụng tài nguyên thấp nhất do chỉ đảm nhiệm việc lưu trữ các giá trị bias của mô hình.

4.5.2 Kết quả Power Analysis

Hình 4.3 thể hiện kết quả Power Analysis sau Implementation



Hình 4.3 Kết quả Power Report sau Implementation

Hình 4.3 thể hiện kết quả Power Analysis sau Implementation

Bảng 4.5 và Bảng 4.6 tổng hợp kết quả ước tính công suất từ Vivado Power Report

Bảng 4.5 Các thành phần công suất

Thành phần	Công suất	Tỷ lệ
Total On-Chip Power	3,027 W	100%
Dynamic Power	2,724 W	90%
Static Power	0,302 W	10%

Bảng 4.6 Phân bổ trong Dynamic Power

Thành phần	Công suất	Tỷ lệ
Clocks	0,046 W	2%
Signals	0,132 W	5%
Logic	0,130 W	5%
PS	2,417 W	88%

Điều kiện đo:

- Junction Temperature: 32 °C
- Ambient Temperature: 25.0°C
- Thermal Margin: 53 °C (22,5 W)

Phân tích tổng quan:

- Total On-Chip Power đạt 3,027 W, cho thấy hệ thống có mức tiêu thụ năng lượng tương đối thấp khi triển khai trên FPGA KV260.
- Dynamic Power đạt 2,724 W, chiếm 90% tổng công suất tiêu thụ. Đây là thành phần công suất phát sinh do hoạt động chuyển mạch của các tín hiệu, khối logic và các phần tử chức năng trong quá trình tính toán.

- Static Power đạt 0,302 W, chiếm 10% tổng công suất tiêu thụ. Thành phần này chủ yếu đến từ dòng rò của các phân tử bán dẫn trên FPGA và tồn tại ngay cả khi hệ thống không thực hiện tính toán.

Phân bổ trong Dynamic Power:

- PS (Processing System) tiêu thụ 2,417 W, chiếm 88% Dynamic Power, là thành phần tiêu thụ năng lượng lớn nhất trong hệ thống. Nguyên nhân là PS đảm nhiệm việc thực thi hệ điều hành Linux, quản lý bộ nhớ DDR, điều khiển luồng dữ liệu và giao tiếp với bộ gia tốc phần cứng.
- Signals tiêu thụ 0,132 W, chiếm 5% Dynamic Power. Công suất này phát sinh từ hoạt động truyền dữ liệu giữa các khối chức năng trong hệ thống thông qua các kết nối nội bộ.
- Logic tiêu thụ 0,130 W, chiếm 5% Dynamic Power. Thành phần này phản ánh mức năng lượng tiêu thụ của các khối xử lý được triển khai trên vùng Programmable Logic (PL), bao gồm PE, SBA, Controller và các khối bộ nhớ.
- Clocks tiêu thụ 0,046 W, chiếm 2% Dynamic Power. Kết quả này cho thấy mạng phân phối xung nhịp được thiết kế hiệu quả và không tạo ra mức tiêu thụ công suất đáng kể.

Kết luận:

Nhìn chung, phần lớn công suất động của hệ thống đến từ khối PS, trong khi vùng PL chứa bộ xử lý 1-D CNN chỉ chiếm một tỷ lệ nhỏ. Điều này cho thấy bộ gia tốc CNN được thiết kế hiệu quả về mặt năng lượng, đồng thời khẳng định khả năng triển khai các ứng dụng suy luận trên FPGA với mức tiêu thụ công suất thấp.

4.6 So sánh với các nghiên cứu liên quan

Kết quả so sánh giữa các nghiên cứu được trình bày như Bảng 4.7

Bảng 4.7 So sánh giữa các nghiên cứu

Tiêu chí	[3]	[4]	[5]	[6]	Đề tài này
Tập dữ liệu	Bonn EEG	EEG (Open BCI)	Bonn EEG	ECG, EEG, EMG	Bonn EEG
Mô hình	CNN- LSTM	CWT- CNN	Hjorth + KNN	1-D CNN	1-D CNN (dựa trên ResNet)
Tổng tham số	10609	52288	-	~3,8 triệu	7243
Độ chính xác	99.33%	91,65%	90,74%	99%	94,22%
Triển khai FPGA	Zynq-7000	Zynq-7000	Zynq-7000	Zynq-7000	Zynq UltraScale+
LUTs	3653	9334	4699	1067	24514
FFs	3128	8947	9496	-	6558
BRAM	37	264	0	50	0

Bảng 4.7 trình bày sự so sánh giữa đề tài thực hiện với một số nghiên cứu liên quan dựa trên các tiêu chí gồm tập dữ liệu, mô hình phân loại, tổng số tham số, độ chính xác, nền tảng triển khai FPGA và mức sử dụng tài nguyên phần cứng. Thông qua bảng so sánh có thể thấy rằng đề tài đạt được sự cân bằng tốt giữa độ chính xác phân loại, độ phức tạp mô hình và khả năng triển khai trên phần cứng FPGA.

Một điểm nổi bật của đề tài là số lượng tham số của mô hình thấp. Mô hình đề xuất chỉ có 7243 tham số, thấp hơn so với Hybrid CNN-LSTM ở nghiên cứu [3] với 10609 tham số và thấp hơn rất nhiều so với nghiên cứu [4] với 52288 tham số. Đặc biệt, so với mô hình 1-D CNN trong nghiên cứu [6] có khoảng 3,8 triệu tham số, mô hình của đề tài nhỏ hơn khoảng hơn 500 lần. Điều này cho thấy kiến trúc

đề xuất có độ phức tạp thấp hơn đáng kể, giúp giảm dung lượng lưu trữ trọng số, giảm số phép toán cần thực hiện và thuận lợi hơn khi ánh xạ lên FPGA. Với số tham số nhỏ, mô hình cũng phù hợp hơn với các hệ thống tài nguyên hạn chế, đồng thời làm giảm yêu cầu về bộ nhớ và băng thông truy xuất dữ liệu trong quá trình suy luận.

Xét về độ chính xác, đề tài đạt 94,22%. Kết quả này cao hơn nghiên cứu [4] sử dụng CWT-CNN với độ chính xác 91,65% và cao hơn nghiên cứu [5] sử dụng Hjorth + KNN với độ chính xác 90,74%. Điều này cho thấy mô hình 1-D CNN của đề tài có khả năng học đặc trưng tốt hơn so với phương pháp dựa trên biến đổi CWT-CNN trong nghiên cứu [4] và phương pháp trích xuất đặc trưng thủ công trong nghiên cứu [5]. Mặc dù độ chính xác của đề tài thấp hơn nghiên cứu [3] và [6], cần xét đến sự khác biệt về độ phức tạp mô hình và dữ liệu đầu vào. Nghiên cứu [3] sử dụng CNN-LSTM, có khả năng học đặc trưng thời gian mạnh hơn nhưng cũng phức tạp hơn. Nghiên cứu [6] đạt độ chính xác 99% nhưng mô hình có tới khoảng 3,8 triệu tham số. Do đó, với việc chỉ sử dụng tín hiệu EEG và mô hình có 7243 tham số, độ chính xác 94,22% là kết quả cạnh tranh và thể hiện hiệu quả của kiến trúc đề xuất.

Về tài nguyên phần cứng, thiết kế của đề tài sử dụng 24.514 LUT và 6.558 FF. Số LUT cao hơn một số nghiên cứu khác, tuy nhiên điều này có thể giải thích do kiến trúc đề tài thực hiện nhiều phép toán CNN trực tiếp trên phần cứng, trong khi một số nghiên cứu sử dụng mô hình đơn giản hơn hoặc chỉ triển khai các khối tính toán đặc trưng. Điểm nổi bật nhất là thiết kế của đề tài không sử dụng BRAM, trong khi các nghiên cứu [3], [4] và [6] lần lượt sử dụng 37, 264 và 50 BRAM. Việc không sử dụng BRAM cho thấy kiến trúc bộ nhớ của đề tài được tối ưu theo hướng dùng tài nguyên logic hoặc distributed memory, giúp giải phóng tài nguyên BRAM cho các khối chức năng khác trong hệ thống. Đây là ưu điểm đáng chú ý khi triển khai trên FPGA, vì BRAM là tài nguyên quan trọng và thường bị giới hạn trong các thiết kế xử lý dữ liệu lớn.

Từ các phân tích trên có thể thấy, ưu điểm chính của đề tài không nằm ở việc đạt độ chính xác cao nhất tuyệt đối, mà nằm ở sự cân bằng giữa độ chính xác, độ

gọn của mô hình và khả năng triển khai phần cứng. Mô hình đề xuất có số tham số nhỏ nhất trong nhóm các mô hình học sâu được so sánh, không yêu cầu biến đổi thời gian-tần số, không cần trích xuất đặc trưng thủ công, không sử dụng BRAM và vẫn đạt độ chính xác cao hơn một số nghiên cứu EEG trước đó. Điều này chứng minh rằng kiến trúc 1-D CNN dựa trên ResNet trong đề tài là một giải pháp phù hợp cho bài toán phân loại tín hiệu EEG trên FPGA, đặc biệt trong các ứng dụng yêu cầu xử lý thời gian thực, mô hình nhỏ gọn và khả năng triển khai trên thiết bị nhúng.

CHƯƠNG 5 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết luận

Nhóm đã nghiên cứu và phát triển thành công đề tài “Thiết kế và triển khai bộ xử lý 1-D CNN trên FPGA cho phân loại điện não EEG” với độ chính xác và hiệu năng cao. Sau khi hoàn thành nhóm nhận thấy hệ thống có những ưu điểm và khuyết điểm sau:

Ưu điểm:

- Mô hình CNN có số lượng tham số thấp, giúp giảm đáng kể yêu cầu về bộ nhớ lưu trữ trọng số và tài nguyên tính toán. Nhờ đó, mô hình phù hợp để triển khai trên các thiết bị nhúng hoặc thiết bị đeo có năng lực xử lý và dung lượng bộ nhớ hạn chế, đồng thời giảm mức tiêu thụ năng lượng trong quá trình vận hành.
- Kiến trúc phần cứng được thiết kế theo hướng tái cấu hình và linh hoạt, cho phép hỗ trợ nhiều mô hình CNN khác nhau thông qua việc thay đổi các tham số cấu hình như kích thước kernel, số lượng kênh đầu vào/đầu ra hoặc số lớp xử lý. Điều này giúp giảm thời gian phát triển hệ thống và hạn chế việc phải sửa đổi hoặc thiết kế lại logic phần cứng khi triển khai các mô hình mới.
- Kích thước phần cứng nhỏ gọn, sử dụng hiệu quả các tài nguyên FPGA như LUT, Register. Mức sử dụng tài nguyên thấp tạo điều kiện tích hợp hệ thống trên các nền tảng FPGA có quy mô vừa và nhỏ, đồng thời vẫn còn dư tài nguyên cho các chức năng mở rộng khác của hệ thống.
- Độ chính xác phân loại cao, đảm bảo khả năng nhận diện và phân loại tín hiệu một cách đáng tin cậy. Mô hình có khả năng tự động trích xuất đặc trưng quan trọng từ dữ liệu đầu vào, giúp nâng cao hiệu suất phân loại và duy trì độ ổn định khi làm việc với các tập dữ liệu thực tế có nhiễu hoặc biến thiên lớn.

Nhược điểm:

- Kiến trúc phần cứng hiện tại chưa hỗ trợ các lớp Global Average Pooling (GAP), Dense và Fully Connected, do đó phạm vi các mô hình CNN có thể triển khai còn hạn chế. Một số kiến trúc mạng phải tính toán các lớp này trên PL.
- Chức năng giám sát và hiển thị dữ liệu trên giao diện web phụ thuộc vào kết nối Wi-Fi, vì vậy khả năng truy cập và theo dõi từ xa sẽ bị ảnh hưởng khi mạng không ổn định hoặc không có kết nối Internet.
- Hệ thống chưa tích hợp các khối tiền xử lý và xử lý nhiễu tín hiệu, do đó chất lượng dữ liệu đầu vào có thể bị ảnh hưởng bởi nhiễu từ môi trường hoặc quá trình thu nhận tín hiệu, làm suy giảm hiệu quả phân loại trong một số trường hợp thực tế.
- Mô hình được đánh giá trên tập dữ liệu Bonn với tín hiệu EEG đơn kênh, trong khi các hệ thống EEG thực tế thường sử dụng nhiều kênh để khai thác đầy đủ thông tin không gian của hoạt động não bộ. Vì vậy, mức độ phức tạp của bài toán còn tương đối thấp và kết quả đạt được có thể chưa phản ánh đầy đủ hiệu năng của hệ thống trong các ứng dụng thực tế đa kênh.

5.2 Hướng phát triển

Đề tài đã đạt được các thành công nhất định nhưng vẫn có thể phát triển thêm để có thể ứng dụng trong thực tế. Một số hướng phát triển đề tài này có thể kể đến như sau:

- Áp dụng các kỹ thuật tối ưu mô hình như Weight Pruning nhằm loại bỏ các trọng số ít quan trọng trong mạng CNN, từ đó giảm số lượng phép tính, tiết kiệm tài nguyên phần cứng và giảm mức tiêu thụ năng lượng mà vẫn duy trì độ chính xác phân loại ở mức chấp nhận được.
- Mở rộng và đánh giá hệ thống trên các tập dữ liệu EEG đa kênh, giúp khai thác thêm thông tin không gian của tín hiệu não bộ. Điều này cho

phép kiểm chứng khả năng mở rộng của kiến trúc phần cứng đối với các bài toán thực tế có độ phức tạp cao hơn.

- Tích hợp các khối tiền xử lý và xử lý nhiều tín hiệu, chẳng hạn như các bộ lọc số hoặc các phương pháp loại bỏ nhiễu đặc trưng của tín hiệu sinh học. Việc này giúp nâng cao chất lượng dữ liệu đầu vào và cải thiện độ tin cậy của kết quả phân loại.
- Bổ sung hỗ trợ cho các lớp Global Average Pooling (GAP), Dense và Fully Connected trong kiến trúc phần cứng, nhằm mở rộng khả năng triển khai nhiều mô hình CNN khác nhau và tăng tính linh hoạt của bộ gia tốc khi làm việc với các kiến trúc mạng hiện đại.

Những hướng phát triển trên có thể giúp hệ thống toàn diện hơn, có thể áp dụng vào các ứng dụng những thực tế.

Tài liệu tham khảo

- [1]. X. Wang, X. Wang, W. Liu, Z. Chang, T. Kärkkäinen, F. Cong . “One dimensional convolutional neural networks for seizure onset detection using long-term scalp and intracranial EEG”. *Neurocomputing*, Vol. 459, 2021, pp. 212-222
- [2]. "Bonn University EEG Dataset Overview," EmergentMind. [Online]. Available: <https://www.emergentmind.com/topics/bonn-university-eeg-dataset>. [Accessed: Jun. 17, 2026].
- [3]. N. Akhtar, A. R. Buzdar, and J. Fan, "CNN–LSTM-based EEG epileptic seizure detection: A hardware–software co-design approach," *Int. J. Adv. Appl. Sci.*, vol. 13, no. 3, pp. 74–85, 2026, doi: 10.21833/ijaas.2026.03.008.
- [4]. Y. Xie, T. Majoros, and S. Oniga, "FPGA-based hardware accelerator on portable equipment for EEG signal patterns recognition," *Electronics*, vol. 11, no. 15, p. 2410, Aug. 2022, doi: 10.3390/electronics11152410.
- [5]. A. Rizal, S. Hadiyoso, and A. Z. Ramdani, "FPGA-based implementation for real-time epileptic EEG classification using Hjorth descriptor and KNN," *Electronics*, vol. 11, no. 19, p. 3026, Sep. 2022, doi: 10.3390/electronics11193026.
- [6]. K. Jameil and H. Al-Raweshidy, "Efficient CNN architecture on FPGA using high level module for healthcare devices," *IEEE Access*, vol. 10, pp. 60486–60495, 2022, doi: 10.1109/ACCESS.2022.3180829
- [7]. H. L. Pham, T. D. Tran, V. T. D. Le, and Y. Nakashima. “MINA: A Hardware-Efficient and Flexible Mini-InceptionNet Accelerator for ECG Classification in Wearable Devices.” *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 72, 2025, pp. 2740-2753.
- [8]. A. F. Jaramillo Rueda, L. Y. Vargas Pacheco, and C. A. Fajardo, "A Computational Architecture for Inference of a Quantized-CNN for Detecting Atrial Fibrillation," *Ingeniería y Ciencia*, vol. 16, no. 32, pp. 135-149, Jul.-Dec. 2020.
- [9]. Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.
- [10]. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016
- [11]. O'Shea and R. Nash, “An introduction to convolutional neural networks,” arXiv preprint arXiv:1511.08458, 2015.
- [12]. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, Jun. 2016, pp. 770–778.
- [13]. G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, Honolulu, HI, USA, Jul. 2017, pp. 4700–4708.

- [14]. Szegedy, W. Liu, Y. Jia et al., "Going deeper with convolutions," in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), Boston, MA, USA, Jun. 2015, pp. 1–9
- [15]. H. L. Pham, T. D. Tran, V. T. D. Le, and Y. Nakashima. "MINA: A Hardware-Efficient and Flexible Mini-InceptionNet Accelerator for ECG Classification in Wearable Devices." IEEE Transactions on Circuits and Systems I: Regular Papers, vol. 72, 2025, pp. 2740-2753.
- [16]. T. T. Điềm, P. H. Luân, and N. V. Tình, Thiết kế SoC. TP. Hồ Chí Minh, Việt Nam: Nhà xuất bản Đại học Quốc gia Thành phố Hồ Chí Minh, 2024, ch. 4, tr. 28-36.
- [17]. A. Vina, "What is ResNet-50 and what is its relevance in computer vision?," Ultralytics Blog, May 27, 2025. [Online]. Available: <https://www.ultralytics.com/blog/what-is-resnet-50-and-what-is-its-relevance-in-computer-vision>. [Accessed: Jun. 24, 2026].